UNIVERSIDADE PAULISTA
JULIA CRISTINA SILVA DE SOUZA
LETICIA OLIVEIRA DE JESUS

CYBER CHASE: Desenvolvimento de uma Plataforma Web Educacional para Mitigação de Ataques Cibernéticos através da Conscientização Digital.

SÃO PAULO
2026

UNIVERSIDADE PAULISTA
JULIA CRISTINA SILVA DE SOUZA
LETICIA OLIVEIRA DE JESUS

CYBER CHASE: Desenvolvimento de uma Plataforma Web Educacional para Mitigação de Ataques Cibernéticos através da Conscientização Digital.

Trabalho de Conclusão de Curso apresentado ao Curso de Sistemas de Informação da Universidade Paulista, como requisito parcial para obtenção do título de Bacharel em Sistemas de Informação.

SÃO PAULO
2026

Dedicamos este trabalho à Isabella Santos, cunhada de Leticia, que sofreu um golpe digital recentemente. Isabella adentrou um site falso de uma loja de calçados conhecida, e perdeu o seu dinheiro em uma compra falsa. Isabella relatou a situação, de modo a encorajar o desenvolvimento da Cyber Chase, por ela e por outras pessoas que poderiam sofrer com situações afins.

Ao Pedro Santos, namorado de Leticia, que a motivou a persistir na entrega deste trabalho, mesmo em meio a momentos difíceis de saúde e tratamento.

RESUMO

Com a digitalização avançada no atual cenário global e o aumento das interações em rede que acabaram elevando as ocorrência de crimes cibernéticos, expondo a vulnerabilidade de usuários diante de fraudes cada vez mais sofisticadas, o presente trabalho propôs a criação do Cyber Chase, uma plataforma de educação cibernética que tem como objetivo virar ferramenta estratégica para a mitigação de ataques digitais por meio da da conscientização digital utilizando Inteligência Artificial e Machine Learning, cuja proposta se baseaia na premissa de que o "fator humano" é o elo mais crítico da segurança da informação, demandando métodos de ensino mais dinâmicos e tecnológicos do que os tradicionais que acompanhem as novas formas de se cair em um ataque cibernético e pensando nisso nasce a Cyber Chase, que consiste em um ambiente interativo onde o usuário navega por conteúdos que abordam desde a identificação de engenharia social e sites falsos até práticas seguras de navegação. A metodologia de desenvolvimento foca na estruturação de uma interface intuitiva e na curadoria de conteúdos que abordam conceitos de segurança. Com a implementação desta proposta, busca-se oferecer uma solução pedagógica intuitiva que molde a postura de seus usuários de reativa para proativa.

Palavras-chave: Cibersegurança. Educação Digital. Plataforma Web. Mitigação de Ataques. Conscientização. Inteligência Artificial. Machine Learning.

ABSTRACT

With the advanced digitization in the current global scenario and the increase in network interactions that have led to a rise in cybercrime, exposing users' vulnerability to increasingly sophisticated fraud, this work proposes the creation of Cyber Chase, a cybersecurity education platform aimed at becoming a strategic tool for mitigating digital attacks through digital awareness using Artificial Intelligence and Machine Learning. The proposal is based on the premise that the "human factor" is the most critical link in information security, demanding more dynamic and technological teaching methods than traditional ones, methods that keep pace with the new ways of falling victim to a cyberattack. With this in mind, Cyber Chase was created, consisting of an interactive environment where users navigate content covering topics ranging from identifying social engineering and fake websites to safe browsing practices. The development methodology focuses on structuring an intuitive interface and curating content that addresses security concepts. With the implementation of this proposal, the aim is to offer an intuitive pedagogical solution that shapes the posture of its users from reactive to proactive.

Keywords: Cybersecurity. Digital Education. Web Platform. Attack Mitigation. Awareness. Artificial Intelligence. Machine Learning.

SUMÁRIO

INTRODUÇÃO **8**
2 DESENVOLVIMENTO **12**
2.1 Metodologias 12
2.2.1 Requisitos Funcionais 14
2.2.2 Requisitos Não Funcionais 17
2.2.3 Requisitos Implícitos 20
2.3 Modelagem do Sistema 20
2.3.1 Diagrama de Caso de Uso 20
2.3.2 Diagrama de Classe 23
2.3.3 Diagrama de Sequência 25
2.3.4 Diagrama de Componentes 29
2.3.5 Diagrama de Atividades 30
2.3.6 Diagrama de Estado 33
2.3.7 Diagrama de Arquitetura 35
2.4 Desenvolvimento Back-end 37
2.4.1 Framework Flask 38
2.4.2 Rotas 39
2.4.3 Autenticação 40
2.4.4 CRUD (Create, Read, Update, Delete) 41
2.4.5 Banco de dados 43
2.5 Inteligência Artificial e Machine Learning 46
2.5.1 Aplicação de Inteligência Artificial na Validação de Respostas dos Cursos 47
2.5.2 Aplicação de Inteligência Artificial no Chatbot 48
2.7 Desenvolvimento Front-End 49
2.7.1 Interface de login 52

2.7.2 Interface do Administrador 54
2.7.3 Interface Sobre Nós 56
2.7.4 Interface de Contato 57
2.7.5 Interface da home 59
2.7.6 Interface de Cursos 61
2.7.7 Página dos cursos 64
2.8 Conteúdo dos cursos 68
2.8.1 Ataques Cibernéticos 68
2.8.2 Engenharia Social 73
2.8.3 Phishing 75
2.8.4 Vírus 77
2.8.5 Segurança em Dispositivos Móveis 80
2.8.6 Forense Digital e Respostas a Incidentes 82
3 RESULTADOS 86
3.1 Testes de funcionalidades 86
3.2 Análise dos resultados obtidos 96
CONCLUSÃO 97
REFERÊNCIAS BIBLIOGRÁFICAS 98

INTRODUÇÃO

Com o avanço da tecnologia e a transformação digital, houve uma transformação na forma como a sociedade se comunica, trabalha, realiza transações financeiras e acessa serviços essenciais devido a essas facilidades e à globalização, onde o crescimento do uso da internet, junto à expansão de dispositivos móveis e à digitalização de processos empresariais e governamentais trouxe benefícios para a população. Porém, apesar do avanço da tecnologia, observou-se também um crescimento expressivo das ameaças cibernéticas, o que fez com que a segurança digital se tornasse uma das principais preocupações da atualidade.

Nos últimos anos, os ataques cibernéticos passaram a atingir indivíduos, empresas e órgãos públicos em escala mundial, de modo a gerar impactos financeiros, sociais e institucionais significativos. De acordo com um levantamento divulgado pela IBSEC, os ataques cibernéticos globais apresentaram aumento de 44% em 2025, evidenciando um cenário de crescimento acelerado das ameaças digitais e do fortalecimento das organizações criminosas voltadas para crimes virtuais (IBSEC, 2025). Isso demonstra que o desenvolvimento tecnológico não foi acompanhado, na mesma proporção, por investimentos em conscientização digital e educação voltada à segurança da informação.

Ademais, relatórios internacionais demonstram que os criminosos utilizam métodos cada vez mais robustos para explorar vulnerabilidades humanas e tecnológicas. O relatório “2026 Global Threat Report”, desenvolvido pela CrowdStrike, destaca que ataques baseados em engenharia social, roubo de credenciais e invasões por meio de phishing continuam entre as principais ameaças globais, por conta da dificuldade dos usuários em identificar tentativas de fraude digital (CROWDSTRIKE, 2026). Isto posto, é perceptível que a vulnerabilidade humana tem sido cada vez mais explorada pelos criminosos no ambiente web.

No Brasil, o cenário é alarmante, onde lidera os índices de ataques cibernéticos na América Latina, sendo um dos maiores alvos de criminosos digitais. De acordo com reportagem divulgada pela CNN Brasil, o Brasil ocupa posição de destaque negativo em relação ao número de ataques cibernéticos registrados na região latino-americana (CNN Brasil, 2025). Isto é agravado pela ampla utilização de meios digitais para

serviços bancários, compras online e comunicação pessoal, fatores que abrem margem para a disponibilidade de ataques para cibercriminosos.

Nota-se, também, um crescimento dos golpes virtuais contra os brasileiros. Segundo uma reportagem publicada pela CNN Brasil, os golpes digitais cresceram significativamente em 2025 e passaram a atingir usuários de diferentes faixas etárias e níveis sociais (CNN BRASIL, 2025). Entre os principais golpes identificados estão fraudes bancárias, clonagem de aplicativos de mensagens, phishing, roubo de identidade digital e golpes envolvendo engenharia social, explorando a falta de conhecimento técnico dos usuários. Isto torna a conscientização digital um elemento importante para a prevenção de incidentes cibernéticos.

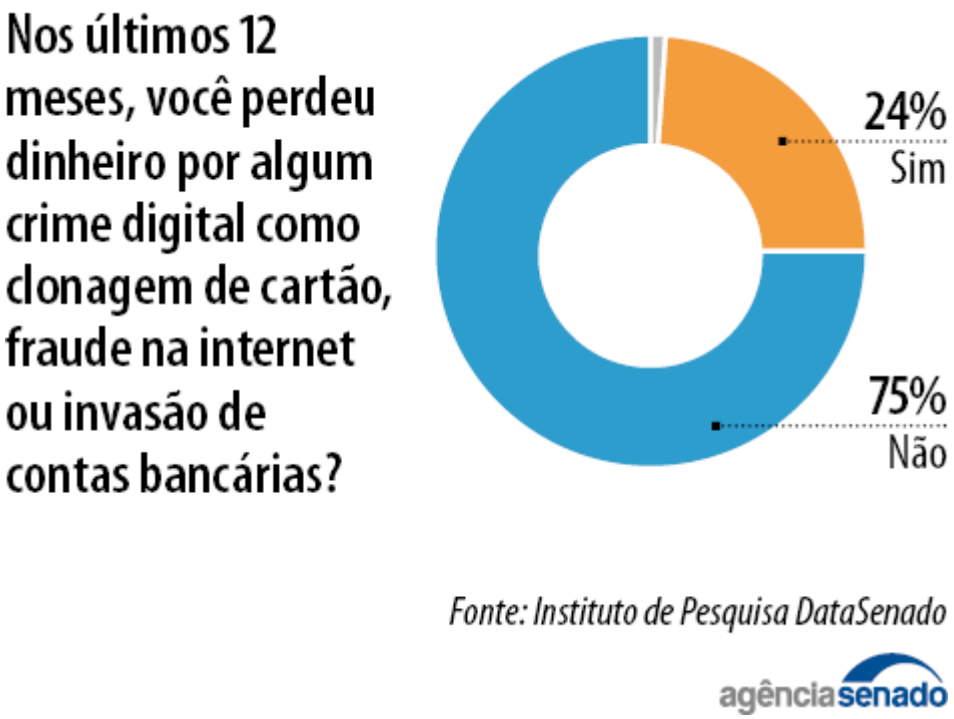
Nesse contexto, é notório que a vulnerabilidade se encontra nos sistemas computacionais, e sobretudo na falta de preparo dos usuários para lidar com as ameaças digitais. Muitos ataques poderiam ser evitados por meio de práticas simples de segurança e educação digital adequada, sendo necessária a conscientização cibernética como uma estratégia para reduzir riscos e fortalecer a segurança dos usuários no ambiente virtual.

As investigações policiais também registraram dados de aumento de crimes digitais. Conforme levantamento divulgado sobre a atuação da Polícia Federal, o número de pessoas indiciadas por crimes cibernéticos aumentou 221% em apenas dois anos, atingindo uma média de cinco indiciamentos por dia (Polícia Federal, 2025).

Além disso, dados divulgados pelo Instituto Brasileiro de Direito de Família (IBDFAM) apontam crescimento significativo nas denúncias relacionadas a crimes cibernéticos no Brasil (IBDFAM, 2025), com ocorrências envolvendo vazamento de dados pessoais, invasão de contas, exposição indevida de informações e golpes financeiros. Isso demonstra que os impactos da criminalidade digital afetam não apenas o setor econômico, mas também aspectos sociais e psicológicos da população, sendo necessária atenção.

Em 2024, o Instituto DataSenado entrevistou 22 mil pessoas com o objetivo de colher resultados por amostragem para ilustrar a situação de aumento de casos de ataques cibernéticos e golpes digitais no Brasil, onde notou que 24% dos entrevistados havia perdido dinheiro devido a crimes na Internet, conforme imagem abaixo:

Figura 1: resultado da pesquisa de crimes digitais



Fonte: Instituto de Pesquisa DataSenado (2024)

Isto posto, faz-se necessário criação de ferramentas educacionais que auxiliem os usuários a identificar e se prevenirem contra ameaças digitais, de modo que a educação em segurança cibernética se torna uma agente na redução do sucesso de ataques digitais, cenário este onde o desenvolvimento de plataformas digitais educativas pode contribuir para a disseminação de conhecimento e interativo sobre segurança da informação.

Assim, nasce o projeto Cyber Chase: Desenvolvimento de uma Plataforma Web Educacional para Mitigação de Ataques Cibernéticos através da Conscientização Digital. Ele propõe a criação de uma plataforma web interativa voltada à educação digital e à conscientização sobre segurança cibernética buscando oferecer conteúdos educativos por meio de uma interface interativa para apoiar os usuários na identificação de ameaças virtuais e na adoção de práticas seguras no ambiente digital.

O principal problema do presente trabalho é a falta de conhecimento técnico dos usuários, o que contribui para o sucesso de ataques cibernéticos baseados em engenharia social e phishing, por exemplo, onde em muitos casos, os usuários não conhecem os procedimentos básicos de proteção digital e acabam sendo vítimas de golpes que poderiam ser evitados por meio de conscientização adequada, de tal modo

que se nota que investir em educação digital pode se tornar uma estratégia para fortalecer a segurança da informação.

O objetivo geral deste trabalho consiste em desenvolver o site “Cyber Chase”, uma plataforma web educacional voltada à conscientização digital e à mitigação de ataques cibernéticos por meio de métodos interativos de aprendizagem. A proposta visa proporcionar aos usuários uma experiência educativa dinâmica, com o uso de Inteligência Artificial e Machine Learning incentivando a adoção de boas práticas de segurança digital.

Para alcançar esse objetivo geral, foi necessário objetivos específicos como: pesquisar as principais vulnerabilidades exploradas em ataques cibernéticos, especialmente aquelas relacionadas ao comportamento humano e à engenharia social; a definição da arquitetura do site, considerando aspectos relacionados à usabilidade, acessibilidade e organização das funcionalidades da plataforma; implementação de funcionalidades baseadas em inteligência artificial, para aprimorar a experiência do usuário e oferecer mecanismos mais eficientes de aprendizado e interação; e validar a usabilidade da plataforma por meio de testes e avaliações que permitam verificar sua eficiência educacional e sua capacidade de conscientização digital.

A plataforma Cyber Chase se fundamenta na ideia de que a conscientização digital é uma das formas de combate às ameaças cibernéticas, onde a educação digital atua no fator humano, auxiliando usuários a reconhecer tentativas de fraude, proteger dados pessoais e adotar práticas seguras no ambiente virtual. Assim, busca-se contribuir para a formação de usuários mais conscientes, preparados e resistentes a ataques cibernéticos.

2 DESENVOLVIMENTO

2.1 Metodologias

Considerando que todo o processo de análise, modelagem, implementação e testes foi realizado individualmente, a metodologia utilizada para desenvolver a Cyber Chase foi a adaptação da metodologia ágil Scrum, aplicada de maneira simplificada e adequada ao contexto de um projeto acadêmico desenvolvido por uma única pessoa e por isso a metodologia Scrum tradicional foi adaptada em vista da quantidade de

papéis específicos, como Product Owner, Scrum Master e equipe de desenvolvimento, neste trabalho tais funções não foram formalmente aplicadas, porém, foram mantidos os principais conceitos da metodologia ágil, principalmente o desenvolvimento incremental, a divisão por etapas funcionais e a validação contínua das funcionalidades implementadas.

A escolha da metodologia Scrum ocorreu devido à sua capacidade de dividir o desenvolvimento em pequenas etapas incrementais, denominadas sprints, permitindo a implementação gradual das funcionalidades do sistema.

O desenvolvimento da plataforma foi estruturado em entregas progressivas, iniciando pela construção da interface básica do sistema e avançando gradativamente para funcionalidades mais complexas, incluindo integração com banco de dados, autenticação de usuários e implementação de recursos de inteligência artificial. As etapas do desenvolvimento foram organizadas da seguinte forma:

- Sprint 1: desenvolvimento da estrutura inicial da aplicação utilizando HTML;
- Sprint 2: estilização das páginas da plataforma utilizando CSS;
- Sprint 3: implementação da estrutura inicial do back-end utilizando o framework Flask em Python;
- Sprint 4: criação das tabelas do banco de dados em MySQL e integração com o framework Flask por meio do SQLAlchemy;
- Sprint 5: desenvolvimento das funcionalidades de autenticação de usuários, incluindo login e gerenciamento de sessões;
- Sprint 6: implementação do sistema de gerenciamento de cursos por meio das operações de cadastro, edição, visualização e exclusão (CRUD);
- Sprint 7: desenvolvimento das funcionalidades relacionadas ao chatbot inteligente da plataforma, utilizando conceitos de inteligência artificial e machine learning;
- Sprint 8: implementação da exibição dinâmica dos cursos disponíveis para os usuários da plataforma;
- Sprint 9: desenvolvimento do mecanismo de validação inteligente das respostas fornecidas pelos usuários nos testes dos cursos, utilizando técnicas de processamento de linguagem natural e cálculo de similaridade textual;

- Sprint 10: realização de testes funcionais e refinamentos gerais da plataforma, visando garantir estabilidade, desempenho e funcionamento adequado das funcionalidades implementadas.

Além da abordagem ágil baseada no Scrum adaptado, o desenvolvimento do sistema também utilizou conceitos de prototipação incremental, que contribuiu para melhorias relacionadas à usabilidade, organização visual da interface, experiência do usuário e validação prática das funcionalidades desenvolvidas. Dessa forma, foi possível realizar ajustes contínuos na plataforma conforme novas necessidades eram identificadas durante o desenvolvimento do projeto.

A adoção conjunta da metodologia ágil e da prototipação incremental proporcionou maior controle sobre o desenvolvimento da aplicação, favorecendo a organização das etapas do projeto e contribuindo para a construção de uma plataforma funcional, escalável e alinhada aos objetivos propostos neste trabalho.

Além disso, o código foi implementado com versionamento e controle de código por meio de um repositório remoto no github, disponível em: https://github.com/leticiaoj/tcc.

2.2 Levantamento de Requisitos

Para garantir o desenvolvimento de uma plataforma funcional, foi realizado o levantamento de requisitos para a Cyber Chase. O levantamento de requisitos define as funcionalidades, restrições e características necessárias para que a aplicação atenda aos objetivos propostos previamente.

2.2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades e serviços disponibilizados pela plataforma aos usuários e administradores. Esses requisitos especificam o comportamento esperado do sistema diante das interações realizadas pelos usuários.

No cenário da Cyber Chase, a plataforma possui um módulo completo de autenticação e gerenciamento de usuários. O sistema permite o cadastro de novos usuários mediante preenchimento de nome, e-mail e senha, existe um mecanismo de validação para impedir o registro de e-mails duplicados, garantindo maior integridade

dos dados armazenados e, após o cadastro, o sistema disponibiliza funcionalidades de autenticação, permitindo que usuários realizem login por meio de credenciais previamente cadastradas. Além disso foi criado um perfil administrativo responsável pelo gerenciamento dos cursos disponíveis na plataforma. O sistema também controla as sessões, mantendo sessões autenticadas durante a navegação do usuário, restringindo o acesso às páginas internas apenas para usuários autenticados de modo que funcionalidades críticas da plataforma permanecem protegidas contra acessos não autorizados.

Em relação ao gerenciamento de cursos, o sistema permite listar todos os cursos cadastrados, bem como visualizar informações detalhadas de cada curso, incluindo título, descrição, conteúdo programático, links externos e perguntas avaliativas e o administrador possui permissões para realizar operações de criação, edição e exclusão de cursos. Essas funcionalidades caracterizam a implementação de operações do tipo CRUD (Create, Read, Update e Delete), fundamentais em sistemas de gerenciamento de conteúdo educacional.

Um ponto forte da aplicação é a automatização proveniente da utilização de Inteligência Artificial e Machine Learning, onde plataforma permite que cada curso possua perguntas avaliativas e respostas de referência cadastradas previamente pelo administrador, e após o envio da resposta pelo aluno, o sistema realiza uma análise textual baseada em técnicas de Processamento de Linguagem Natural (PLN), comparando semanticamente a resposta submetida com a resposta considerada correta. Além de utilizar a PLN para as respostas, oz sistema também o utiliza em seu assistente virtual “Cybot” (chatbot), que possui finalidade de fornecer suporte automatizado aos usuários, respondendo perguntas relacionadas à plataforma, segurança digital, cursos e certificações, utilizando técnicas de vetorização textual e cálculo de similaridade semântica para identificar a resposta mais adequada.

Adicionalmente, foram identificadas páginas institucionais, como páginas “Sobre” e “Contato”, responsáveis por apresentar informações organizacionais e canais de comunicação da plataforma.

São portanto os requisitos funcionais os seguintes:

- RF01 - Cadastro de Usuários: O sistema deverá permitir o cadastro de novos usuários mediante fornecimento de nome, endereço de e-mail e senha;
- RF02 - Validação de Unicidade de E-mail: O sistema deverá impedir o cadastro de usuários com endereço de e-mail previamente registrado na base de dados;

- RF03 - Autenticação de Usuários: O sistema deverá permitir autenticação de usuários por meio de e-mail e senha;
- RF04 - Autenticação Administrativa: O sistema deverá disponibilizar acesso administrativo mediante credenciais específicas previamente configuradas;
- RF05 - Gerenciamento de Sessão: O sistema deverá manter sessões autenticadas durante a navegação do usuário;
- RF06 - Encerramento de Sessão: O sistema deverá permitir que o usuário realize logout, encerrando sua sessão ativa;
- RF07 - Restrição de Acesso: O sistema deverá restringir o acesso às funcionalidades internas aos usuários autenticados;
- RF08 - Exibição de Mensagens do Sistema: O sistema deverá apresentar mensagens de feedback relacionadas a autenticação, cadastro e operações realizadas;
- RF09 - Listagem de Cursos: O sistema deverá disponibilizar a listagem de todos os cursos cadastrados na plataforma;
- RF10 - Visualização de Detalhes do Curso: O sistema deverá permitir a visualização detalhada de cada curso, incluindo título, descrição, conteúdo, links complementares, perguntas avaliativas e certificados em caso de aprovação;
- RF11 - Exibição de Informações do Usuário: O sistema deverá exibir o nome do usuário autenticado e a data atual durante a navegação nos cursos;
- RF12 - Cadastro de Cursos: O sistema deverá permitir ao administrador cadastrar novos cursos;
- RF13 - Edição de Cursos: O sistema deverá permitir ao administrador editar cursos existentes;
- RF14 - Exclusão de Cursos: O sistema deverá permitir ao administrador remover cursos cadastrados;
- RF15 - Gerenciamento de Conteúdo Educacional: O sistema deverá permitir o gerenciamento de descrições, conteúdos textuais e links externos relacionados aos cursos;
- RF16 - Cadastro de Perguntas Avaliativas: O sistema deverá permitir o cadastro de perguntas avaliativas associadas aos cursos;
- RF17 - Cadastro de Respostas de Referência: O sistema deverá permitir o cadastro de respostas corretas utilizadas no processo de avaliação automática;

- RF18 - Submissão de Respostas: O sistema deverá permitir que os alunos submetam respostas textuais para avaliação;
- RF19 - Avaliação Automática por Inteligência Artificial: O sistema deverá utilizar técnicas de Processamento de Linguagem Natural para avaliar as respostas dos usuários;
- RF20 - Validação por Similaridade Textual: O sistema deverá aprovar respostas cuja similaridade textual seja superior ao limiar estabelecido pelo sistema;
- RF21 - Feedback de Avaliação: O sistema deverá informar ao usuário quando a resposta submetida estiver incorreta ou insuficiente;
- RF22 - Liberação Automática sem Avaliação: O sistema deverá aprovar automaticamente cursos que não possuam avaliação cadastrada;
- RF23 - Disponibilização de Chatbot Inteligente: O sistema deverá disponibilizar um assistente virtual denominado “Cybot”;
- RF24 - Atendimento Mediante Autenticação: O chatbot deverá responder apenas usuários autenticados;
- RF25 - Processamento de Perguntas: O chatbot deverá receber e processar perguntas enviadas pelos usuários;
- RF26 - Base de Conhecimento Institucional: O chatbot deverá responder perguntas relacionadas à plataforma, cursos e segurança cibernética;
- RF27 - Processamento Semântico: O chatbot deverá utilizar técnicas de vetorização textual e cálculo de similaridade semântica;
- RF28 - Respostas Automatizadas: O chatbot deverá retornar respostas automáticas com base na maior similaridade identificada;
- RF29 - Tratamento de Perguntas Desconhecidas: O chatbot deverá informar quando não possuir conhecimento suficiente para responder determinada pergunta;
- RF30 - Página Inicial do Sistema: O sistema deverá disponibilizar página inicial após autenticação do usuário;
- RF31 - Página Institucional “Sobre”: O sistema deverá disponibilizar página institucional contendo informações sobre a plataforma;
- RF32 - Página de Contato: O sistema deverá disponibilizar página de contato para comunicação institucional;
- RF33 - Interface Baseada em Templates: O sistema deverá renderizar páginas HTML dinâmicas por meio do mecanismo de templates do framework utilizado;

- RF34 - Retorno de Códigos HTTP: O sistema deverá retornar códigos HTTP apropriados para autenticação e comunicação entre cliente e servidor.

2.2.2 Requisitos Não Funcionais

Os requisitos não funcionais descrevem características de qualidade e restrições técnicas relacionadas ao funcionamento da plataforma, não são funcionalidades diretamente perceptíveis ao usuário, mas definem propriedades para o desempenho do sistema. No caso da Cyber Chase, foi implementado com o framework Flask, amplamente utilizado no desenvolvimento de aplicações web em Python, e para persistência de dados, o sistema utiliza banco de dados relacional MySQL em conjunto com a biblioteca SQLAlchemy do Python, responsável pelo mapeamento objeto-relacional (ORM) para a manipulação das entidades da aplicação com abstração no acesso aos dados.

Para Inteligência Artificial e Machine Learning, o sistema utiliza técnicas leves de Processamento de Linguagem Natural, como TF-IDF (Term Frequency–Inverse Document Frequency) e similaridade cosseno, permitindo respostas rápidas tanto para o chatbot quanto para o mecanismo de avaliação textual automatizada.

Também é garantida a usabilidade por meio de mensagens de feedback ao usuário em situações de autenticação, cadastro, avaliação e operações administrativas. E para garantir a modularidade, o sistema apresenta separação lógica entre rotas, modelos e templates, favorecendo manutenção, escalabilidade e futuras expansões da aplicação. Já na confiabilidade, o sistema realiza tratamento de exceções durante operações de escrita no banco de dados, evitando falhas críticas em casos de erro durante inserções ou atualizações.

São portanto os requisitos não funcionais os seguintes:

- RNF01 - Arquitetura Web: O sistema deverá operar em arquitetura cliente-servidor baseada em aplicação web;
- RNF02 - Framework Backend: O sistema deverá utilizar o framework Flask para desenvolvimento do backend;
- RNF03 - Banco de Dados Relacional: O sistema deverá utilizar banco de dados relacional MySQL para persistência das informações;

- RNF04 - Utilização de ORM: O sistema deverá utilizar SQLAlchemy como ferramenta ORM para manipulação do banco de dados;
- RNF05 - Persistência de Dados: O sistema deverá garantir persistência das informações armazenadas entre execuções da aplicação;
- RNF06 - Controle de Sessão: O sistema deverá utilizar mecanismos de sessão para autenticação e controle de acesso;
- RNF07 - Integridade de Dados: O sistema deverá garantir integridade das operações de escrita no banco de dados mediante utilização de transações;
- RNF08 - Unicidade de Usuários: O sistema deverá garantir unicidade do endereço de e-mail dos usuários cadastrados;
- RNF09 - Comunicação Estruturada: O sistema deverá utilizar o formato JSON para troca de informações entre frontend e backend;
- RNF10 - Disponibilidade de Serviços: As funcionalidades principais deverão permanecer disponíveis aos usuários autenticados;
- RNF11 - Tratamento de Exceções: O sistema deverá realizar tratamento de exceções durante operações críticas;
- RNF12 - Desempenho de Processamento: O sistema deverá fornecer respostas em tempo adequado para consultas ao chatbot e avaliações automáticas;
- RNF13 - Inteligência Artificial Baseada em PLN: O sistema deverá utilizar técnicas de Processamento de Linguagem Natural para análise textual;
- RNF14 - Vetorização Textual: O sistema deverá utilizar o algoritmo TF-IDF para representação vetorial de textos;
- RNF15 - Similaridade Semântica: O sistema deverá utilizar similaridade cosseno para comparação textual;
- RNF16 - Modularidade: O sistema deverá possuir organização modular baseada em rotas e componentes distintos;
- RNF17 - Escalabilidade Funcional: O sistema deverá permitir expansão da quantidade de cursos e conteúdos cadastrados;
- RNF18 - Extensibilidade da Base de Conhecimento: O sistema deverá permitir ampliação futura da base de perguntas e respostas do chatbot;
- RNF19 - Usabilidade: O sistema deverá apresentar mensagens claras e compreensíveis aos usuários;

- RNF20 - Navegação Intuitiva: O sistema deverá disponibilizar navegação estruturada por páginas e menus;
- RNF21 - Feedback Imediato: O sistema deverá apresentar feedback imediato sobre operações executadas;
- RNF22 - Confiabilidade: O sistema deverá garantir funcionamento consistente durante operações de autenticação, navegação e avaliação;
- RNF23 - Criação Automática de Estruturas: O sistema deverá criar automaticamente as tabelas do banco de dados quando necessário;
- RNF24 - Compatibilidade HTTP: O sistema deverá operar utilizando protocolo HTTP;
- RNF25 - Renderização Dinâmica: O sistema deverá gerar páginas dinamicamente por meio de templates HTML.

2.2.3 Requisitos Implícitos

Requisitos Implícitos são além dos requisitos explicitamente implementados no código-fonte, sendo eles: a finalidade educacional do sistema (educação cibernética); emissão de certificados de conclusão; oferta de cursos gratuitos e suporte automatizado por meio do assistente virtual Cybot.

São, portanto, os requisitos funcionais os seguintes:

- RI01 - Educação em Segurança Cibernética: A plataforma deverá fornecer conteúdos educacionais voltados à segurança cibernética;
- RI02 - Emissão de Certificados: A plataforma deverá disponibilizar certificados de conclusão aos usuários aprovados após validação da IA de perguntas e respostas;
- RI03 - Assistente Virtual Educacional: A plataforma deverá disponibilizar suporte automatizado por meio de assistente virtual;
- RI04 - Plataforma Gratuita: A plataforma deverá oferecer acesso gratuito aos conteúdos educacionais disponibilizados;

2.3 Modelagem do Sistema

2.3.1 Diagrama de Caso de Uso

O diagrama de caso de uso tem como objetivo representar visualmente funcionalidades disponibilizadas pelo sistema e a interação entre os atores e os recursos oferecidos pela aplicação, modelando os requisitos funcionais de um sistema e permitindo uma visão geral das operações realizadas pelos usuários e administradores da plataforma.

O diagrama da Cyber Chase possui três atores: o Usuário, o Administrador e o Cybot IA.

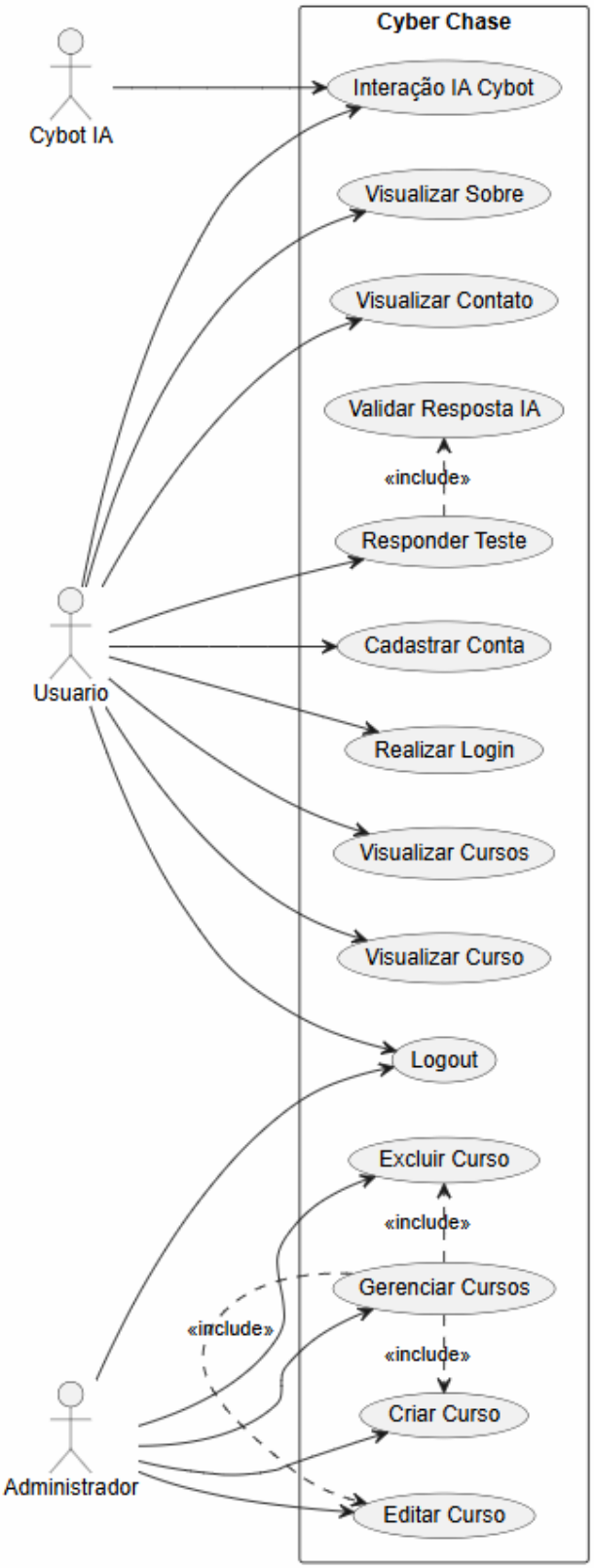
O ator Usuário representa quem acessa os conteúdos educacionais, com funcionalidades de cadastro de conta, realização de login, visualização de cursos, acesso aos detalhes dos cursos, resposta aos testes de validação e interação com o chatbot, acessar páginas institucionais, como “Sobre” e “Contato” e fazer logout ao encerrar a sessão. A autenticação deles foi modelada como um caso de uso de segurança da aplicação, onde o usuário deve realizar login antes de acessar áreas restritas do sistema, mecanismo implementado por meio de sessões no framework Flask, além do cadastro de usuários ser armazenado em banco de dados MySQL com o ORM SQLAlchemy. Ademais, há o caso de uso “Responder Teste”, com relação do tipo <> com o caso de uso “Validar Resposta IA”, devido ao fato de que a validação com IA das respostas é uma obrigação antes de liberar o certificado, utilizando, para isso, técnicas de Inteligência Artificial baseadas em TF-IDF (Term Frequency–Inverse Document Frequency) e similaridade de cosseno para comparar semanticamente a resposta fornecida pelo usuário com a resposta correta cadastrada no banco de dados e se a similaridade for maior que o range definido pelo sistema, a resposta é considerada válida.

O ator Administrador possui acesso privilegiado às funcionalidades administrativas da plataforma, na qual seu principal caso de uso é o gerenciamento de cursos, que engloba as ações de criar e editar informações como título, descrição, conteúdo, links externos e questões avaliativas associadas aos cursos, e excluir cursos disponíveis no sistema

O ator Cybot IA possui funcionalidade de atendimento automatizado aos usuários. O chatbot foi implementado com técnicas de processamento de linguagem

natural, utilizando vetorização TF-IDF e cálculo de similaridade textual para identificar perguntas semelhantes às cadastradas na base de conhecimento do sistema por meio de Machine Learning, onde a Inteligência Artificial faz com que o Cybot responda automaticamente as dúvidas dos usuários em relação à Cyber Chase e seus cursos.

Figura 2: Diagrama de Caso de Uso



Fonte: construído via https://plantuml.com/fr/ (2026)

O diagrama de casos de uso acima contém as principais interações funcionais ocorridas na Cyber Chase, permitindo visualizar as funcionalidades disponíveis no sistema. O usuário comum possui acesso às funcionalidades de autenticação,

visualização de cursos, realização de testes e interação com o chatbot inteligente, que também é um ator, visto que media o input do usuário e o retorna por meio de Machine Learning e Inteligência Artificial; enquanto o administrador possui permissões adicionais relacionadas ao gerenciamento dos cursos disponíveis na plataforma.

2.3.2 Diagrama de Classe

O diagrama de classes possui a estrutura lógica do sistema, permitindo visualizar os componentes principais do sistema, seus atributos, métodos e relacionamentos.

A classe principal do sistema é denominada FlaskApp, responsável pelo gerenciamento geral da aplicação web visto que foi desenvolvida com o framework Flask, que possui funcionalidades relacionadas à inicialização da aplicação, configuração do banco de dados e execução do servidor, além de estabelecer dependência com as classes Usuario, Curso, ChatbotIA e ValidadorTesteIA.

A classe Usuario representa os usuários cadastrados na plataforma e possui como atributos um identificador (id), nome, e-mail e senha, além de possuir métodos relacionados ao processo de autenticação que envolve ações de cadastro, login e logout, nas quais o sistema utiliza sessões para controlar o acesso às páginas restritas.

A classe Curso é responsável pelo armazenamento e gerenciamento das informações dos cursos disponibilizados, com os atributos título, descrição, conteúdo, link complementar, pergunta de teste e resposta correta, cuja estrutura permite que cada curso possua materiais educativos e mecanismos de avaliação automatizados. Esta também classe possui operações de criação de curso, edição de cursos com alteração dos atributos citados acima, exclusão e visualização de cursos, o que caracteriza um sistema CRUD (Create - Criar -, Read - Ler -, Update - Atualizar - e Delete - Deletar -).

Há também a classe ChatbotIA, que é a responsável pelo funcionamento do assistente virtual Cybout, utilizando técnicas de Processamento de Linguagem Natural (PLN), implementadas por meio da biblioteca scikit-learn com Machine Learning, especificamente utilizando o algoritmo TF-IDF e o cálculo de similaridade por cosseno para compor a Inteligência Artificial, de modo que o chatbot “aprenda” uma base de

conhecimento contendo perguntas e respostas pré-definidas, e possa, a partir delas, responder automaticamente às dúvidas dos usuários por meio do cálculo de similaridade citado. Além disso, o Machine Learning e a Inteligência Artificial são aplicados de forma similar à classe ValidadorTesteIA, que atua na validação automática das respostas discursivas enviadas pelos alunos nos testes dos cursos. O processo ocorre através da comparação textual entre a resposta fornecida pelo usuário e a resposta cadastrada no banco de dados, onde, da mesma forma, é realizado o cálculo de similaridade textual baseado em TF-IDF. Caso o nível de similaridade ultrapasse o range estabelecido, a resposta é considerada correta.

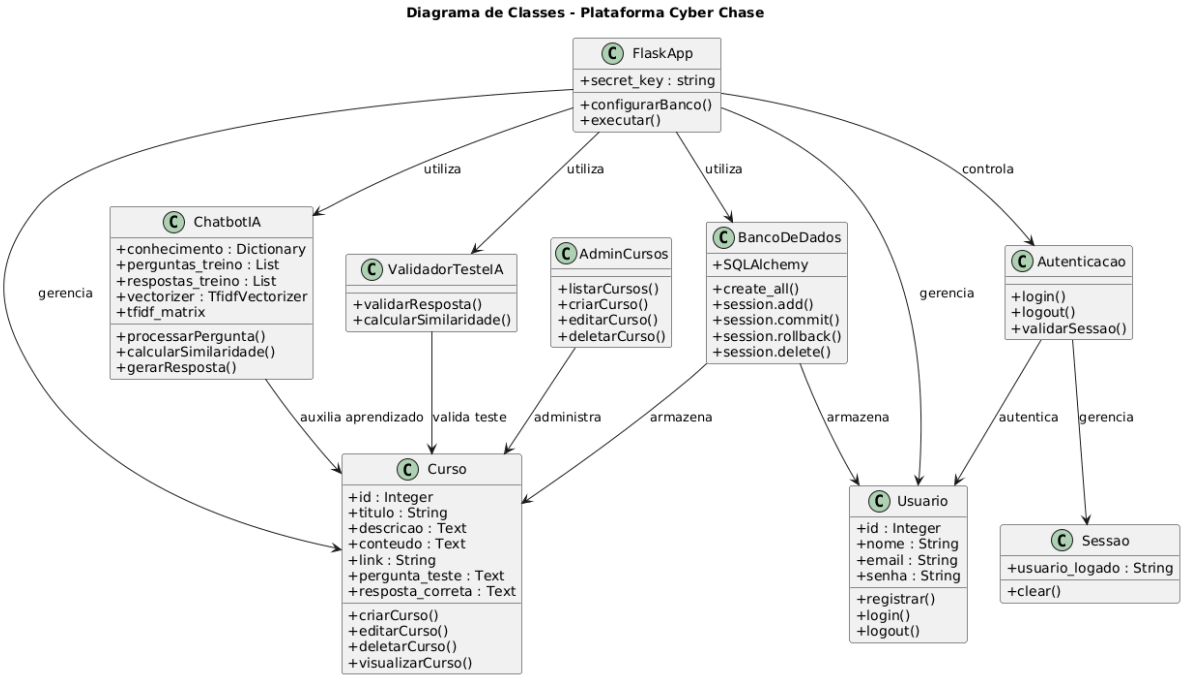
A classe BancoDeDados é implementada por meio da biblioteca SQLAlchemy integrada ao banco de dados MySQL, cuja responsabilidade se dá pelas operações de armazenamento, atualização, exclusão e recuperação das informações do sistema, garantindo a comunicação entre a aplicação e o banco relacional.

A classe Autenticacao possui mecanismos de controle de acesso e gerenciamento de sessões, e verifica se o usuário está autenticado antes de permitir o acesso às funcionalidades protegidas da plataforma, onde é armazenado temporariamente as informações do usuário autenticado na classe Sessao.

Diferentemente, a classe AdminCursos concentra a função administrativa de gerenciamento dos cursos cadastrados na plataforma, onde administradores podem criar, editar e excluir os cursos.

Em resumo, se nota a relação descrita acima no diagrama a seguir:

Figura 3: Diagrama de Classes



Fonte: construído via https://plantuml.com/fr/ (2026)

2.3.3 Diagrama de Sequência

O diagrama de sequência apresentado constitui o funcionamento da Inteligência Artificial implementada na Cyber Chase, de modo a ilustrar a interação entre o usuário, a interface do sistema, o servidor back-end em Python Flask, o módulo de Inteligência Artificial e o banco de dados MySQL, com o objetivo de ilustrar a autenticação, disponibilização de cursos, respostas e validações de perguntas ao Chatbot e validação das respostas dos cursos com as técnicas de Processamento de Linguagem Natural.

O fluxo começa com a autenticação do usuário, quando o usuário insere suas credenciais na interface front-end, desenvolvida em HTML e JavaScript, que encaminha uma requisição HTTP do tipo POST para a rota /login do servidor Flask e o back-end consulta o banco de dados MySQL para verificar a se o usuário existe na tabela e validar suas credenciais. Caso as informações estejam corretas, uma sessão é criada e o usuário é redirecionado para a página principal da plataforma, senão, o sistema retorna uma mensagem de erro informando falha na autenticação.

Após o login, o usuário pode acessar os cursos disponíveis na plataforma. Quando um curso é selecionado, o front-end envia uma requisição GET para a rota

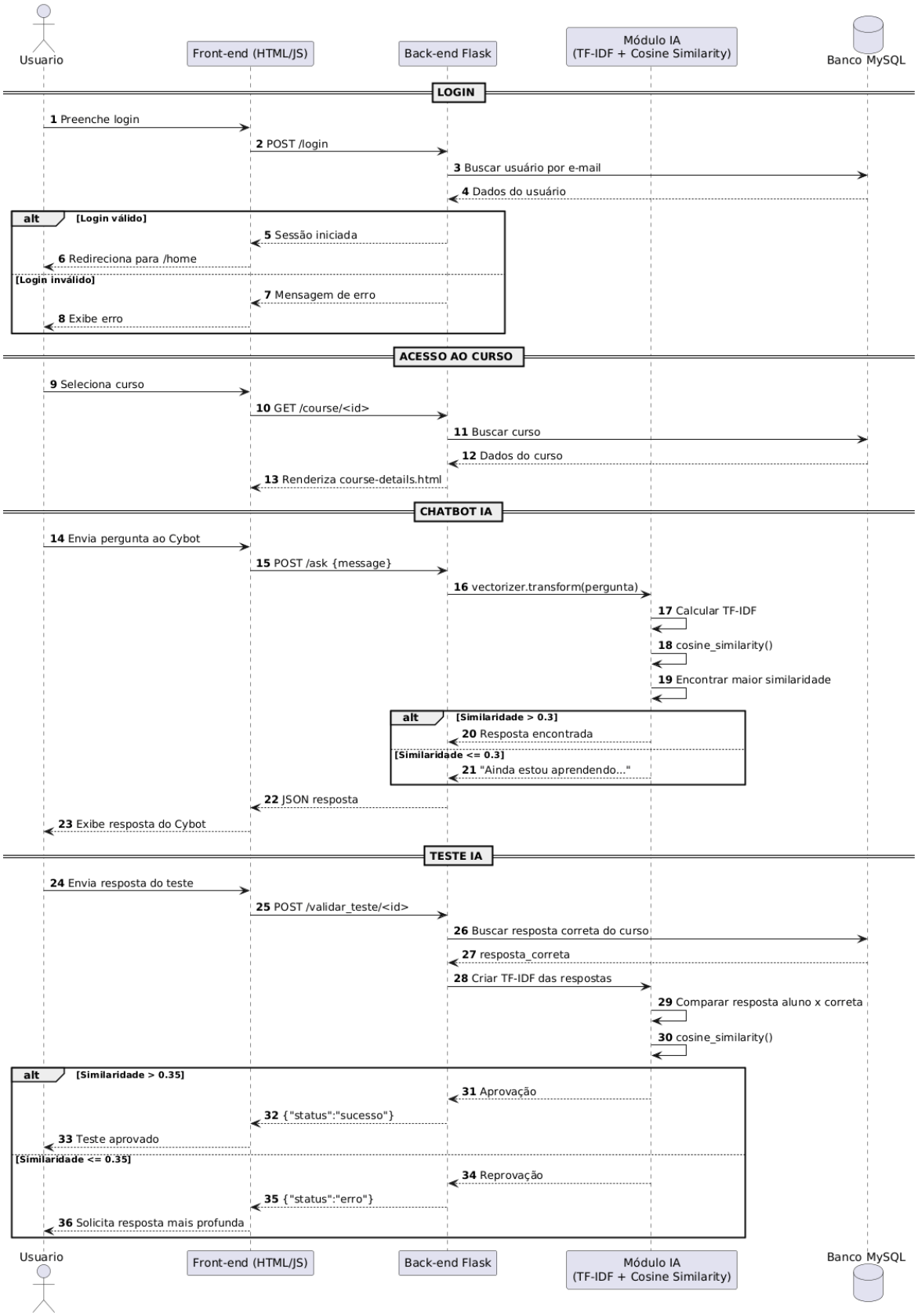
/course/<id> e o servidor consulta o banco de dados para chamar as informações do curso de título, descrição, conteúdo e perguntas, que são renderizadas na página de detalhes do curso e disponibilizadas ao usuário.

Há o componente do chatbot “Cybot” com Inteligência Artificial e Machine Learning, onde sua funcionalidade consiste nas técnicas de vetorização textual por meio do algoritmo TF-IDF (Term Frequency – Inverse Document Frequency), em conjunto com o cálculo de similaridade por cosseno (Cosine Similarity), que, juntos, fazem com que ao enviar uma pergunta do usuário, a interface realiza uma requisição POST para a rota /ask, e envia a mensagem. O módulo de IA transforma o texto da pergunta em um vetor numérico com o TfidfVectorizer da biblioteca Scikit-learn, com a qual se calcula a similaridade entre a pergunta enviada e as perguntas previamente cadastradas na base de treinamento com Machine Learning do sistema, onde o algoritmo identifica a pergunta mais próxima semanticamente da mensagem do usuário, indicado pelo índice de proximidade superior a 0,3, que faz com que o sistema retorne a resposta correspondente cadastrada na base de treinamento. Se o chatbot não encontrar na base de dados uma similaridade calculada entre a pergunta e a resposta, ele responde que ainda está aprendendo sobre o assunto.

Além do chatbot, também há aplicação de Inteligência Artificial para validar testes discursivos dos cursos automaticamente de forma similar à validação do Chatbot, de modo que, quando o usuário envia uma resposta para determinada atividade, o front-end envia uma requisição POST para a rota /validar_teste/<id> e o servidor consulta no banco de dados a resposta considerada correta para aquele curso e realiza uma comparação textual entre a resposta do usuário e a resposta oficial cadastrada no banco de dados que serviu como base de conhecimento, a fim de buscar similaridades, onde também é utilizado TF-IDF e similaridade por cosseno para transformar os textos em vetores matemáticos e medir o grau de proximidade semântica entre eles, se for maior que 35%, a resposta é considerada correspondente e o usuário é aprovado automaticamente para gerar o certificado por meio do botão que é gerado. Senão, o sistema retorna uma mensagem informando que a resposta não possui profundidade suficiente ou está incorreta.

Esta relação é ilustrada no diagrama a seguir:

Figura 4: Diagrama de Sequência



Fonte: construído via https://plantuml.com/fr/ (2026)

2.3.4 Diagrama de Componentes

A Cyber Chase foi estruturada em camadas que separam a interface do usuário, a lógica de negócio, os serviços de inteligência artificial e a persistência de dados, que é representada no Diagrama de Componentes, que possui diversas camadas, sendo uma delas a camada de apresentação que é composta por páginas HTML renderizadas dinamicamente pelo Flask com todas as interfaces, de modo que a camada de apresentação é a que interage diretamente com o usuário, e envia requisições HTTP ao backend e exibe os resultados processados pelo servidor.

Já a camada de aplicação é o centro da aplicação e é representado pelo componente principal Flask, com o controle de rotas, regras de negócio e integração entre os demais módulos do sistema, sendo subdividida em componentes funcionais como o Módulo de Autenticação que atua no registro e validação de usuários, o Gerenciador de Sessões, que usa o mecanismo de session do Python para manter o estado atual do usuário, o Gerenciamento de cursos com o controle da exibição dos cursos e seus detalhes, o CRUD Administrativo com operações de criação, leitura, atualização e exclusão de cursos por administradores e o Cybot, que é o chatbot com IA e Machine Learning que atende usuários de forma automatizada por meio do processamento de linguagem natural.

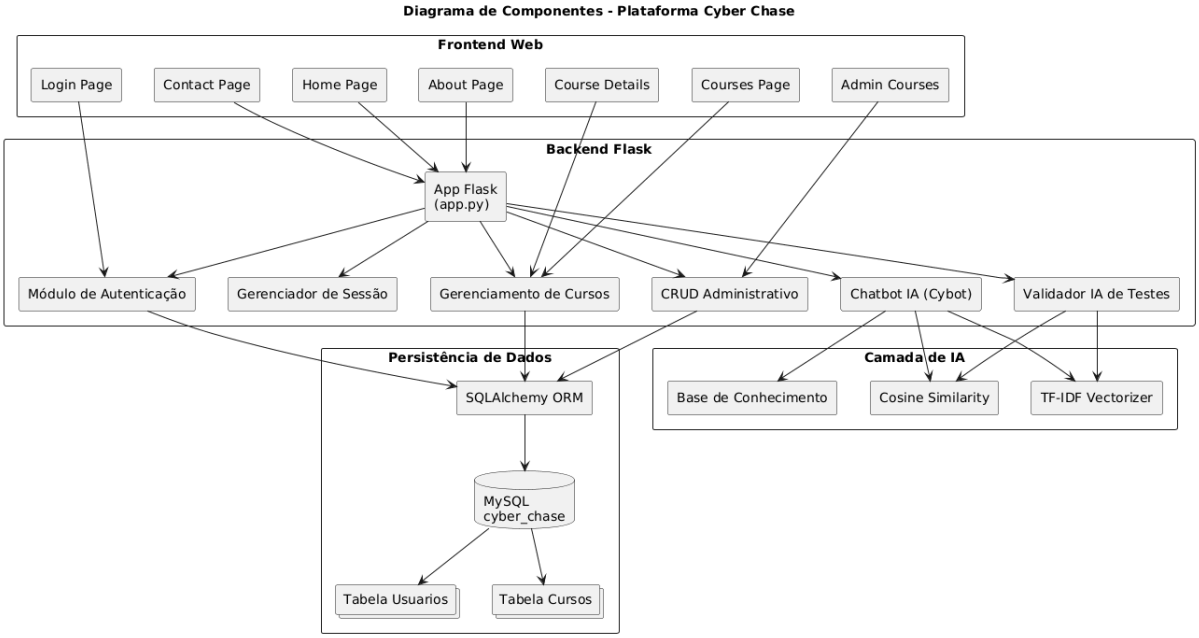
Apesar da presença do Cybot na camada de aplicação, há uma camada exclusiva de Inteligência Artificial com as técnicas de Processamento de Linguagem Natural (PLN). Este módulo utiliza o algoritmo TF-IDF (Term Frequency–Inverse Document Frequency) para vetorização de textos e a métrica de similaridade do cosseno para comparação semântica, onde o chatbot opera a partir de uma base de conhecimento pré-definida por meio do Machine Learning, e permite a resposta automática a perguntas. Porém, apesar do Cybot interagir com as duas camadas de aplicação e Inteligência Artificial, esta se torna exclusiva para a questão de validação das perguntas dos cursos utilizando os mesmos recursos utilizados pelo Chatbot de processamento de linguagem natural e cálculo com cosseno e vetor matemático de similaridade para comparar respostas de usuários com respostas corretas armazenadas no banco de dados, atribuindo uma taxa de similaridade, que, se for maior do que 35% são consideradas corretas.

Há também a Camada de Persistência de Dados, que está presente no ORM SQLAlchemy, que realiza a comunicação entre a aplicação Flask e o banco de dados

MySQL. O sistema possui as entidades Usuarios com as informações de cadastro (nome, e-mail e senha) e Curso com as informações dos cursos (título, descrição, conteúdo, link, pergunta de avaliação e resposta esperada para a avaliação).

Para que todos esses componentes se comuniquem, ocorre requisições HTTP e chamadas internas entre módulos do backend, onde o Python Flask conecta a interface do usuário, os serviços de inteligência artificial e o banco de dados.

Figura 5: Diagrama de Componentes



Fonte: construído via https://plantuml.com/fr/ (2026)

2.3.5 Diagrama de Atividades

O diagrama de atividades apresentado representa o fluxo completo de funcionamento da Cyber Chase, desde o acesso inicial do usuário até as interações com cursos, testes avaliativos e o chatbot, sendo utilizado para modelar o comportamento dinâmico do sistema, de modo a ilustrar a sequência de ações, decisões e possíveis ramificações dentro dos processos.

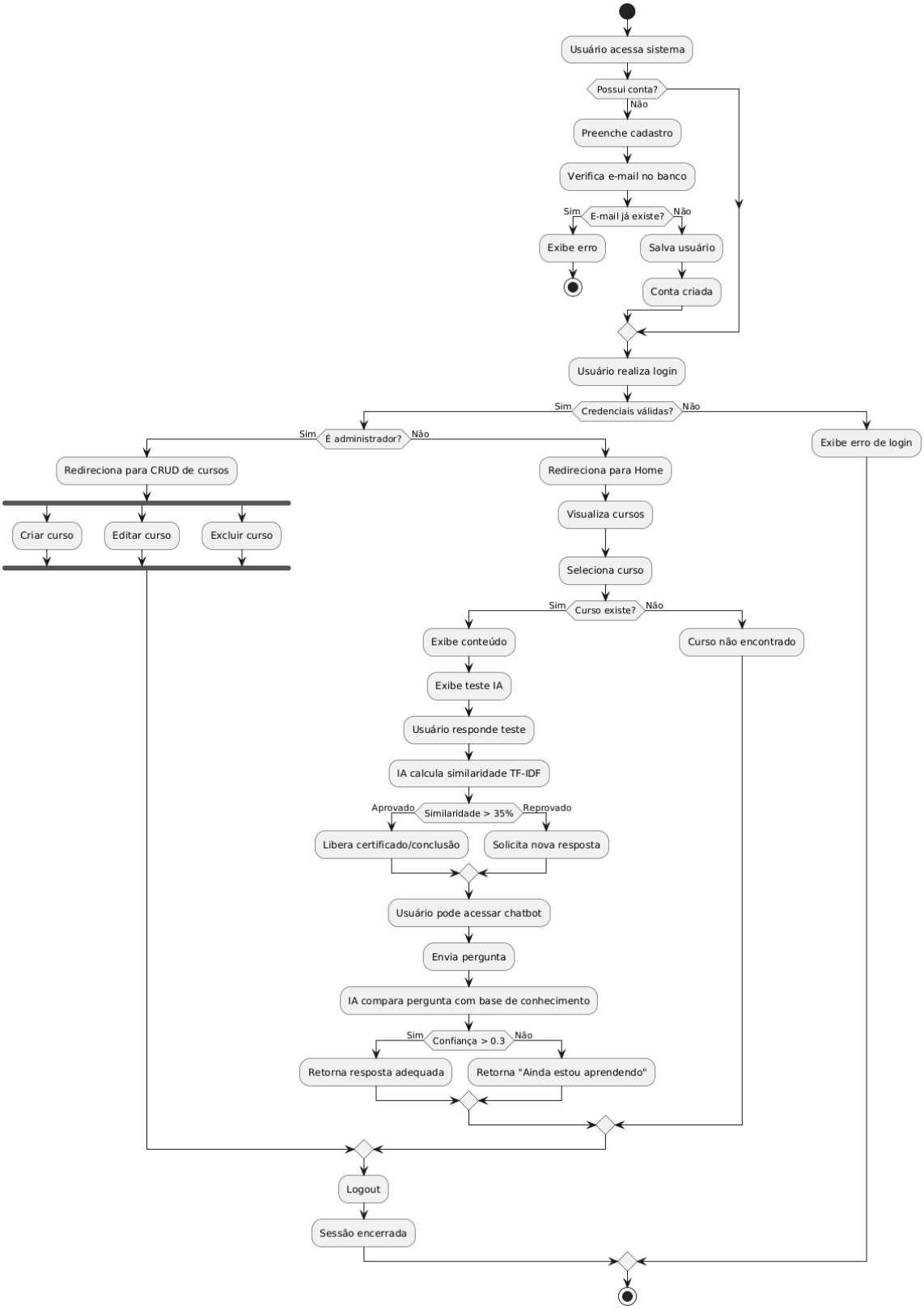
No diagrama, descreve-se o sistema permitindo que o usuário acesse a plataforma, sendo necessário verificar se ele já possui uma conta cadastrada, se não, é direcionado ao cadastro, no qual informa seus dados e o sistema realiza a validação

do e-mail no banco de dados e se o e-mail já estiver registrado, o sistema exibe uma mensagem de erro e interrompe o processo, senão, o novo usuário é salvo no banco de dados e sua conta é criada com sucesso, podendo então fazer login no qual suas credenciais são verificadas e, se forem inválidas, o sistema retorna uma mensagem de erro, e se forem válidas, o sistema identifica o tipo de usuário, distinguindo entre administrador e usuário comum, se for identificado como administrador, ele é redirecionado para a área de gerenciamento de cursos, onde ele pode realizar o CRUD dos cursos; se for um usuário comum, o sistema o direciona para a página inicial, onde ele pode ser redirecionado para a interface de cursos, visualizá-los e selecionar um curso específico. Caso o curso exista, o sistema exibe seu conteúdo e disponibiliza um teste avaliativo com inteligência artificial. O usuário responde ao teste e o sistema aplica um algoritmo de similaridade textual baseado em TF-IDF (Term Frequency–Inverse Document Frequency) para comparar a resposta do aluno com a resposta correta armazenada no banco de dados. Se a similaridade for superior a 35%, o aluno é considerado aprovado e pode receber a conclusão do curso; caso contrário, o sistema solicita que ele refaça a resposta, incentivando a melhoria do aprendizado.

Além disso, o usuário pode interagir com o chatbot ao enviar uma pergunta, onde o sistema utiliza novamente o modelo TF-IDF para comparar a mensagem com uma base de conhecimento previamente definida e se o nível de confiança da resposta for superior a 0,3, o sistema retorna uma resposta correspondente; se não encontrar correspondência, informa que ainda está em processo de aprendizado.

Por fim, independentemente do tipo de usuário, o fluxo é finalizado com o logout, sendo mostrado a seguir:

Figura 6: Diagrama de Atividades

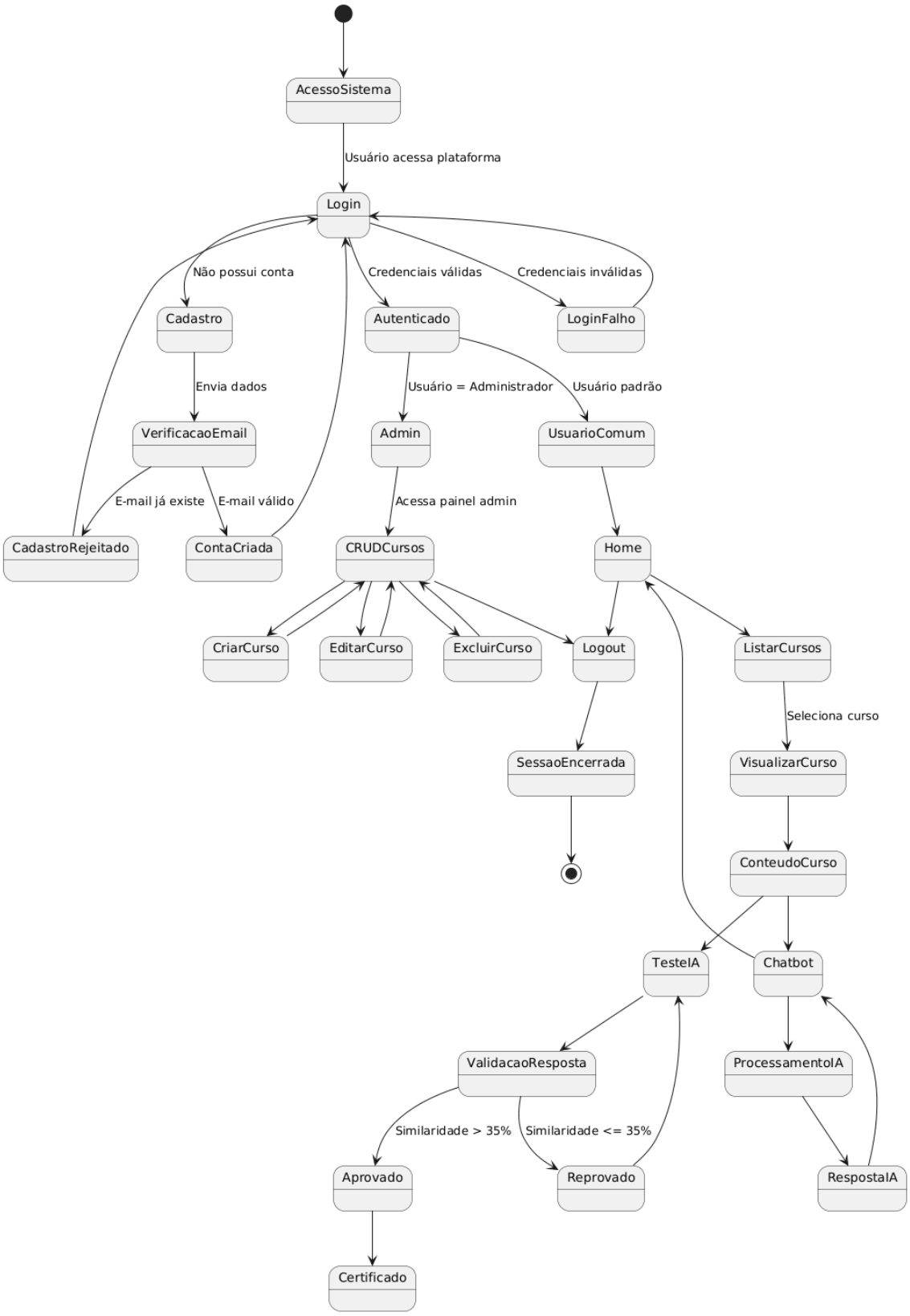


Fonte: construído via https://plantuml.com/fr/ (2026)

2.3.6 Diagrama de Estado

O diagrama de estados apresentado tem como finalidade descrever o comportamento dinâmico da Cyber Chase ao longo do ciclo de vida de um usuário dentro do sistema, desde o acesso inicial à plataforma até o encerramento da sessão. Inicialmente, o sistema encontra-se no estado de acesso, no qual o usuário tenta ingressar na plataforma. Após isso, ocorre a transição para o estado de autenticação, no qual o sistema verifica se o usuário já possui uma conta cadastrada, se não houver, o sistema direciona para o estado de cadastro, onde são coletadas as informações necessárias. Após isso, ocorre a verificação do e-mail no banco de dados, onde se o e-mail já estiver registrado, o sistema entra no estado de cadastro rejeitado, e retorna o usuário ao processo de login, senão, a conta é criada com sucesso e o sistema retorna ao estado de login. Após validar as credenciais, o sistema transita para o estado de autenticação bem-sucedida. Nesse ponto, ocorre a bifurcação do fluxo, distinguindo o tipo de usuário: administrador ou usuário comum. Se o usuário autenticado for administrador, o sistema entra no estado de gerenciamento de cursos (CRUD), se o usuário for comum, o sistema o direciona ao estado inicial da área do usuário (home). A partir desse ponto, o usuário pode acessar a lista de cursos disponíveis e selecionar um curso específico e ao acessar um curso válido, o sistema entra no estado de visualização de conteúdo, seguido pelo estado de realização de teste avaliativo baseado em inteligência artificial, se for acima de 35% com o algoritmo em TF-IDF, é aprovado; senão, é retornado para o usuário rever seu input, ocorrendo essa validação, também, na interação com o Cybot, onde ele retorna uma resposta aproximada em 30%, e se não, diz que não aprendeu o ainda a resposta para a pergunta inserida.

Figura 7: Diagrama de Estados



Fonte: construído via https://plantuml.com/fr/ (2026)

2.3.7 Diagrama de Arquitetura

A arquitetura da Cyber Chase foi desenvolvida utilizando o padrão de aplicação web monolítica em camadas, com Python usando o Flask como framework para implementação do backend, integrada ao sistema gerenciador de banco de dados MySQL e complementada por mecanismos de Inteligência Artificial e Machine Learning voltados ao processamento de linguagem natural, de modo que o diagrama de arquitetura foi estruturada em quatro camadas principais: camada de apresentação, camada de aplicação, camada de inteligência artificial e camada de persistência de dados.

A camada de apresentação é responsável pela interação entre usuário e sistema, sendo composta pelas interfaces web renderizadas por meio de templates HTML integrados ao Flask. Nessa camada encontram-se as páginas de autenticação, área inicial do usuário, listagem de cursos, visualização detalhada dos conteúdos, páginas institucionais e painel administrativo. O frontend também utiliza recursos estáticos, como folhas de estilo CSS, scripts JavaScript e elementos visuais responsáveis pela experiência de navegação.

A camada de aplicação concentra a regra de negócio da plataforma, onde há as rotas HTTP responsáveis pelo processamento das requisições dos usuários. Entre as funcionalidades presentes destacam-se o sistema de autenticação, gerenciamento de sessões, controle de acesso, consulta de cursos, operações administrativas de cadastro, edição e exclusão de conteúdos, além da comunicação entre frontend e banco de dados.

Há também o mecanismo de autenticação utiliza sessões internas do Flask para controlar o estado de login dos usuários. Após a validação das credenciais, o sistema armazena informações da sessão por meio da estrutura session, permitindo o controle de acesso às páginas protegidas da aplicação.

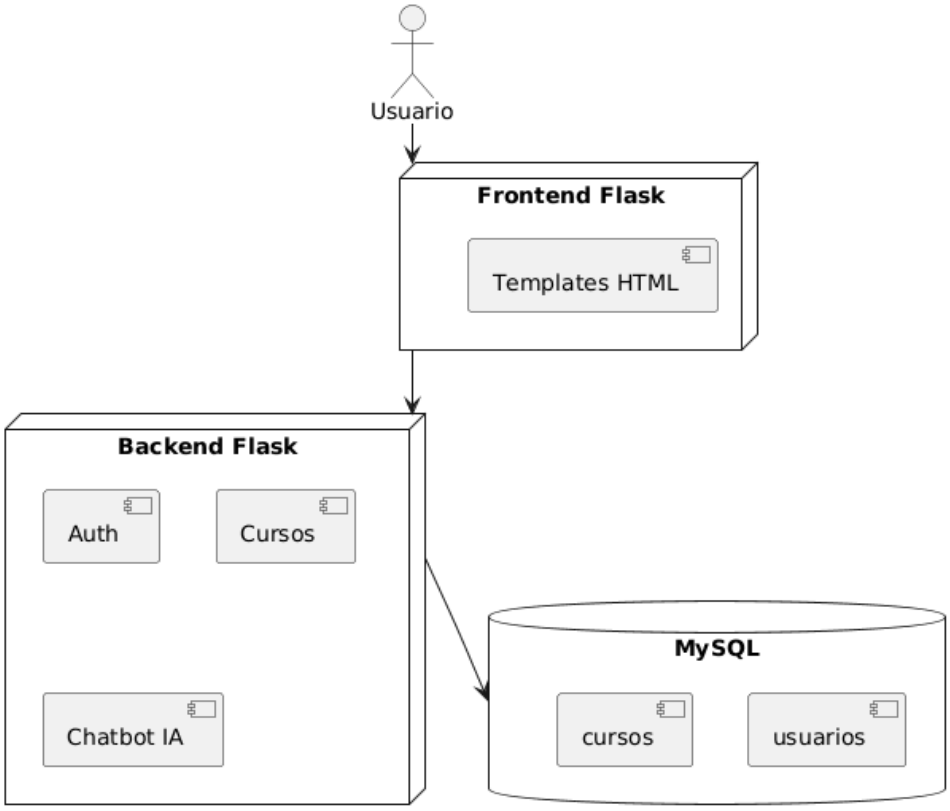
A camada de persistência é composta pelo banco de dados MySQL, acessado através do ORM SQLAlchemy. O sistema possui duas entidades principais: usuários e cursos. A entidade de usuários armazena informações de identificação e autenticação, enquanto a entidade de cursos contém os materiais educacionais, conteúdos programáticos, links externos e informações relacionadas às avaliações inteligentes.

A comunicação entre a aplicação Flask e o banco de dados ocorre por meio do SQLAlchemy, responsável por abstrair operações SQL e facilitar o gerenciamento das tabelas e consultas da aplicação.

Há a camada de Inteligência Artificial com técnicas de Processamento de Linguagem Natural (PLN) implementadas com a biblioteca Scikit-learn, especialmente os algoritmos TF-IDF (Term Frequency–Inverse Document Frequency) e Similaridade do Cosseno para duas funcionalidades principais: o “Cybot”, responsável por responder perguntas relacionadas à plataforma e conceitos básicos de segurança cibernética, cujo funcionamento ocorre através da vetorização textual das perguntas cadastradas e comparação semântica entre a entrada do usuário e a base de conhecimento previamente definida; e a validação automatizada de respostas discursivas aplicadas nos testes dos cursos. Nesse processo, a resposta fornecida pelo aluno é comparada semanticamente à resposta correta armazenada no banco de dados. A comparação é realizada utilizando TF-IDF associado ao cálculo de similaridade do cosseno, permitindo identificar o nível de correspondência textual entre as respostas. Quando a similaridade ultrapassa o range estabelecido pelo sistema, a resposta é considerada válida.

Do ponto de vista arquitetural, o modelo é centralizado, onde o frontend, backend, regras de negócio e IAs são integrados em uma única aplicação Flask, conforme é ilustrado a seguir:

Figura 8: Diagrama de Arquitetura



Fonte: construído via https://plantuml.com/fr/ (2026)

2.4 Desenvolvimento Back-end

O desenvolvimento do back-end do sistema foi construído com o microframework Flask importado do Python, com bibliotecas voltadas ao desenvolvimento web e à ciência de dados. A arquitetura adotada no projeto possui uma estrutura próxima ao padrão MVC (Model-View-Controller), porém de forma simplificada, onde as rotas desempenham o papel de controladores, os modelos representam as entidades do banco de dados e as páginas HTML funcionam como camada de visualização.

A aplicação foi integrada ao SQLAlchemy, um ORM (Object Relational Mapper) que permite a manipulação do banco de dados MySQL de forma orientada a objetos, reduzindo a necessidade de consultas SQL explícitas e facilitando a manutenção do código.

2.4.1 Framework Flask

O Flask foi utilizado como núcleo da aplicação back-end, gerenciando requisições HTTP, definição de rotas e controle do fluxo da aplicação.

Figura 9: trecho do código com as importações

main.py > Usuario

```python
...
from flask import Flask, render_template, request, redirect, url_for, flash, session, jsonify
from flask_sqlalchemy import SQLAlchemy
import os
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from datetime import datetime

#configuracao inicial
base_dir = os.path.dirname(os.path.abspath(__file__))
app = Flask(__name__, template_folder=base_dir, static_folder=base_dir, static_url_path='')
app.secret_key = "cyber_chase_secret_key"

#banco de dados em mysql
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+mysqlconnector://root:root@127.0.0.1:3306/cyber_chase'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

#models bd
...
class Usuario(db.Model):
    __tablename__ = 'usuarios'
    id = db.Column(db.Integer, primary_key=True)
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

Além disso, o Flask oferece suporte nativo a sessões, redirecionamentos e renderização de templates, utilizados no projeto para controle de autenticação e navegação entre páginas. A integração com bibliotecas externas, como o SQLAlchemy e o scikit-learn, para o desenvolvimento de funcionalidades avançadas de Machine Learning e Inteligência Artificial do chatbot e a validação de respostas com base em similaridade textual.

2.4.2 Rotas

As rotas são um dos principais componentes da aplicação, pois mapeiam URLs específicas para funções no back-end. Cada rota define um ponto de entrada para uma funcionalidade do sistema, controlando o comportamento da aplicação conforme o tipo de requisição HTTP recebida.

Figura 10: trecho do código com exemplos de rotas para telas como a inicial e a de cursos exigindo que o usuário esteja logado, e rota de interação do chatbot utilizando machine learning.

```python
@app.route('/home')
def index():
    if 'usuario_logado' not in session:
        return redirect(url_for('login_page'))
    return render_template('index.html')

@app.route('/ask', methods=['POST'])
def ask_bot():
    if 'usuario_logado' not in session:
        return jsonify({"response": "Por favor, faça login primeiro."}), 401
    data = request.json
    user_message = data.get("message", "")
    if not user_message:
        return jsonify({"response": "Diga algo para eu te ajudar!"})
    query_vec = vectorizer.transform([user_message])
    similaridade = cosine_similarity(query_vec, tfidf_matrix)
    indice_melhor = np.argmax(similaridade)
    score_confianca = similaridade[0][indice_melhor]
    resposta = respostas_treino[indice_melhor] if score_confianca > 0.3 else "Ainda estou aprendendo sobre isso."
    return jsonify({"response": resposta})

@app.route('/courses')
def page_courses():
    if 'usuario_logado' not in session:
        return redirect(url_for('login_page'))
    cursos = Curso.query.all()
    return render_template('courses.html', cursos=cursos)

@app.route('/course/<int:id>')
def visualizar_curso(id):
    if 'usuario_logado' not in session:
        return redirect(url_for('login_page'))
    curso = db.session.get(Curso, id)
    if not curso:
        flash("Curso não encontrado.")
        return redirect(url_for('page_courses'))
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

No sistema desenvolvido, as rotas foram organizadas em diferentes categorias funcionais, sendo as rotas de autenticação para login, registro e logout de usuários; as rotas principais do sistema para as páginas; as rotas de cursos para visualização de cada curso; as rotas administrativas para o gerenciamento dos cursos com CRUD; e as rotas de API para comunicação entre back-end e front-end para o chatbot e a validação das perguntas dos cursos.

2.4.3 Autenticação

O sistema possui autenticação baseada em sessões, onde as informações do usuário autenticado são armazenadas temporariamente no servidor na variável session do Flask

Figura 11: autenticação de login de administrador e dos demais usuários seguido da rota de logout e da validação com a session se o usuário está logado para acessar a página principal

```python
@app.route('/login', methods=['POST'])
def login():
    email = request.form.get('nome')
    senha = request.form.get('senha')
    if email == "admin@cyberchase.com.br" and senha == "root":
        session['usuario_logado'] = "Administrador"
        return redirect(url_for('listar_cursos'))
    usuario = Usuario.query.filter_by(email=email).first()
    if usuario and usuario.senha == senha:
        session['usuario_logado'] = usuario.nome
        return redirect(url_for('index'))
    flash("E-mail ou senha incorretos!")
    return redirect(url_for('login_page'))

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login_page'))


@app.route('/home')
def index():
    if 'usuario_logado' not in session:
        return redirect(url_for('login_page'))
    return render_template('index.html')
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

O processo de autenticação ocorre no cadastro, onde dados são armazenados no banco de dados; e no login, no qual há comparação entre as credenciais informadas e o banco de dados. Além dessas, há credenciais fixas no código para diferenciar o acesso e verificação do administrador para acesso a funcionalidades restritas de gerenciamento de cursos.

Se o usuário não estiver autenticado, o sistema não concede o acesso às rotas e direciona para a página de login, e para encerrar a sessão ativa e revogar o acesso, é utilizado o logout

2.4.4 CRUD (Create, Read, Update, Delete)

O sistema possui operações CRUD para a entidade Curso, permitindo o gerenciamento total dos conteúdos educacionais disponibilizados na plataforma.

Figura 12: trecho do código com as rotas de CRUD do administrador da plataforma

```python
@app.route('/admin/cursos')
def listar_cursos():
    if 'usuario_logado' not in session: return redirect(url_for('login_page'))
    cursos = Curso.query.all()
    return render_template('admin-courses.html', cursos=cursos)

@app.route('/admin/cursos/novo', methods=['POST'])
def criar_curso():
    if 'usuario_logado' not in session: return redirect(url_for('login_page'))
    novo = Curso(
        titulo=request.form.get('titulo'),
        descricao=request.form.get('descricao'),
        conteudo=request.form.get('conteudo'),
        link=request.form.get('link') or None,
        pergunta_teste=request.form.get('pergunta_teste'),
        resposta_correta=request.form.get('resposta_correta')
    )
    db.session.add(novo)
    db.session.commit()
    return redirect(url_for('listar_cursos'))

@app.route('/admin/cursos/editar/<int:id>', methods=['POST'])
def editar_curso(id):
    if 'usuario_logado' not in session: return redirect(url_for('login_page'))
    curso = db.session.get(Curso, id)
    if curso:
        curso.titulo = request.form.get('titulo')
        curso.descricao = request.form.get('descricao')
        curso.conteudo = request.form.get('conteudo')
        curso.link = request.form.get('link')
        curso.pergunta_teste = request.form.get('pergunta_teste')
        curso.resposta_correta = request.form.get('resposta_correta')
        db.session.commit()
    return redirect(url_for('listar_cursos'))

@app.route('/admin/cursos/deletar/<int:id>')
def deletar_curso(id):
    if 'usuario_logado' not in session: return redirect(url_for('login_page'))
    curso = db.session.get(Curso, id)
    if curso:
        db.session.delete(curso)
        db.session.commit()
    return redirect(url_for('listar_cursos'))
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

A operação de Create (criação) é realizada por meio da rota /admin/cursos/novo, onde para criação de novos cursos com os atributos título,

descrição, conteúdo, link de apoio, pergunta de avaliação e resposta esperada, preenchendo o banco de dados ao suceder a operação.

A operação de Read (leitura) é responsável pela consulta e exibição dos dados armazenados. Ela ocorre em diferentes partes do sistema, como na listagem geral de cursos e na visualização detalhada de cada curso individual, permitindo ao usuário acessar o conteúdo educacional de forma organizada.

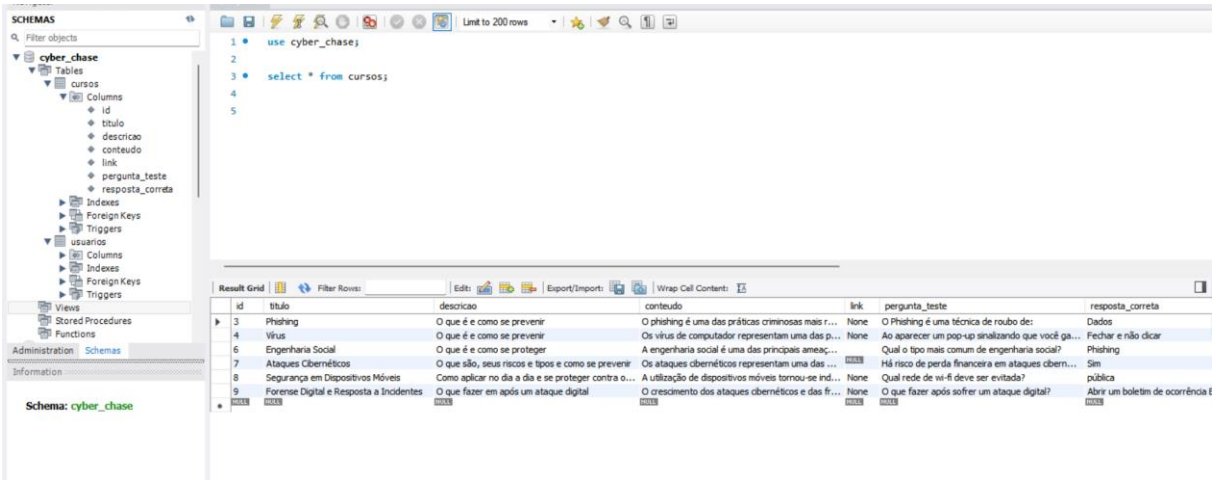
A operação de Update (atualização) é implementada na rota /admin/cursos/editar/<id>, para editar cursos existentes.

Por fim, a operação de Delete (exclusão) é realizada pela rota /admin/cursos/deletar/<id>, para excluir registros do banco de dados de forma definitiva.

2.4.5 Banco de dados

O banco de dados do sistema foi desenvolvido com o SGBD MySQL, integrado ao framework Flask por meio da biblioteca SQLAlchemy, que atua como um ORM (Object Relational Mapper), permitindo a manipulação das tabelas do banco de dados através de classes em Python, o sistema possui como principais entidades os usuários e os cursos, que representam os elementos centrais da aplicação.

Figura 13: trecho da estrutura do banco de dados no MySQL Workbench



Fonte: captura de tela do banco de dados desenvolvido para o presente projeto (2026)

Na imagem é possível notar ao lado esquerdo a estrutura do banco de dados da Cyber Chase criado no MySQL, onde há as tabelas cursos e usuarios. Na tela principal, há uma consulta de todos os registros da tabela de cursos por meio de um comando select, e um output onde se nota o resultado da linha de código, onde se observa as colunas da tabela e seus respectivos conteúdos. Essa tabela é chamada no código em python como na seguinte imagem:

Figura 14: trecho do código em python que faz a conexão com o banco de dados no MySQL

```python
#banco de dados em mysql
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+mysqlconnector://root:root@127.0.0.1:3306/cyber_chase'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

#models bd
...
class Usuario(db.Model):
    __tablename__ = 'usuarios'
    id = db.Column(db.Integer, primary_key=True)
    nome = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)
    senha = db.Column(db.String(255), nullable=False)

...
class Curso(db.Model):
    __tablename__ = 'cursos'
    id = db.Column(db.Integer, primary_key=True)
    titulo = db.Column(db.String(100), nullable=False)
    descricao = db.Column(db.Text, nullable=False)
    conteudo = db.Column(db.Text, nullable=True)
    link = db.Column(db.String(900), nullable=True)
    pergunta_teste = db.Column(db.Text, nullable=True)
    resposta_correta = db.Column(db.Text, nullable=True)

with app.app_context():
    db.create_all()
```

Fonte: captura de tela do código desenvolvido para o presente projeto

A base de dados é composta principalmente pelas tabelas usuarios e cursos, que foram definidas por meio de classes no SQLAlchemy.

A tabela usuarios é responsável pelo armazenamento das informações de autenticação e identificação dos usuários da plataforma, com os campos de id como chave primária e identificador único do usuário, nome para armazenar o nome completo do usuário, email como campo único de identificação de login e senha com a credencial.

Já a tabela cursos representa os conteúdos educacionais disponibilizados na plataforma, com os campos de id como chave primária e identificador único do curso, título do curso, descrição breve para aparecer nos cards, conteúdo com o material do curso, link como referência externa se necessário, pergunta_teste como questão utilizada para avaliação do aprendizado e resposta_correta como resposta ideal da pergunta parametrizada.

O SQLAlchemy foi utilizado como camada de abstração entre a aplicação e o banco de dados MySQL, por meio do qual as tabelas são representadas como classes Python, e os registros como objetos, onde é possível realizar operações de inserção,

consulta, atualização e exclusão de forma orientada a objetos, além de gerenciar a conexão com o banco de dados.

No código, para fins de inicialização, criação das tabelas é realizada automaticamente por meio do comando db.create_all().

Atualmente, não há relacionamento direto entre as tabelas, porém a arquitetura adotada foi projetada para suportar facilmente a implementação de chaves estrangeiras, como implementações futuras no sistema.

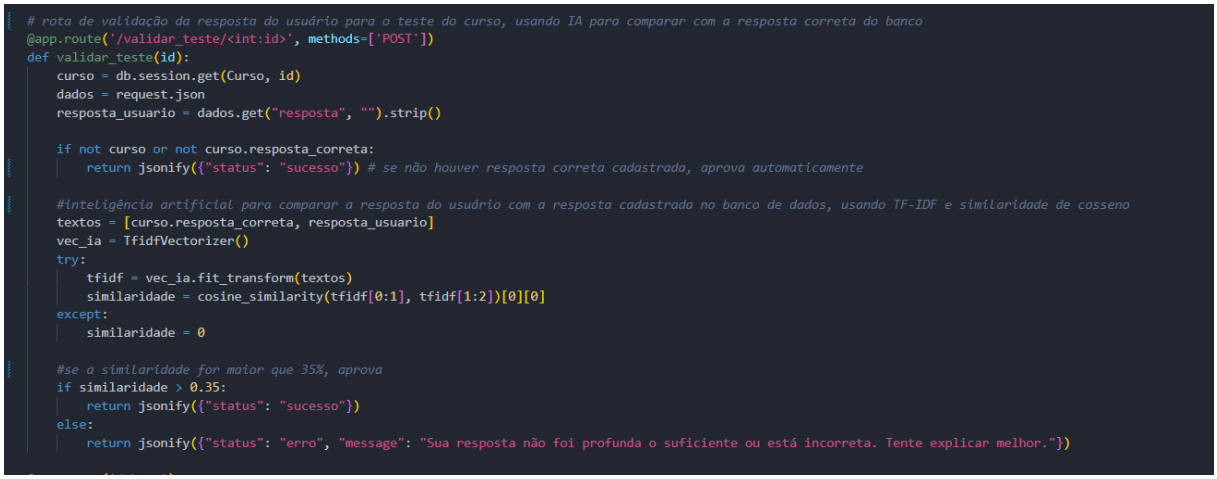
2.5 Inteligência Artificial e Machine Learning

O sistema desenvolvido incorpora técnicas de Inteligência Artificial (IA) e Machine Learning (ML) deixando a experiência mais interativa, inteligente e adaptativa ao comportamento do usuário, tais tecnologias foram aplicadas em duas funcionalidades principais: a validação automática das respostas dos testes dos cursos e o chatbot inteligente (Cybot), da interação com o usuário. Com isso, o sistema interpreta textos com base em similaridade semântica, utilizando técnicas de Processamento de Linguagem Natural (PLN), e o método TF-IDF (Term Frequency – Inverse Document Frequency) junto à métrica de similaridade de cosseno (cosine similarity), de modo a transformar textos em representações numéricas, ou seja, vetores matemáticos, de modo que o sistema compare diferentes entradas, onde respostas e perguntas podem ser analisadas pelo contexto e significado aproximado além da correspondência exata.

2.5.1 Aplicação de Inteligência Artificial na Validação de Respostas dos Cursos

Uma das aplicações de machine learning e inteligência artificial no sistema é a validação automática das respostas dos testes dos cursos, cujo objetivo é avaliar as respostas fornecidas pelos usuários de forma semântica.

Figura 15: trecho do código de uso de Machine Learning e Inteligência Artificial para validar a resposta enviada pelo usuário com a cadastrada no banco de dados para cada curso.



```python
# rota de validação da resposta do usuário para o teste do curso, usando IA para comparar com a resposta correta do banco
@app.route('/validar_teste/<int:id>', methods=['POST'])
def validar_teste(id):
    curso = db.session.get(Curso, id)
    dados = request.json
    resposta_usuario = dados.get("resposta", "").strip()

    if not curso or not curso.resposta_correta:
        return jsonify({"status": "sucesso"}) # se não houver resposta correta cadastrada, aprova automaticamente

    #inteligência artificial para comparar a resposta do usuário com a resposta cadastrada no banco de dados, usando TF-IDF e similaridade de cosseno
    textos = [curso.resposta_correta, resposta_usuario]
    vec_ia = TfidfVectorizer()
    try:
        tfidf = vec_ia.fit_transform(textos)
        similaridade = cosine_similarity(tfidf[0:1], tfidf[1:2])[0][0]
    except:
        similaridade = 0

    #se a similaridade for maior que 35%, aprova
    if similaridade > 0.35:
        return jsonify({"status": "sucesso"})
    else:
        return jsonify({"status": "erro", "message": "Sua resposta não foi profunda o suficiente ou está incorreta. Tente explicar melhor."})
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

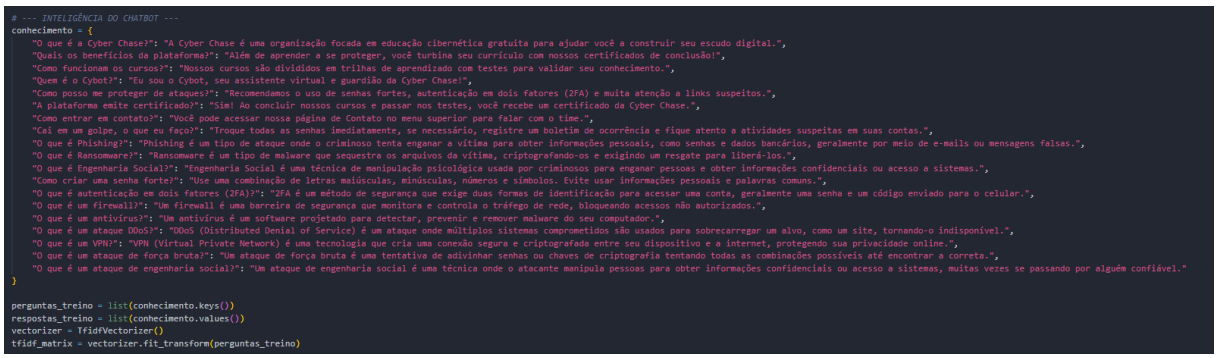
Para isso, foi utilizada a técnica de Processamento de Linguagem Natural (PLN) com o algoritmo TF-IDF (Term Frequency – Inverse Document Frequency) em conjunto com o cálculo de similaridade com cosseno (cosine similarity). Assim, a resposta correta armazenada no banco de dados e a resposta enviada pelo usuário são transformadas em vetores por meio do TfidfVectorizer para que seja calculada a similaridade entre esses vetores. O resultado dessa comparação gera um valor numérico entre 0 e 1, no qual valores mais próximos de 1 indicam maior semelhança entre os textos. E o critério que define se será 0 ou 1 foi definido por um range de 0,35 (35% de similaridade), no qual, se a resposta do usuário atingir ou ultrapassar esse valor, é considerada correta pelo sistema, sem a necessidade de ser totalmente idêntica à resposta esperada, senão, o sistema retorna uma mensagem informando que a resposta não foi suficientemente adequada, incentivando o aluno a reavaliar o conteúdo enviado.

2.5.2 Aplicação de Inteligência Artificial no Chatbot

Outra aplicação de IA e ML no sistema está presente no desenvolvimento do Cybot (chatbot), que interage com usuários da plataforma de forma automatizada.

O chatbot foi desenvolvido com técnicas de Processamento de Linguagem Natural (PLN), com o modelo de vetorização TF-IDF e com o cálculo de similaridade de cosseno, de modo que que o sistema interprete perguntas feitas em linguagem natural e retorne respostas mais adequadas com base em um conjunto de conhecimento previamente definido.

Figura 16: dados de treino do chatbot (Machine Learning)



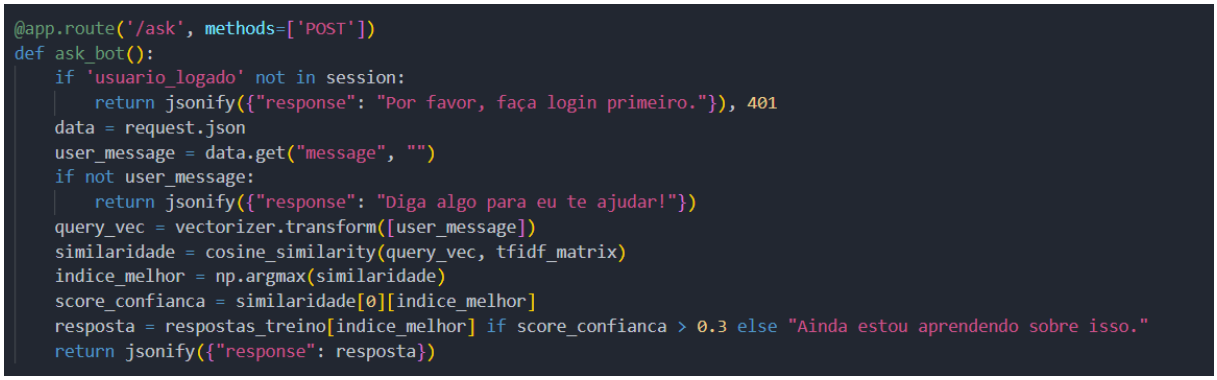
```
# --- INTELIGÊNCIA DO CHATBOT ---
conhecimento = {
    "O que é a Cyber Chase?": "A Cyber Chase é uma organização focada em educação cibernética gratuita para ajudar você a construir seu escudo digital.",
    "Quais os benefícios da plataforma?": "Além de aprender a se proteger, você turbina seu currículo com nossos certificados de conclusão!",
    "Como funcionam os cursos?": "Nossos cursos são divididos em trilhas de aprendizado com testes para validar seu conhecimento.",
    "Quem é o Cybot?": "Eu sou o Cybot, seu assistente virtual e guardião da Cyber Chase!",
    "Como posso me proteger de ataques?": "Recomendamos o uso de senhas fortes, autenticação em dois fatores (2FA) e muita atenção a links suspeitos.",
    "A plataforma emite certificado?": "Sim! Ao concluir nossos cursos e passar nos testes, você recebe um certificado da Cyber Chase.",
    "Como entrar em contato?": "Você pode acessar nossa página de Contato no menu superior para falar com o time.",
    "Caí em um golpe, o que eu faço?": "Troque todas as senhas imediatamente, se necessário, registre um boletim de ocorrência e fique atento a atividades suspeitas em suas contas.",
    "O que é Phishing?": "Phishing é um tipo de ataque onde o criminoso tenta enganar a vítima para obter informações pessoais, como senhas e dados bancários, geralmente por meio de e-mails ou mensagens falsas.",
    "O que é Ransomware?": "Ransomware é um tipo de malware que sequestra os arquivos da vítima, criptografando-os e exigindo um resgate para liberá-los.",
    "O que é Engenharia Social?": "Engenharia Social é uma técnica de manipulação psicológica usada por criminosos para enganar pessoas e obter informações confidenciais ou acesso a sistemas.",
    "Como criar uma senha forte?": "Use uma combinação de letras maiúsculas, minúsculas, números e símbolos. Evite usar informações pessoais e palavras comuns.",
    "O que é autenticação em dois fatores (2FA)?": "2FA é um método de segurança que exige duas formas de identificação para acessar uma conta, geralmente uma senha e um código enviado para o celular.",
    "O que é um firewall?": "Um firewall é uma barreira de segurança que monitora e controla o tráfego de rede, bloqueando acessos não autorizados.",
    "O que é um antivírus?": "Um antivírus é um software projetado para detectar, prevenir e remover malware do seu computador.",
    "O que é um ataque DDoS?": "DDoS (Distributed Denial of Service) é um ataque onde múltiplos sistemas comprometidos são usados para sobrecarregar um alvo, como um site, tornando-o indisponível.",
    "O que é um VPN?": "VPN (Virtual Private Network) é uma tecnologia que cria uma conexão segura e criptografada entre seu dispositivo e a internet, protegendo sua privacidade online.",
    "O que é um ataque de força bruta?": "Um ataque de força bruta é uma tentativa de adivinhar senhas ou chaves de criptografia tentando todas as combinações possíveis até encontrar a correta.",
    "O que é um ataque de engenharia social?": "Um ataque de engenharia social é uma técnica onde o atacante manipula pessoas para obter informações confidenciais ou acesso a sistemas, muitas vezes se passando por alguém confiável."
}

perguntas_treino = list(conhecimento.keys())
respostas_treino = list(conhecimento.values())
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(perguntas_treino)
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

O chatbot funciona com conjunto de pares de perguntas e respostas armazenadas em uma estrutura de dados do tipo dicionário, que são utilizadas como conjunto de treinamento para o modelo, para que sejam transformadas em vetores com o TfidfVectorizer, então, quando o usuário envia uma mensagem, o sistema converte essa entrada em um vetor e calcula sua similaridade em relação às perguntas previamente treinadas e a partir desse cálculo, é identificada a pergunta mais semelhante, e sua respectiva resposta é retornada ao usuário.

Figura 17: cálculo de similaridade da pergunta enviada pelo usuário com os dados de treinamento previamente concebidos ao chatbot.



```
@app.route('/ask', methods=['POST'])
def ask_bot():
    if 'usuario_logado' not in session:
        return jsonify({"response": "Por favor, faça login primeiro."}), 401
    data = request.json
    user_message = data.get("message", "")
    if not user_message:
        return jsonify({"response": "Diga algo para eu te ajudar!"})
    query_vec = vectorizer.transform([user_message])
    similaridade = cosine_similarity(query_vec, tfidf_matrix)
    indice_melhor = np.argmax(similaridade)
    score_confianca = similaridade[0][indice_melhor]
    resposta = respostas_treino[indice_melhor] if score_confianca > 0.3 else "Ainda estou aprendendo sobre isso."
    return jsonify({"response": resposta})
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

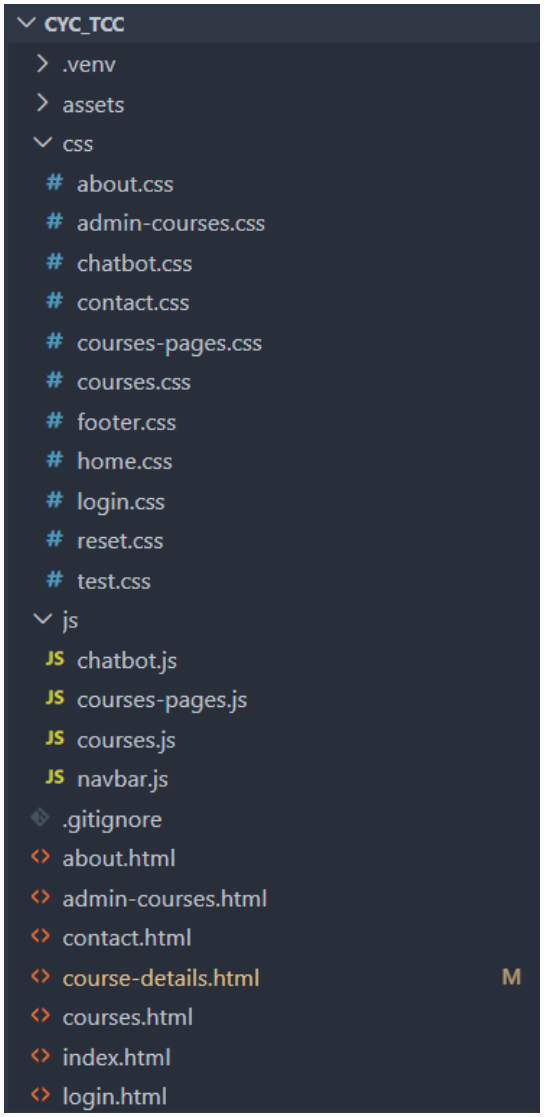
Para garantir maior precisão nas respostas, foi definido um range de confiança de 0,3 (30%), e qualquer similaridade menor faz com que o sistema retorne uma mensagem informando que ainda não possui conhecimento suficiente sobre o

assunto, de modo que o chatbot ofereça respostas coerentes com base na similaridade semântica entre as frases, mesmo quando há variações na forma de escrita ou estrutura das perguntas.

2.7 Desenvolvimento Front-End

O front-end foi desenvolvido utilizando HTML para compor a estrutura do projeto, CSS para estilizar e alguns trechos em JavaScript aplicado diretamente na estrutura do Front-end para atribuir funcionalidades sem a necessidade de incluí-lo no mesmo arquivo de Back-end em Python.

Figura 18: estrutura de pastas e arquivos do Front-End



Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

Para a estilização, apesar de haver diversos arquivos em CSS contendo as especificidades de cada página, todos eles seguem essa mesma estrutura geral exemplificada na imagem a seguir:

Figura 19: trecho de código base para a estilização da aplicação

```css
body {
    margin: 0;
    padding: 0;
    font-family: 'Geologica', sans-serif;
    background-color: #0a0a0a;
    color: #ffffff
}

.navbar {
    background-color: #1f1f1f;
    display: flex;
    justify-content: space-between;
    align-items: center;
    height: 11vh;
}

#Logo {
    height: 15vh;
    margin-right: 3vh;
}

.nav-links {
    list-style: none;
    display: flex;
    justify-content: flex-start;
    margin-right: auto;
}

.nav-links li {
    letter-spacing: 0.3vh;
    margin-right: 5vh;
}

.nav-links li a {
    text-decoration: none;
    color: #fff;
    transition: color 0.3s;
}

.nav-links li a:hover {
    color: #2291F6;
}
```

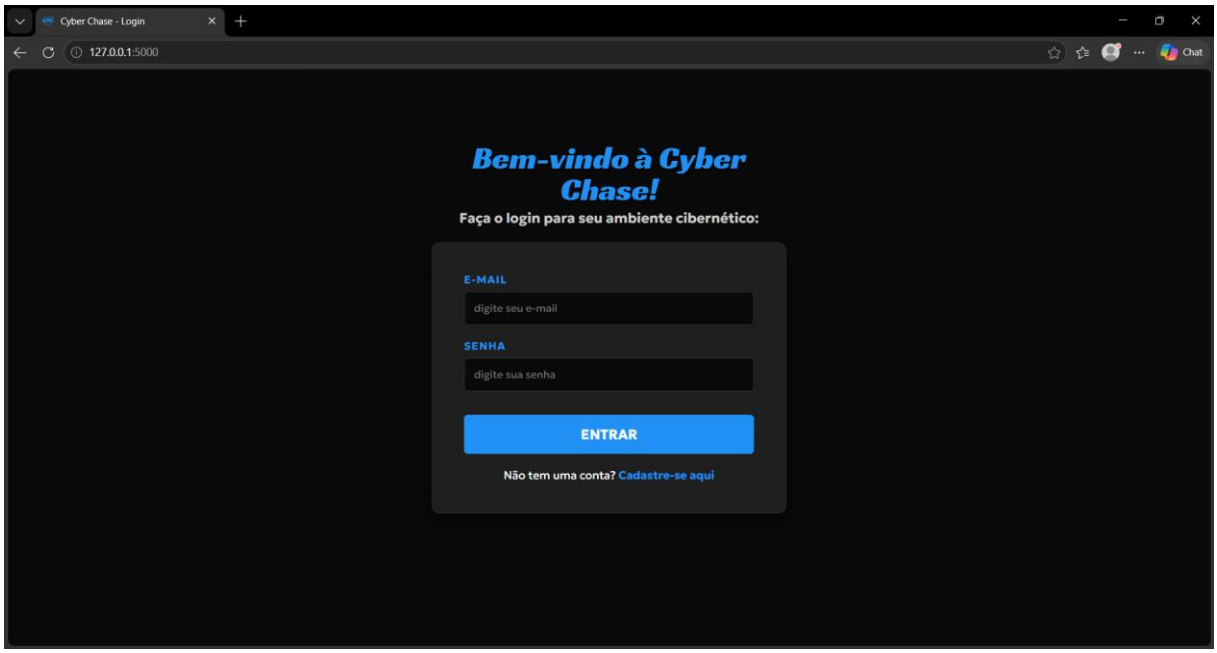
Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

Ao navegar pela plataforma, se nota a presença das fontes e cores contidas no padrão visual estabelecido.

2.7.1 Interface de login

Ao entrar no site, a primeira tela exibida é a de login, onde o usuário pode inserir suas credenciais previamente cadastradas e clicar no botão de entrar para aceder ao site.

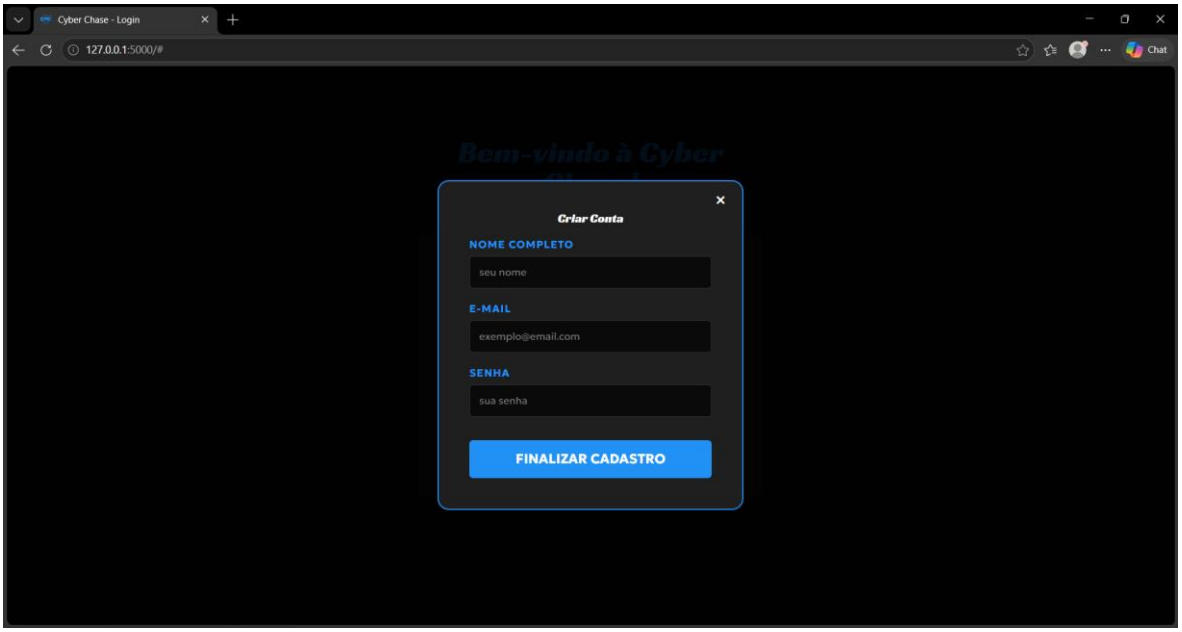
Figura 20: tela de login



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Caso o usuário não possua uma conta cadastrada, é possível clicar no botão de “Cadastre-se aqui”, que abre uma modal onde se insere o nome completo, e-mail e senha.

Figura 21: tela de cadastro de usuários



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Figura 22: trecho do código login.html

```
        <h1 class="welcome-title">Bem-vindo à Cyber Chase!</h1>
        <h2 id="slogan">Faça o login para seu ambiente cibernético:</h2>

        <div id="form_frente">
            <form action="/login" id="login" method="post">
                <label for="nome">e-mail</label>
                <input type="email" name="nome" id="nome" required placeholder="digite seu e-mail">

                <label for="senha">senha</label>
                <input type="password" name="senha" id="senha" required placeholder="digite sua senha">

                <button type="submit" id="btn_login" class="courses-button">entrar</button>
            </form>

            <p style="color: white; text-align: center; margin-top: 20px; font-family: 'Geologica', sans-serif; font-size: 0.9em;">
                Não tem uma conta?
                <a href="#" onclick="abrirCadastro()" style="color: #2291F6; text-decoration: none; font-weight: bold; cursor: pointer;">Cadastre-se aqui</a>
            </p>
        </div>
    </main>      You, 17 hours ago • projeto quase pronto
</div>

<!--modal de cadastro-->
<div id="modalCadastro" class="modal">
    <div class="modal-content">
        <button onclick="fecharCadastro()" class="close-btn">&times;</button>
        <h2 style="color: white; font-family: 'Racing Sans One', cursive; text-align: center; margin-bottom: 20px;">Criar Conta</h2>

        <form action="/registrar" method="POST" id="form_registrar">
            <label>nome completo</label>
            <input type="text" name="nome_cadastro" required placeholder="seu nome">

            <label>e-mail</label>
            <input type="email" name="email_cadastro" required placeholder="exemplo@email.com">

            <label>senha</label>
            <input type="password" name="senha_cadastro" required placeholder="sua senha">

            <button type="submit" class="courses-button">finalizar cadastro</button>
        </form>
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

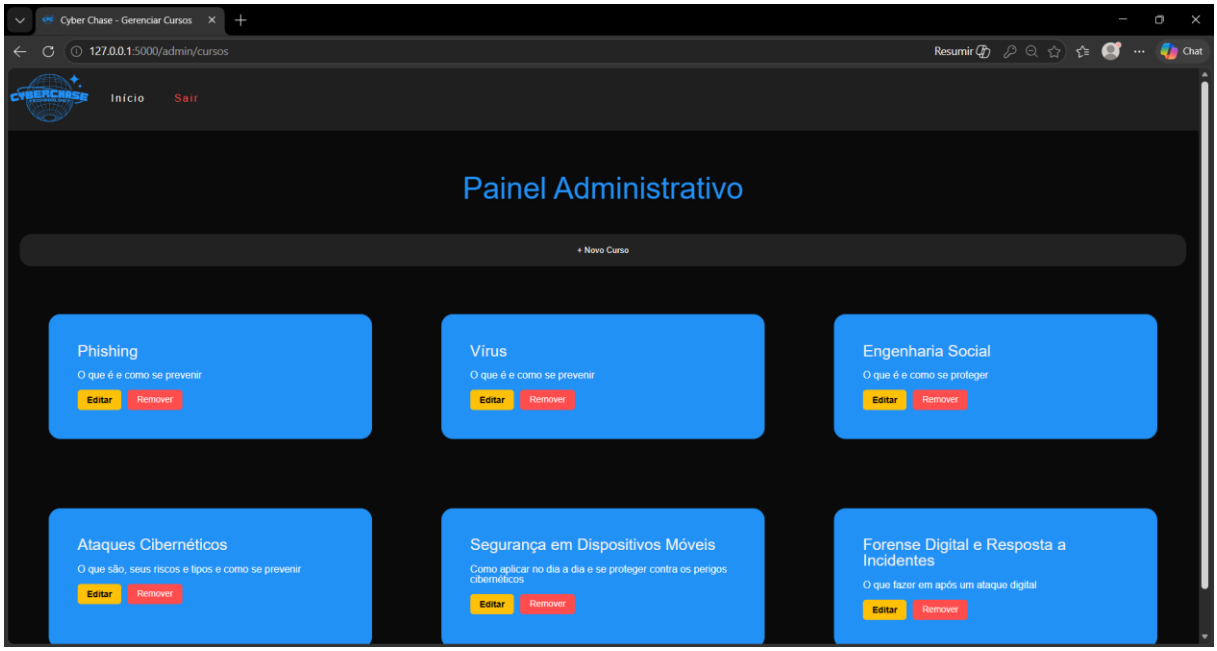
Na imagem codificada, é possível notar a presença, no mesmo arquivo em html, do campo de login e da modal de cadastro, com ações que são chamadas do código em back-end para aplicação da lógica de validação de dados para o login (action

/login) e preenchimento de dados no banco de dados para o cadastro (action /registrar).

2.7.2 Interface do Administrador

Por meio do login de administrador cujos parâmetros foram previamente apresentados na seção de back-end, é possível acessar um painel administrativo exclusivo com as opções de criação, edição e exclusão de cursos.

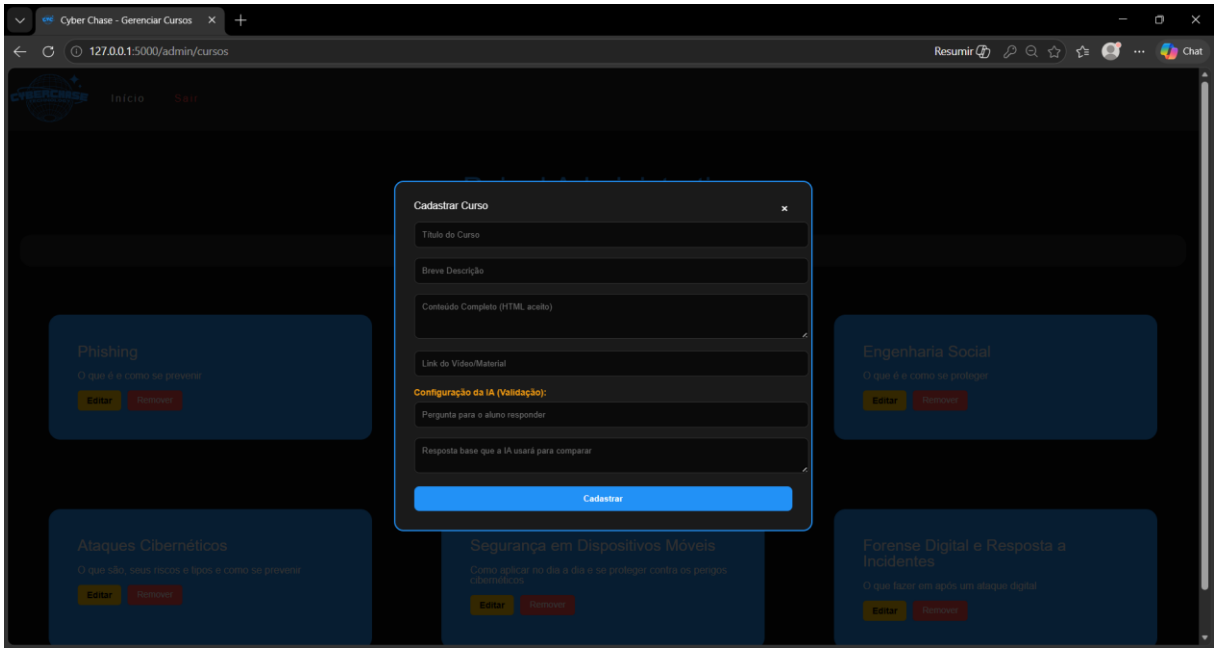
Figura 23: Interface do Painel Administrativo



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

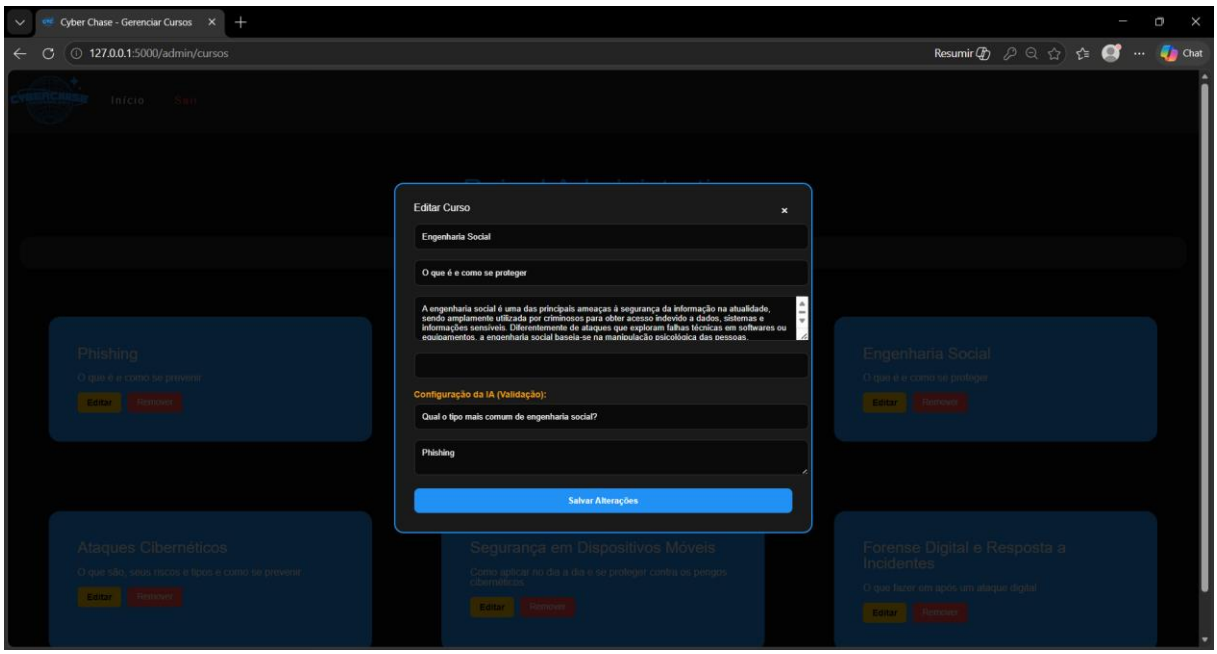
Para as ações de criação e edição de cursos, uma modal é aberta onde o administrador consegue realizar tais ações sem a necessidade de sair da página, como consta nas imagens a seguir:

Figura 24: modal de criação de cursos



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Figura 25: modal de edição de cursos



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Em ambos os casos, os campos preenchidos têm conexão com o back-end e banco de dados, de modo a garantir uma integração completa dos dados.

Figura 26: trecho do código em HTML do gerenciamento de cursos.



```
<!--MODAL NOVO CURSO-->
<div id="modalNovoCurso" class="modal">
    <div class="modal-content">
        <button onclick="this.parentElement.parentElement.style.display='none'" style="float:right; background:none; borde
        <h2>Cadastrar Curso</h2>
        <form action="/admin/cursos/novo" method="POST">
            <input type="text" name="titulo" placeholder="Título do Curso" required>
            <input type="text" name="descricao" placeholder="Breve Descrição" required>
            <textarea name="conteudo" rows="3" placeholder="Conteúdo Completo (HTML aceito)"></textarea>
            <input type="text" name="link" placeholder="Link do Vídeo/Material">

            <span class="label-ia">Configuração da IA (Validação):</span>
            <input type="text" name="pergunta_teste" placeholder="Pergunta para o aluno responder">
            <textarea name="resposta_correta" rows="2" placeholder="Resposta base que a IA usará para comparar"></textarea

            <button type="submit" class="btn-salvar">Cadastrar</button>
        </form>
    </div>
</div>

<!--MODAL EDITAR CURSO-->
<div id="modalEditarCurso" class="modal">
    <div class="modal-content">
        <button onclick="this.parentElement.parentElement.style.display='none'" style="float:right; background:none; borde
        <h2>Editar Curso</h2>
        <form id="formEditar" method="POST">
            <input type="text" name="titulo" id="edit_titulo" required>
            <input type="text" name="descricao" id="edit_descricao" required>
            <textarea name="conteudo" id="edit_conteudo" rows="3"></textarea>      You, 17 hours ago • projeto ok …
            <input type="text" name="link" id="edit_link">
```

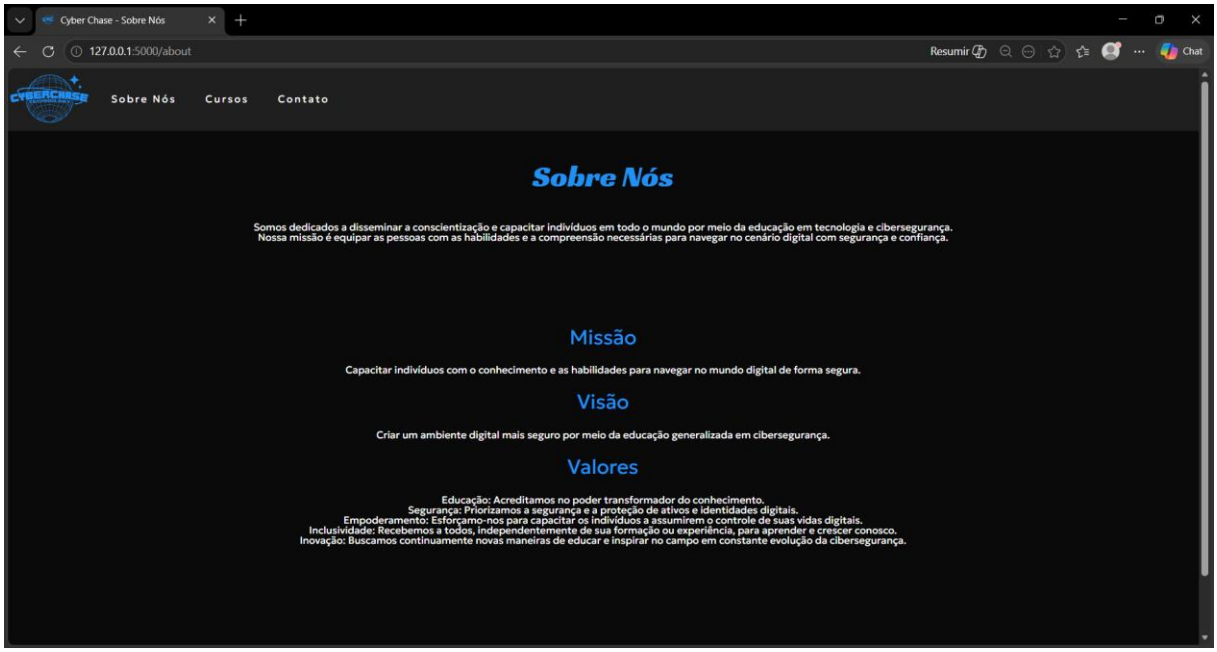
Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

No trecho acima, além de ser notável os campos correspondentes do banco de dados, nota-se uma chamada das ações que foram criadas no back-end para o CRUD.

2.7.3 Interface Sobre Nós

O site comporta uma página simples, porém com grande importância para pessoas que queiram conhecer mais sobre o objetivo da Cyber Chase com a sua missão de educar ciberneticamente.

Figura 27: interface Sobre Nós

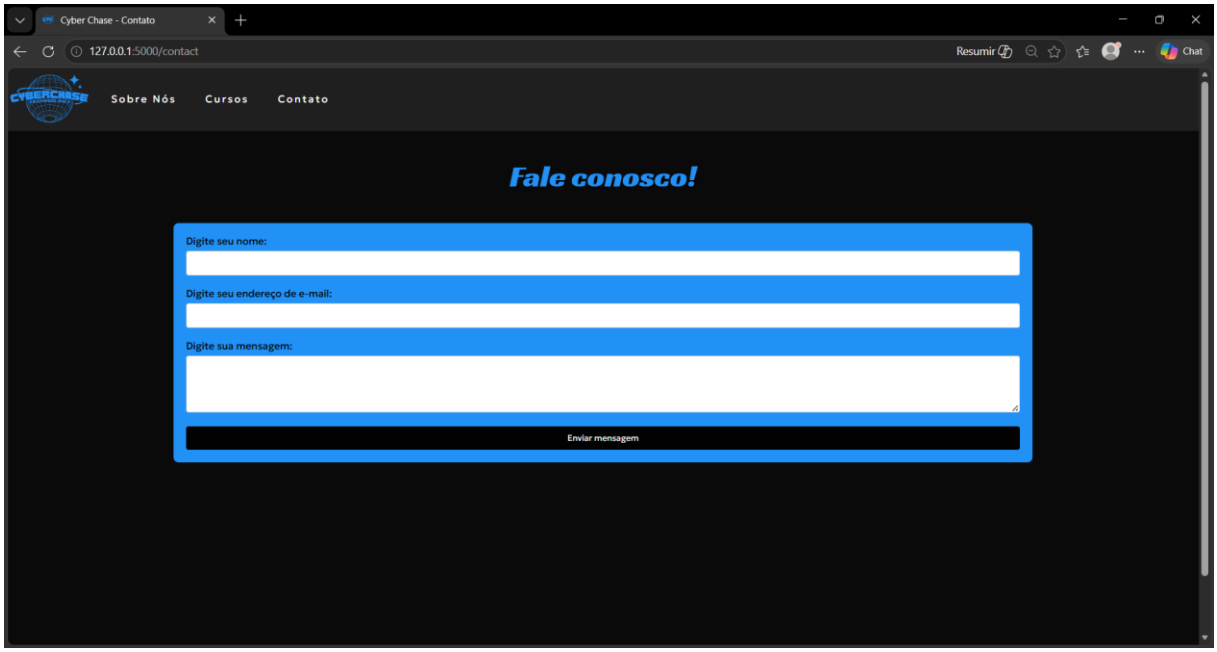


Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

2.7.4 Interface de Contato

Caso o usuário possua alguma dúvida não sanada pela navegação no site, queira dar sugestões ou contatar a equipe administradora por algum motivo, há uma página de Contato.

Figura 28: interface de Contato



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Nesta tela, o usuário pode preencher sua mensagem e suas informações de contato para que a equipe receba a mensagem e retorne com a devolutiva. O código abaixo trata o envio da mensagem para o e-mail de mensagens da Cyber Chase.

Figura 29: trecho do código em HTML

```
<main class="main">

    <h1 class="contact-title">Fale conosco!</h1>
    <div class="container">
        <form class="contact-form" action="https://formsubmit.co/cyber0chase@gmail.com" method="POST" data-form>
            <label for="nome">Digite seu nome: </label>
            <input type="nome" id="name" name="name" class="form-input" required>
            <label for="email">Digite seu endereço de e-mail: </label>
            <input type="email" id="email" name="email" class="form-input" required>
            <label for="message">Digite sua mensagem: </label>
            <textarea id="message" name="message" rows="4" class="form-textarea" required></textarea>
            <button type="submit" class="form-button" data-button>Enviar mensagem</button>

        </form>
    </div>

</main>
```

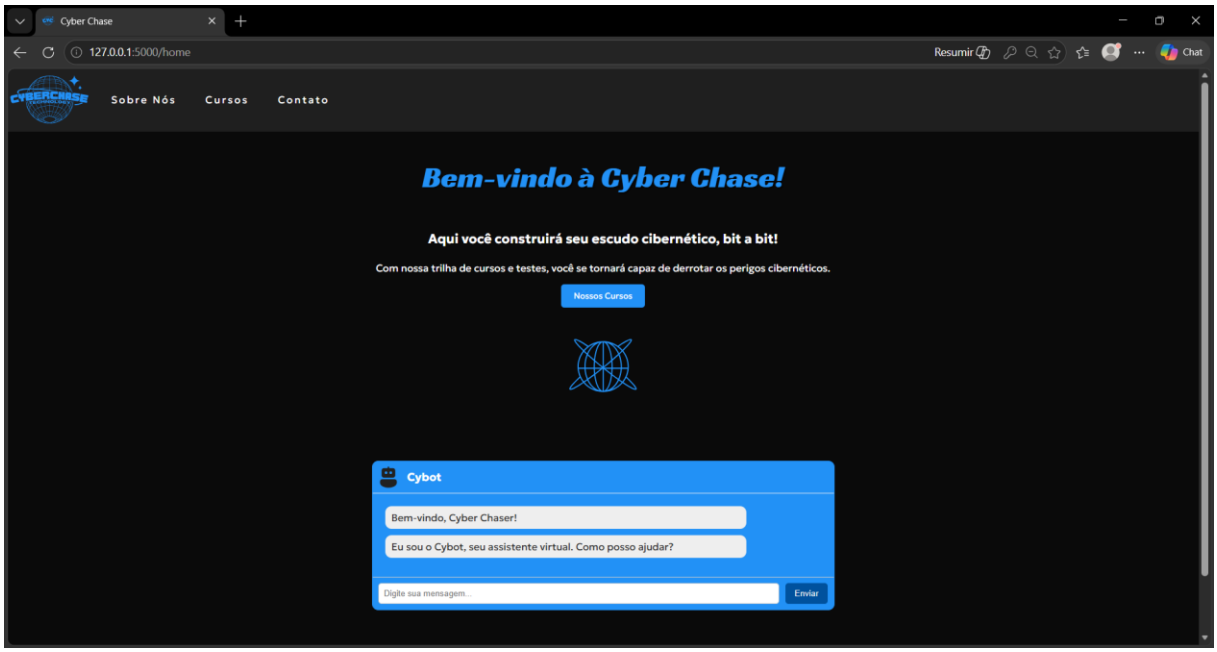
Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

2.7.5 Interface da home

Ao logar como usuário comum, ocorre um redirecionamento para a página principal da home, onde há a opção do usuário interagir com o chatbot, acessar os

cursos diretamente ou acessar outras páginas do site pela barra de navegação (navbar).

Figura 30: interface da home



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Esta página da plataforma foi desenvolvida com o apoio de um código em JavaScript para garantir o funcionamento completo do chatbot, como é possível notar a chamada do script no trecho do código em HTML a seguir.

Figura 31: trecho da estrutura HTML para a home

```
<main>
    <h1 class="welcome-title">Bem-vindo à Cyber Chase!</h1> <br>
    <h2 id="slogan">Aqui você construirá seu escudo cibernético, bit a bit!</h2>
    <h3 class="home-description">Com nossa trilha de cursos e testes, você se tornará capaz de derrotar
        os perigos cibernéticos.</h3>
    <a class="courses-button-a" href="/courses"><button class="courses-button">Nossos Cursos</button></a>
    <div id="cyc-icon1"> <img src="assets/icon/svg/cyc-icon1.svg"> </div>
</main>

<div id="chatbot-container" class="chatbot-container">
    <div class="chatbot-header">
        <img src="assets/icon/svg/shy-bot.svg" class="bot-avatar" alt="Avatar do Bot">
        <span class="bot-name">Cybot</span>
    </div>
    <div class="chatbot-body">
        <div class="chatbot-body" id="chatbot-body">
            <div class="bubble bot-bubble">Bem-vindo, Cyber Chaser!</div>
            <div class="bubble bot-bubble">Eu sou o Cybot, seu assistente virtual. Como posso ajudar?</div>
        </div>
    </div>
    <div class="chatbot-footer">
        <input type="text" class="user-input" placeholder="Digite sua mensagem...">
        <button class="send-button" onclick="sendMessage()">Enviar</button>
    </div>
</div>

<script src="js/chatbot.js"></script>
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

O código parametrizado no back-end em Python apresentado previamente neste trabalho tem como função a utilização de Machine Learning e Inteligência Artificial aplicados aos dados de treino do chatbot, enquanto o arquivo em JavaScript chamado neste HTML tem como função garantir as animações dinâmicas das caixas de texto durante a interação chatbot-usuário, simulando um ambiente de troca de mensagens dinâmico e moderno, conforme imagem a seguir:

Figura 32: trecho do código em JavaScript para auxiliar no funcionamento do chatbot da home

```
const chatBody = document.getElementById('chatbot-body');
const userInput = document.querySelector('.user-input');

//função principal para enviar mensagem
async function sendMessage() {
    const message = userInput.value.trim();
    if (message === "") return;

    //mostrar mensagem do usuário
    addBubble(message, 'user-bubble');
    userInput.value = "";

    try {
        //enviar para o Flask (main.py)
        const response = await fetch('/ask', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ message: message })
        });

        const data = await response.json();

        //mostrar resposta do Cybot
        addBubble(data.response, 'bot-bubble');

        //se a resposta sugerir opções, podemos criar botões (opcional)
        if (message.toLowerCase().includes("ajuda") || message.toLowerCase().includes("oi")) {
            showOptions();
        }

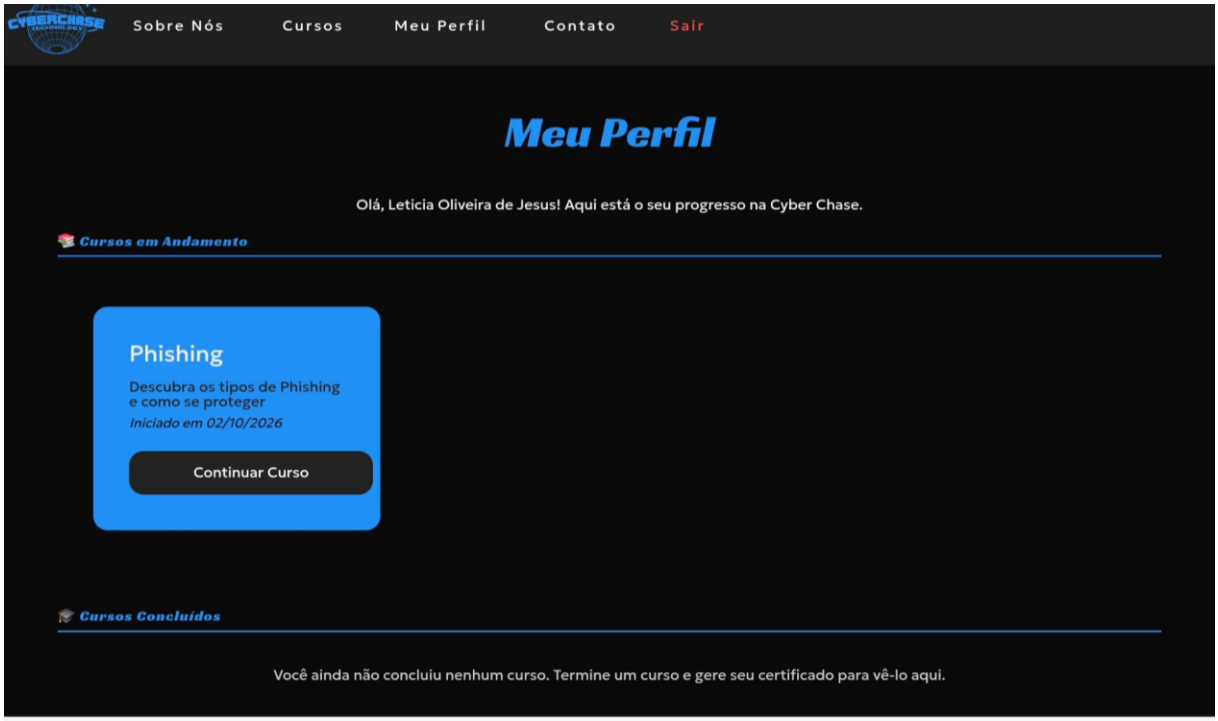
    } catch (error) {
        addBubble("Erro de conexão com o servidor.", 'bot-bubble');
    }
}
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

2.7.6 Interface “Meu Perfil”

O usuário possui a opção de acessar seu perfil e acompanhar um painel onde pode consultar seus cursos em andamento e seus cursos concluídos, bem como os certificados de tais últimos citados, além da opção habilitada para deslogar, conforme imagem a seguir:

Figura 33: interface Meu Perfil



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

A interface Meu Perfil verifica no Banco de Dados os cursos que o usuário iniciou e clusteriza-os na seção de andamento, enquanto os cursos para os quais houve certificado gerado pelo usuário cadastrado no Banco de Dados são exibidos na seção de Cursos Concluídos.

Figura 34: trecho do código da interface Meu Perfil

perfil.html

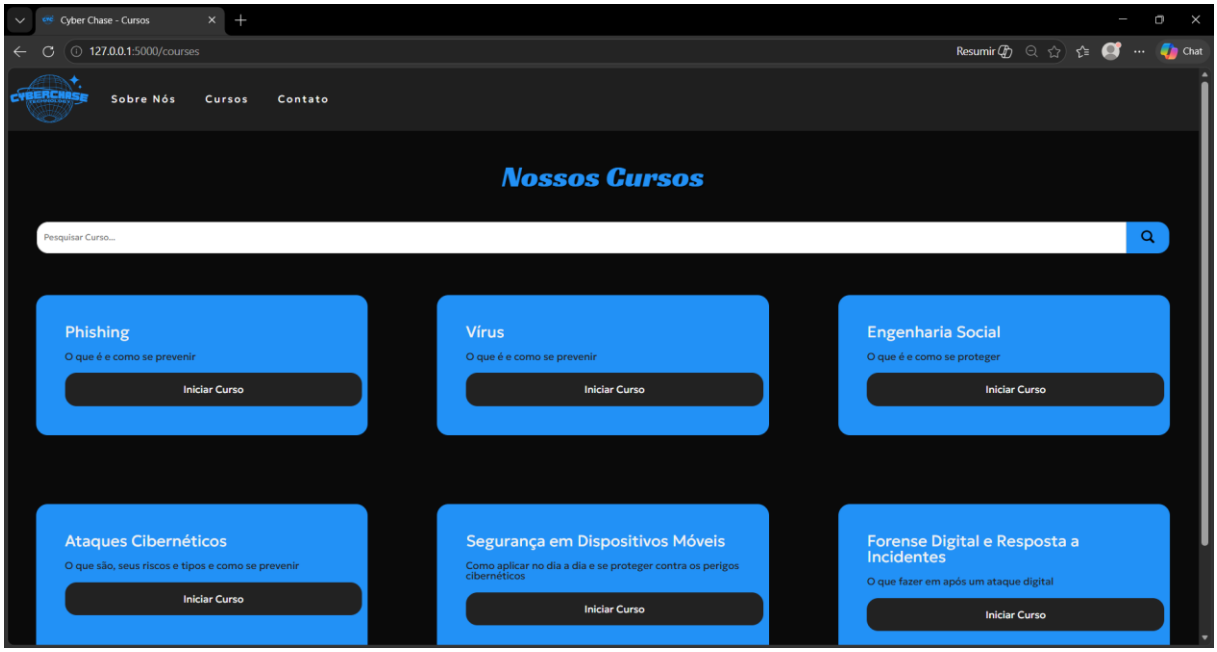
```
49                <ul class="course-list profile-course-list">
50                    {% for mat in em_andamento %}
51                    <li class="course-item">
52                        <h2 class="course-title">{{ mat.curso.titulo }}</h2>
53                        <p class="course-description">{{ mat.curso.descricao }}</p>
54                        <p class="profile-meta">Iniciado em {{ mat.data_inicio.strftime('%d/%m/%Y') if mat.data_inicio else '-' }}</p>
55                        <a href="{{ url_for('visualizar_curso', id=mat.curso.id) }}" class="start-button" style="
text-decoration: none; display: inline-block; text-align: center;">
56                            Continuar Curso
57                        </a>
58                    </li>
59                    {% endfor %}
60                </ul>
61                {% else %}
62                <p class="empty-msg">Você ainda não iniciou nenhum curso. <a href="/courses">Veja os cursos disponíveis</a>.</p>
63                {% endif %}
64            </section>
65
66            <!-- CURSOS CONCLUÍDOS -->
67            <section class="profile-section">
68                <h2 class="section-title">🎓 Cursos Concluídos</h2>
69
70                {% if concluidos %}
71                <ul class="course-list profile-course-list">
72                    {% for mat in concluidos %}
73                    <li class="course-item course-item-done">
74                        <h2 class="course-title">{{ mat.curso.titulo }}</h2>
75                        <p class="course-description">{{ mat.curso.descricao }}</p>
76                        <p class="profile-meta">Concluído em {{ mat.data_conclusao.strftime('%d/%m/%Y') if mat.data_conclusao else
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

2.7.7 Interface de Cursos

Ao optar por acessar a página de cursos, o usuário é direcionado para um catálogo com todos os cursos da Cyber Chase, além de uma barra de pesquisa para que ele possa pesquisar por temas de seu interesse e facilitar a busca.

Figura 35: interface de Cursos



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Cada card de curso existente é consumido do banco de dados e apresentado na tela para o usuário, conforme é possível notar no trecho HTML a seguir os campos da tabela de cursos do banco de dados sendo chamados para consumo de seus dados:

Figura 36: trecho da estrutura HTML da página de cursos.

```
<main>
    <h1 class="page-title">Nossos Cursos</h1>

    <div class="search-bar">
        <input id="searchbar" type="text" placeholder="Pesquisar Curso..." onkeyup="search()">
        <button type="submit">
            <img src="/assets/icon/svg/magnifier1.svg" alt="Ícone de Busca" class="search-img">
        </button>
    </div>

    <ul class="course-list">
        {% for curso in cursos %}
        <li class="course-item">
            <h2 class="course-title">{{ curso.titulo }}</h2>
            <p class="course-description">{{ curso.descricao }}</p>

            <!--link estilizado como botão -->
            <a href="{{ url_for('visualizar_curso', id=curso.id) }}" class="start-button" style="text-decoration: none;
                Iniciar Curso
            </a>
        </li>
        {% endfor %}
    </ul>
</main>
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

Além disso, há uma chamada para uma função “search()” da barra de pesquisa, que é criada e consumida do arquivo em JavaScript a seguir:

Figura 37: código em JavaScript para a barra de pesquisa da página de cursos

```
JS courses.js
function search() {
    let input = document.getElementById('searchbar').value;
    input = input.toLowerCase();
    let x = document.getElementsByClassName('course-item');

    for (let i = 0; i < x.length; i++) {
        if (!x[i].innerHTML.toLowerCase().includes(input)) {
            x[i].style.display = "none";
        } else {
            x[i].style.display = "block";
        }
    }
}

document.getElementById('searchbar').addEventListener('input', function() {
    if (this.value === "") {
        let x = document.getElementsByClassName('course-item');
        for (let i = 0; i < x.length; i++) {
            x[i].style.display = "block";
        }
    } else {
        search();
    }
});
```

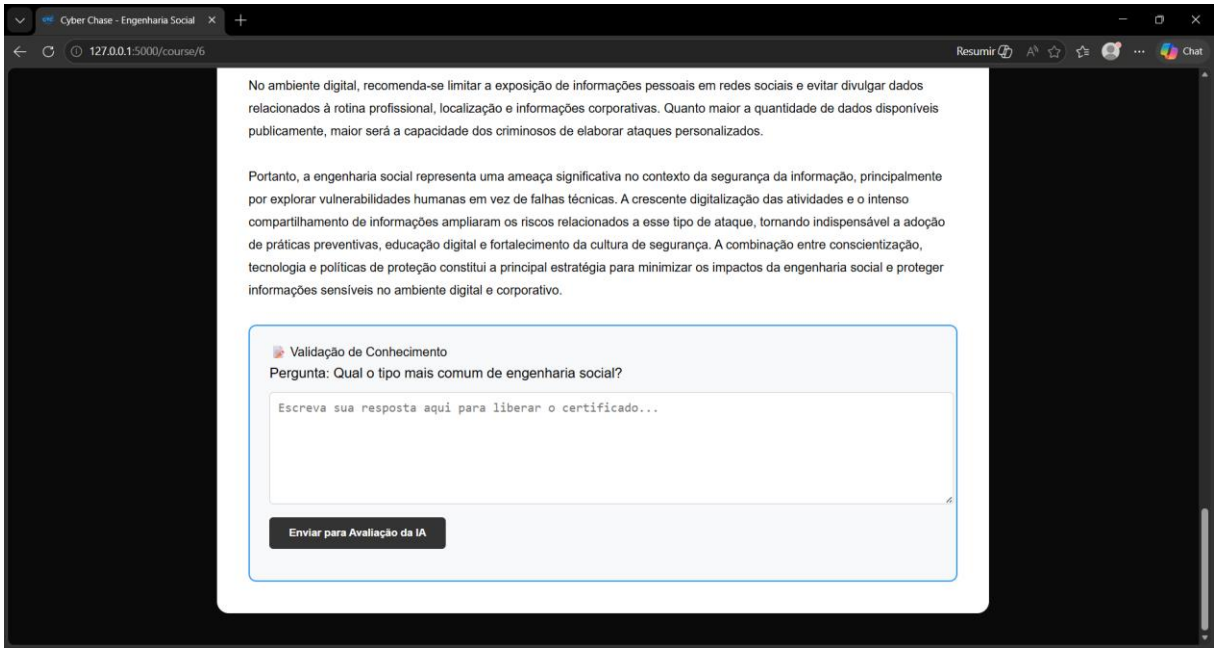
Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

Nesta função, utiliza-se um vetor para percorrer as letras pesquisadas e comparar com as letras dos cursos disponíveis, retornando os resultados que convergirem ao usuário. Em caso de total divergência, um vazio preenche a tela, de modo que fica claro para o usuário que não há correspondências para sua pesquisa.

2.7.8 Página dos cursos

Apesar de cada curso possuir suas especificidades de conteúdo, há uma funcionalidade em comum em todas as páginas de cursos: a validação do aprendizado por meio de uma pergunta antes da liberação do certificado.

Figura 38: Exemplo de aplicação de pergunta na página do curso.



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Figura 38: script adicionado para validar a resposta enviada pelo usuário.

```
<script>
    function validarIA() {
        const resp = document.getElementById('resposta_aluno').value;
        const feedback = document.getElementById('feedback');

        if(resp.length < 10) {
            feedback.innerText = "⚠️ Por favor, escreva uma resposta mais detalhada.";
            feedback.style.color = "orange";
            return;
        }

        fetch('/validar_teste/{{ curso.id }}', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({resposta: resp})
        })
        .then(r => r.json())
        .then(data => {
            if(data.status === "sucesso") {
                feedback.innerText = "✅ Resposta aprovada! Sua compreensão foi validada.";
                feedback.style.color = "green";
                document.getElementById('btnGerarCertificado').style.display = "block";
                document.getElementById('sessaoTeste').style.opacity = "0.6";
            } else {
                feedback.innerText = "❌ " + data.message;
                feedback.style.color = "red";
            }
        });
    }
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

Ao fornecer a resposta, ela é automaticamente analisada pela Inteligência Artificial parametrizada e, em caso de sucesso na resposta, é habilitada a função de geração de certificado do curso em questão para o usuário.

Figura 39: código em HTML para habilitar o certificado



```
    <div class="test-section" id="sessaoTeste">
        <h3> Validação de Conhecimento</h3>
        <p><strong>Pergunta:</strong> {{ curso.pergunta_teste or "Explique resumidamente o que você aprendeu neste módulo." }}
        <textarea id="resposta_aluno" placeholder="Escreva sua resposta aqui para liberar o certificado..."></textarea>
        <button class="btn-action btn-verify" onclick="validarIA()">Enviar para Avaliação da IA</button>
        <p id="feedback" style="margin-top: 15px; font-weight: bold;"></p>
    </div>

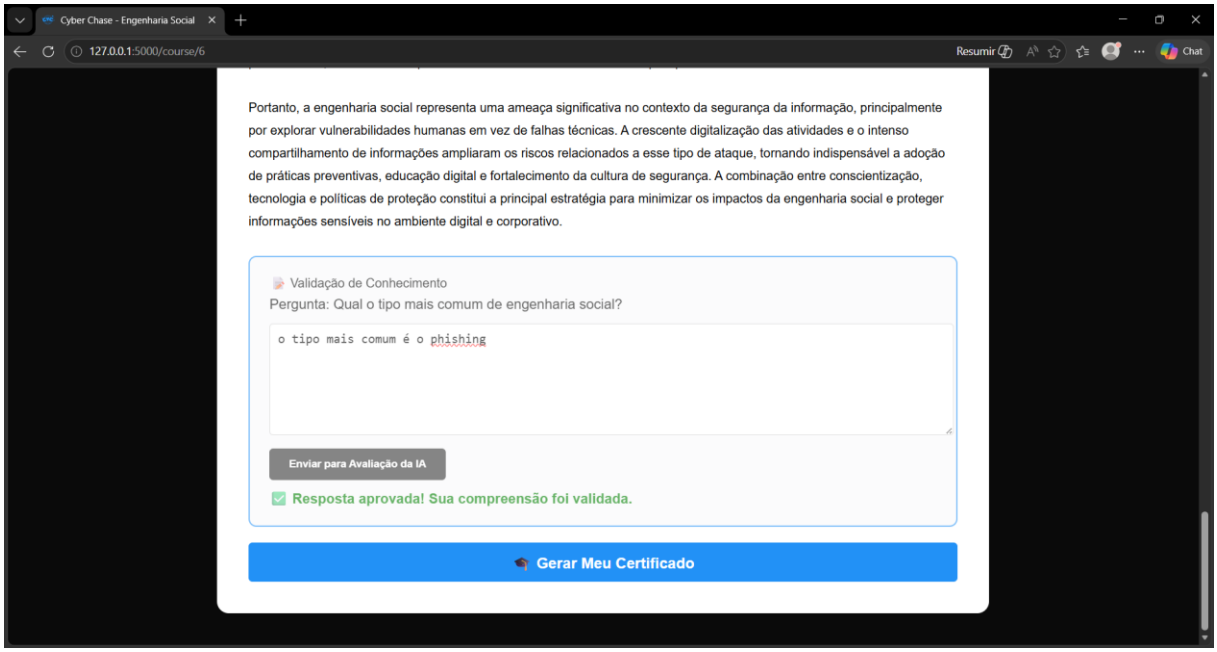
    <button id="btnGerarCertificado" class="btn-action btn-cert" onclick="liberarCertificado()"> Gerar Meu Certificado</butt

    <section id="areaCertificado">
        <div class="cert-box">
            <img src="/assets/logo/svg/Logo.svg" style="width: 150px; margin-bottom: 20px;">
            <h1 style="font-size: 3.5rem; font-family: serif;">CERTIFICADO</h1>
            <p>Certificamos que</p>
            <div class="cert-user">{{ nome_usuario }}</div>
            <p>concluiu com êxito o treinamento de</p>
            <h2 style="font-size: 2rem; margin: 15px 0;">{{ curso.titulo }}</h2>
            <p>Emitido por <strong>Cyber Chase</strong> em {{ data_hoje }}.</p>
            <br><br>
            <button class="btn-print" onclick="window.print()" style="padding: 10px 20px; background: #000; color: #fff; c
        </div>
    </section>
</main>
```

Fonte: captura de tela do código desenvolvido para o presente projeto (2026), disponível em https://github.com/leticiaoj/tcc

Depois de enviada e validada a resposta, é habilitado um botão para o certificado, que é gerado automaticamente possuindo o nome do usuário e o título do curso consumidos do banco de dados chamado no back-end e a data atual definida no back-end por meio da importação da biblioteca datetime no Python.

Figura 40: botão de geração de certificado para a página de um determinado curso



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

2.8 Conteúdo dos cursos

O sistema conta atualmente com seis cursos de educação cibernética apresentados a seguir.

2.8.1 Ataques Cibernéticos

Com o avanço da tecnologia, os ataques cibernéticos se intensificaram e ficaram cada vez mais presentes, com criminosos digitais explorando cada vez mais vulnerabilidades para obter dados indevidamente e, com isso, causar prejuízos financeiros e comprometer serviços essenciais, podendo ocorrer em ambientes pessoais e empresariais.

De acordo com a IBM Brasil, um ataque cibernético é qualquer tentativa intencional de acessar, alterar, destruir ou roubar dados e sistemas computacionais sem autorização, que podem ser feitos por hackers individuais, grupos criminosos organizados, ativistas digitais ou governos envolvidos em espionagem cibernética, cujos objetivos variam entre ganho financeiro, roubo de informações confidenciais, interrupção de serviços e sabotagem digital.

Ainda segundo dados divulgados pela IBM, o custo médio global de uma violação de dados ultrapassa milhões de dólares, devido a prejuízos de paralisação

de operações, perda de dados, recuperação de sistemas e danos à reputação das organizações. Além de vazamento de informações pessoais, fraudes financeiras, interrupção de serviços públicos e comprometimento da privacidade dos usuários, em casos não empresariais.

Entre os principais tipos de ataques cibernéticos está o *malware*, termo utilizado para definir softwares maliciosos criados para danificar sistemas ou roubar dados, podendo alcançar dispositivos por meio de downloads suspeitos, anexos de e-mails maliciosos, sites inseguros e dispositivos contaminados.

O *ransomware* é um dos ataques mais perigosos da atualidade, onde criminosos sequestram os dados da vítima por meio de criptografia e exigem pagamento para liberar o acesso aos arquivos, onde mesmo após o pagamento do resgate, os dados podem não ser recuperados.

Outro ataque bastante comum é o *phishing*, técnica baseada em engenharia social utilizada para enganar usuários e obter dados confidenciais, para tal os criminosos enviam mensagens falsas por e-mail, SMS ou redes sociais simulando comunicações legítimas de bancos, empresas ou órgãos oficiais, induzindo a clicar em links maliciosos ou fornecer dados pessoais, como senhas e informações bancárias.

Uma evolução desse golpe é o *quishing*, modalidade de phishing realizada por meio de QR Codes falsificados que direcionam as vítimas para páginas fraudulentas. Como os QR Codes ocultam o endereço real do site, muitos usuários acabam acessando páginas falsas sem perceber o risco envolvido.

Os ataques de força bruta também são amplamente utilizados por criminosos virtuais, onde são realizadas inúmeras tentativas automáticas de descobrir senhas e credenciais de acesso até encontrar a combinação correta. Senhas fracas, curtas ou previsíveis aumentam significativamente a vulnerabilidade dos sistemas diante desse tipo de ataque.

Outro método bastante perigoso é o *spoofing*. Crominosos falsificam a identidade de pessoas, empresas ou sistemas para enganar uma vítima, seja utilizando e-mails, endereços IP, páginas da internet ou chamadas telefônicas falsas, com o objetivo de roubar informações, instalar malwares ou obter acesso indevido aos sistemas.

O cavalo de Troia (*Trojan Horse*) é um ataque onde o malware se disfarça como um programa legítimo para convencer o usuário a instalá-lo, porém, após a instalação,

o invasor pode obter controle remoto do dispositivo infectado, acessar arquivos pessoais e instalar outros programas maliciosos sem o conhecimento da vítima.

Outro ataque é o *cryptojacking*, onde criminosos utilizam os recursos computacionais do dispositivo da vítima para minerar criptomoedas sem autorização por meio de scripts maliciosos inseridos em sites, aplicativos infectados ou malwares instalados silenciosamente no sistema, de modo que o computador da vítima superaqueça, apresente excesso de lentidão e alto consumo de processamento. Porém, em alguns casos é difícil a percepção do usuário, o que torna este tipo de ataque difícil de detectar.

As *backdoors* (“portas dos fundos”) criam mecanismos ocultos de acesso remoto aos sistemas, para que invasores controlem dispositivos sem o conhecimento do usuário. As *backdoors* podem ser instaladas por *malwares*, falhas de software ou até mesmo por alterações maliciosas em atualizações de sistemas. Um exemplo real ocorreu no ataque à SolarWinds, em 2020, considerado um dos maiores ataques cibernéticos da história. Nesse caso, criminosos inseriram uma backdoor chamada *Sunburst* em atualizações legítimas do software Orion, permitindo acesso não autorizado a milhares de empresas e órgãos governamentais, incluindo instituições dos Estados Unidos.

Há também o *Man-in-the-Middle* (MitM), onde o invasor intercepta a comunicação entre duas partes sem que elas percebam e capture dados de senhas, informações bancárias e mensagens privadas. Esse ataque é comum em redes Wi-Fi públicas inseguras, onde hackers podem monitorar o tráfego de dados dos usuários conectados. Para reduzir os riscos, é importante utilizar conexões criptografadas, VPNs e evitar logar em redes públicas de Wi-Fi.

A injeção de SQL é utilizada para explorar vulnerabilidades em bancos de dados de aplicações web, na qual, comandos SQL maliciosos são inseridos em campos de entrada de formulários ou sistemas vulneráveis, e permite acesso indevido a informações armazenadas.

Com o Tunelamento DNS, comunicações maliciosas são ocultadas dentro do protocolo DNS que é utilizado para tradução de nomes de domínio na internet, e ao ocultar essa informação, os criminosos utilizam esse método para transferir dados roubados ou manter comunicação com sistemas comprometidos sem levantar suspeitas.

Os chamados ataques sem arquivo são ataques utilizando terminais ou scripts ao invés de arquivos, sendo mais difíceis de serem identificados por antivírus tradicionais que identificam ameaças por meio de arquivos.

Outra ameaça importante é a falsificação de DNS (*DNS Spoofing*), onde os criminosos modificam respostas do sistema DNS e direcionam usuários para páginas falsas sem que percebam, de modo que mesmo que digitem o endereço correto de um site legítimo, a vítima pode ser enviada para páginas fraudulentas desenvolvidas para roubo de credenciais e informações pessoais.

Já os ataques de negação de serviço (DDoS – Distributed Denial of Service) funcionam por meio da sobrecarga de servidores, sistemas ou redes com grande volume de acessos simultâneos, tornando os serviços indisponíveis e causando prejuízos financeiros por interrupção das operações, sendo os alvos as empresas de tecnologia, bancos e plataformas digitais. Segundo a IBM, alguns ataques DDoS modernos conseguem gerar tráfego de vários terabits por segundo.

Outro tipo de ameaça são os ataques de engenharia social, nos quais os criminosos manipulam emocionalmente as vítimas para obter informações ou acesso aos sistemas. Esses ataques exploram fatores como medo, urgência, curiosidade e confiança. Em muitos casos, o sucesso do ataque depende mais da falha humana do que de vulnerabilidades técnicas.

Nos *APT* (*Advanced Persistent Threats*), os invasores permanecem infiltrados nos sistemas por longos períodos sem serem detectados, com objetivo de praticar espionagem digital, roubo de informações estratégicas e monitoramento de organizações governamentais ou grandes empresas.

Com o avanço da inteligência artificial criminosos passaram a utilizar ferramentas de IA para automatizar ataques, criar mensagens de phishing mais convincentes e acelerar a exploração de vulnerabilidades. Isso aumenta significativamente a capacidade de atuação dos invasores e dificulta a identificação das ameaças.

Os ataques têm se intensificado, apresentando casos famosos em larga escala, como o ataque do ransomware *WannaCry*, ocorrido em 2017, em mais de 150 países e atingindo hospitais, empresas e órgãos públicos.

Nesse cenário, é importante adotar medidas de proteção contra ataques cibernéticos. Uma das principais recomendações é manter sistemas operacionais, aplicativos e softwares de segurança constantemente atualizados. As atualizações

corrigem vulnerabilidades conhecidas e reduzem as chances de exploração por criminosos virtuais, além de utilizar autenticação multifator (MFA), que adiciona camadas extras de proteção além da senha tradicional, de modo que, mesmo que as senhas fiquem comprometidas, o acesso pode ser bloqueado pela verificação adicional, e mesmo com a MFA, também é importante utilizar senhas únicas e complexas com a combinação de diversos caracteres e em tamanhos grandes, além de combinar essas práticas a ferramentas de segurança como antivírus e firewalls. Outra prática é a realização de backups periódicos para minimizar impactos causados por possíveis ataques como ransomware e armazenar cópias de segurança em ambientes externos ou serviços em nuvem seguros, para garantir a recuperação das informações em caso de incidente. Ademais, é importante evitar clicar em links desconhecidos e verificar a autenticidade de solicitações recebidas por meios digitais.

2.8.2 Engenharia Social

A engenharia social é utilizada por diversos criminosos atualmente para obter acesso indevido a dados, sistemas e informações sensíveis. Ela se baseia na manipulação psicológica das pessoas, explorando emoções, comportamentos e relações de confiança para induzir vítimas a fornecer informações confidenciais ou realizar ações que comprometam a segurança digital.

De acordo com a Agência Brasileira de Inteligência (ABIN), instituições públicas e privadas são constantemente alvo de ações de engenharia social, especialmente devido ao valor estratégico das informações que possuem. A ABIN enfatiza que esse tipo de ameaça está diretamente relacionado à proteção de conhecimentos sensíveis, podendo causar espionagem, sabotagem, vazamento de informações e prejuízos financeiros e institucionais.

A engenharia social ocorre quando o atacante utiliza técnicas de persuasão, manipulação e engano para explorar vulnerabilidades humanas, por meio de técnicas como se passar por outra pessoa (conhecidos, funcionários de empresa etc.) a fim de conquistar confiança para obter informações estratégicas, como senhas, dados bancários, documentos internos ou acesso a sistemas corporativos, podendo ocorrer por ligações telefônicas fraudulentas, e-mails falsos, mensagens instantâneas, redes sociais e aplicativos de conversa.s. Segundo a ABIN, redes sociais também

representam um ambiente de alto risco, pois criminosos utilizam informações publicadas pelos usuários para construir ataques mais convincentes e personalizados.

Um dos tipos mais conhecidos de engenharia social é o *phishing*, que consiste no envio de mensagens falsas que simulam comunicações legítimas de bancos, empresas ou órgãos oficiais tentando convencer a vítima a clicar em links maliciosos, fornecer credenciais de acesso ou baixar arquivos contaminados. O phishing geralmente explora senso de urgência, medo ou curiosidade para aumentar as chances de sucesso do ataque.

Outra modalidade bastante utilizada é o *smishing*, realizado por meio de mensagens SMS fraudulentas, com mensagens informando supostos bloqueios bancários, promoções ou problemas cadastrais, levando a vítima a acessar links maliciosos ou fornecer informações pessoais; ou o *vishing* utilizando chamadas telefônicas falsas nas quais os atacantes fingem ser representantes de instituições confiáveis para obter dados sensíveis.

Há também o *pretexting*, onde o criminoso cria uma história falsa e convincente para manipular a vítima.

No *baiting*, o criminoso utiliza recompensas ou objetos atrativos para enganar as vítimas, como abandonando propositalmente dispositivos USB infectados em locais públicos ou corporativos a fim de que uma vítima o utilize e instale um malware em seu dispositivo.

A exposição das pessoas nas redes sociais podem dar insumos à engenharia social, visto que o excesso de informações compartilhadas publicamente, como localização, rotina, ambiente de trabalho, contatos pessoais e hábitos cotidianos, podem ser utilizadas por criminosos para personalizar ataques e aumentar a credibilidade das abordagens. Segundo a ABIN, a exposição excessiva de informações facilita ações de espionagem e coleta de dados estratégicos.

Entre os riscos da engenharia social estão o vazamento de dados confidenciais, fraudes financeiras, invasões de sistemas, roubo de identidade, espionagem corporativa e interrupção de serviços. Em ambientes empresariais, um único colaborador enganado pode comprometer toda a infraestrutura da organização, além dos prejuízos financeiros, danos à reputação institucional e comprometimento de informações estratégicas. Segundo o Programa Nacional de Proteção do Conhecimento Sensível (PNPC), conhecimentos estratégicos e sigilosos precisam de

medidas especiais de proteção devido ao impacto que seu vazamento pode causar à sociedade e às organizações.

Para se proteger dos riscos da engenharia social, é importante desconfiar de solicitações inesperadas de informações pessoais, senhas ou dados bancários, especialmente quando realizadas por telefone, e-mail ou mensagens instantâneas, e verificar a identidade de remetentes e evitar clicar em links suspeitos ou baixar arquivos desconhecidos. Em ambientes corporativos, deve-se confirmar solicitações diretamente com os setores responsáveis antes de compartilhar informações sensíveis.

Nas redes sociais, é importante evitar exposição excessiva de forma pública, principalmente de dados comprometedores ou que possam servir de insumo para a engenharia social.

2.8.3 Phishing

O phishing é uma das práticas criminosas mais praticadas atualmente contra usuários ou organizações. O termo “phishing” deriva da palavra inglesa *fishing* (“pesca”), fazendo analogia ao ato de “pescar” informações sensíveis de vítimas por meio de técnicas fraudulentas, sendo utilizado por cibercriminosos para obter dados confidenciais, como senhas, números de cartões bancários, documentos pessoais e credenciais de acesso a sistemas corporativos.

De acordo com a IBM Brasil, o phishing é um ataque de engenharia social no qual os criminosos manipulam psicologicamente os usuários para que realizem ações que comprometam sua segurança digital, explorando emoções, como medo, urgência, curiosidade ou confiança, levando a vítima a agir impulsivamente sem verificar a legitimidade da mensagem recebida.

A engenharia social é considerada um elemento do phishing, pois os ataques dependem, além de vulnerabilidades técnicas, de falhas humanas, onde criminosos tentam se passar por empresas conhecidas, instituições financeiras, órgãos governamentais ou pessoas de confiança, criando mensagens visualmente semelhantes às comunicações oficiais dessas entidades com o objetivo de induzir o usuário a fornecer informações sigilosas ou clicar em links maliciosos que podem instalar softwares nocivos em seu dispositivo.

O funcionamento do phishing acontece separado por etapas, onde o atacante desenvolve uma mensagem falsa utilizando elementos visuais convincentes, como logotipos, assinaturas digitais e linguagem corporativa, envia a mensagem para um grande número de usuários ou para um alvo específico e o conteúdo geralmente solicita atualização de dados cadastrais, redefinição de senha, confirmação de pagamento ou acesso a um link urgente, de modo que quando a vítima interage com o conteúdo, ela pode ser redirecionada para páginas falsas idênticas às originais, onde suas informações são capturadas pelos criminosos, onde, um clique em um anexo pode resultar na instalação de malwares, ransomwares ou programas espiões no equipamento da vítima.

No Phishing por e-mail, os criminosos enviam mensagens eletrônicas fraudulentas simulando comunicações legítimas de bancos, plataformas digitais, empresas de comércio eletrônico ou redes sociais com alertas falsos sobre problemas na conta do usuário, promoções enganosas ou solicitações urgentes de autenticação. O objetivo é induzir a vítima a fornecer informações pessoais ou acessar links maliciosos.

Quando ocorre por SMS, se chama *smishing* , onde os criminosos enviam mensagens de texto contendo links fraudulentos ou solicitações de confirmação de dados pessoais que informam supostos bloqueios bancários, entregas pendentes ou cobranças indevidas, criando um senso de urgência que aumenta as chances de sucesso do golpe.

O *vishing*, funciona quando os golpes são aplicados em chamadas telefônicas, quando os criminosos entram em contato com as vítimas fingindo ser representantes de instituições financeiras, empresas de tecnologia ou órgãos governamentais e tentam convencer o usuário a informar senhas, códigos de autenticação, dados bancários ou até mesmo realizar transferências financeiras.

O *spear phishing* é uma técnica de Phishing mais detalhada, onde é direcionada de forma específica pois os criminosos fizeram um estudo e coleta de informações da vítima (seja nas redes sociais ou em vazamentos, por exemplo) previamente a fim de personalizar ainda mais a mensagem e aumentar as chances da vítima cair no golpe.

Além dessa personalização, há *whaling*, conhecido como “caça à baleia”, onde o ataque é direcionado especificamente para executivos, gestores e profissionais de

alto nível hierárquico dentro de organizações devido ao acesso a informações altamente sigilosas que essas pessoas possuem.

Ademais, há os *scams*, que são associados ao phishing por possuírem falsas promoções que induzem pessoas a clicarem, ou golpes financeiros, fraudes em compras na internet, investimentos inexistentes e outras formas de obtenção de vantagem econômica indevida, manipulando o senso de ganância das vítimas para extrair dinheiro delas.

Para se proteger contra o phishing, é necessário observar os indícios, que podem ser a presença de erros gramaticais, falhas ortográficas e formatação inadequada nas mensagens recebidas. Empresas legítimas geralmente mantêm padrões profissionais de comunicação, revisando cuidadosamente seus conteúdos antes do envio aos clientes, além de verificar a autenticidade do remetente, analisando endereços de e-mail, números telefônicos e domínios de sites, pois os criminosos costumam utilizar pequenas alterações visuais para enganar as vítimas. Um exemplo comum é a substituição de caracteres do alfabeto latino por símbolos semelhantes de outros alfabetos, tornando os links aparentemente legítimos.

Também é importante desconfiar de mensagens que criam senso de urgência ou exigem ações imediatas, e, anexos inesperados podem conter arquivos maliciosos destinados à infecção do sistema, sendo necessário confirmar antes de tomar qualquer ação de abertura do arquivo.

É importante combinar essas boas práticas com firewalls, antivírus, e sistemas operacionais e navegadores atualizados constantemente, além de utilizar autenticação multifatorial para evitar qualquer acesso indevido em caso de comprometimento de senha, além de adicionar filtros antispam nas caixas de mensagens.

É importante lembrar que, em caso de desconfiança da legitimidade de uma mensagem recebida, antes de abrir, o usuário deve procurar por canais oficiais para validar se o remetente é confiável, ou não.

2.8.4 Vírus

Os vírus estão presentes na internet há muito tempo, porém com o avanço tecnológico, eles têm se intensificado e se tornado cada vez mais sofisticados para atingir um número maior de vítimas.

Segundo a Kaspersky, o termo *malware* deriva da junção das palavras inglesas *malicious* e *software*, sendo utilizado para definir qualquer software desenvolvido com finalidade maliciosa, termo este que engloba os vírus de computador, worms, trojans, spyware, ransomware e diversos outros programas criados para comprometer sistemas computacionais sem o consentimento do usuário, que podem causar lentidão no desempenho do dispositivo, roubo de informações sigilosas, exclusão de arquivos e controle remoto do sistema por criminosos virtuais.

Os vírus de computador receberam essa denominação devido à sua capacidade de “infectar” arquivos e se propagar para outros sistemas de forma semelhante aos vírus biológicos, onde, quando um arquivo contaminado é executado, o código malicioso se replica e passa a comprometer outros arquivos ou dispositivos conectados.

O primeiro vírus de computador conhecido foi denominado *Brain*, criado em 1986, criado para tentar combater a pirataria de software, infectando o setor de inicialização de disquetes, e se espalhando para outros computadores durante o processo de cópia dos arquivos. Com o tempo, os vírus evoluíram e chegaram ao ponto onde são capazes de infectar computadores, smartphones, servidores e até dispositivos conectados à Internet das Coisas (IoT).

Os *worms* são um tipo de vírus diferente dos vírus tradicionais, devido ao fato de se propagarem automaticamente e sem necessidade de interação humana, explorando vulnerabilidades de rede para se espalhar rapidamente entre dispositivos conectados após a infecção, podendo consumir recursos computacionais excessivamente, provocar lentidão em redes corporativas e disseminar cargas maliciosas adicionais.

Os *trojans* ou “cavalos de Troia” se disfarçam de softwares legítimos para que as vítimas realizem o download, após esta ação, permitem que os invasores tenham acesso remoto ao dispositivo infectado, possibilitando roubo de dados, espionagem e instalação de outros malwares.

Já o *ransomware,* após sua infecção, exige resgate financeiro para a liberação do sistema, porém, em alguns casos, mesmo após o pagamento do resgate, os criminosos não liberam o dispositivo infectado.

O *spyware* corresponde a programas espiões desenvolvidos para monitorar secretamente as atividades do usuário, coletando senhas, hábitos de navegação, dados bancários e localização geográfica para serem vendidas para terceiros ou fraudes digitais.

Já o *adware* consiste na exibição excessiva de anúncios que podem ser inofensivos, ou que podem roubar dados do usuário, ou, caso o usuário clique em algum anúncio, pode redirecioná-lo para uma página maliciosa.

Há também o *rootkit*, feito para ocultar sua presença no sistema operacional e permitir controle privilegiado do dispositivo infectado, conseguem evitar mecanismos tradicionais de detecção, dificultando sua remoção e são utilizados para instalar outros tipos de malware no sistema comprometido.

Para infectar-se com um vírus, basta fazer o download de arquivos infectados, anexos maliciosos em e-mails, links fraudulentos enviados por redes sociais ou aplicativos de mensagens, dispositivos USB contaminados e exploração de vulnerabilidades em sistemas desatualizados, sendo necessário evitar essas práticas e ter atenção.

Se o usuário notar lentidão excessiva do sistema, travamentos frequentes, programas desconhecidos executando em segundo plano, aumento inesperado de anúncios pop-up, alteração da página inicial do navegador e dificuldade de acesso à internet ou funções administrativas do sistema operacional, ele deve desconfiar de vírus em seu dispositivo, apesar de muitos malwares modernos operarem silenciosamente, dificultando sua identificação pelos usuários.

Para evitar os vírus, cabe algumas medidas como manter sistemas operacionais, aplicativos e softwares antivírus constantemente atualizados, utilização de softwares antivírus confiáveis e soluções de proteção em tempo real, porém,conforme destacado pela Kaspersky, nenhum antivírus é capaz de impedir completamente todas as ameaças existentes, tornando indispensável a adoção de práticas seguras de navegação e uso consciente da tecnologia, então, é importante evitar abrir anexos desconhecidos, não clicar em links suspeitos, verificar a autenticidade de páginas da internet e desconfiar de mensagens alarmantes são atitudes fundamentais para reduzir riscos de infecção. Além disso, vale realizar

backups periódicos dos arquivos importantes, preferencialmente em ambientes externos ou na nuvem para facilitar a recuperação desses arquivos em casos de perdas por incidentes de vírus.

2.8.5 Segurança em Dispositivos Móveis

Atualmente, os dispositivos móveis fazem parte do dia a dia da população, estando amplamente presente em suas atividades. Porém, ter este portal para o mundo cibernético com facilidade em mãos exige determinados cuidados.

De acordo com a URMOBI, a *mobile security* corresponde ao conjunto de práticas, ferramentas e tecnologias voltadas à proteção de dispositivos móveis contra ameaças digitais, acessos não autorizados, vazamento de dados e ataques cibernéticos, sendo fundamental para usuários comuns e organizações, especialmente diante do aumento de ataques direcionados a smartphones e tablets.

Os dispositivos móveis possuem grande quantidade de informações sensíveis, como senhas, fotos, documentos, dados bancários, e-mails e informações corporativas. Além disso, permanecem constantemente conectados à internet por meio de redes móveis, Wi-Fi e aplicativos online. Essa conectividade contínua transforma os dispositivos em alvos atrativos para criminosos virtuais. Segundo a IBM Brasil, os dispositivos móveis possuem vulnerabilidades específicas relacionadas ao uso frequente de redes públicas, múltiplos aplicativos e sistemas desatualizados.

A falta de segurança em dispositivos móveis pode trazer riscos como os ataques de malware, phishing, roubo de dados, espionagem digital e acessos não autorizados, malwares móveis por aplicativos falsos, downloads suspeitos, links maliciosos e mensagens fraudulentas. Se instalados, esses programas conseguem monitorar atividades, capturar senhas, acessar arquivos e até controlar remotamente o dispositivo infectado, o que pode piorar considerando a grande parcela de pessoas que possuem aplicativos bancários no celular

Ademais, também é importante ter cuidado com redes Wi-Fi públicas, que podem ser utilizadas por invasores para interceptar dados transmitidos pelos usuários. Ataques do tipo *Man-in-the-Middle* permitem que criminosos capturem informações sensíveis, como credenciais bancárias e dados pessoais, sem que a vítima perceba.

A perda ou roubo de dispositivos móveis também pode se tornar um problema de perda de dados e aumento do prejuízo financeiro. Sem mecanismos adequados de proteção, criminosos podem acessar aplicativos bancários, contas pessoais, e-mails corporativos e documentos armazenados no aparelho. Segundo a Kaspersky Brasil, muitos usuários não percebem que seus dispositivos foram comprometidos até identificarem atividades suspeitas em suas contas digitais.

Atualizações de segurança corrigem vulnerabilidades conhecidas que podem ser exploradas por hackers. Então, sistemas operacionais desatualizados aumentam os riscos de ataques cibernéticos. Dispositivos antigos ou sem suporte do fabricante tornam-se mais vulneráveis a malwares e invasões.

Assim, é importante manter boas práticas de segurança no dia a dia dos dispositivos móveis. Algumas medidas são manter o sistema operacional e os aplicativos sempre atualizados, utilizar senhas fortes e autenticação multifator (MFA), uso de biometria, reconhecimento facial e códigos adicionais de verificação que dificultam acessos indevidos mesmo em casos de perda ou roubo do aparelho. Além do bloqueio remoto e localização do dispositivo em caso de furto.

Ademais, é recomendado instalar aplicativos apenas de lojas oficiais e, ainda assim, os usuários devem analisar as permissões solicitadas pelos aplicativos, evitando conceder acesso desnecessário à câmera, microfone, localização e arquivos pessoais, principalmente devido ao aumento de aplicativos espiões conhecidos como *stalkerware*, que monitoram secretamente as atividades de uma pessoa por meio do smartphone ou tablet, geralmente sem o consentimento da vítima. Os aplicativos podem capturar mensagens, ligações, localização em tempo real, histórico de navegação, fotos, áudios e até acessar a câmera e o microfone do dispositivo. Sua presença pode ser identificada por meio de um aumento incomum no consumo de bateria, pelo superaquecimento do aparelho mesmo sem uso intenso, lentidão excessiva, aumento no consumo de dados móveis, aparecimento de aplicativos desconhecidos e comportamentos anormais no dispositivo, como reinicializações inesperadas ou ativação involuntária de câmera e microfone, ruídos estranhos durante chamadas telefônicas ou notificações incomuns relacionadas a permissões do sistema, já que o aplicativo permanece funcionando em segundo plano para coletar e transmitir informações. Também, é importante verificar regularmente a lista de aplicativos instalados no dispositivo, observando programas desconhecidos ou com nomes genéricos.

Outra prática recomendada é utilizar soluções de antivírus e ferramentas de segurança móvel capazes de detectar stalkerwares e outros aplicativos maliciosos, manter o sistema operacional atualizado e evitar desbloqueios inseguros do aparelho, como *root* ou *jailbreak*, utilizar senhas fortes, biometria e autenticação multifator também dificulta o acesso físico não autorizado ao dispositivo. Em caso de infecção, o ideal é uma restauração para as configurações de fábrica para garantir que não fiquem vestígios do software e registrar uma ocorrência às autoridades competentes em caso de perseguição digital.

Outra medida importante é evitar conexões em redes Wi-Fi públicas sem proteção adequada. Quando necessário utilizar redes abertas, recomenda-se o uso de VPN (*Virtual Private Network*), tecnologia que criptografa o tráfego de dados e reduz os riscos de interceptação das informações. Além das medidas técnicas, é importante evitar clicar em links suspeitos, instalação de aplicativos inseguros e compartilhamento indevido de informações.

2.8.6 Forense Digital e Respostas a Incidentes

O aumento das fraudes digitais fez com que a forense digital e a resposta a incidentes se tornassem indispensáveis para a segurança da informação nas organizações e para a proteção dos usuários no ambiente digital. Com o aumento de crimes virtuais envolvendo roubo de dados, invasões de sistemas, ransomware e golpes financeiros, foram desenvolvidos mecanismos capazes de identificar, investigar, conter e minimizar os impactos causados por incidentes cibernéticos.

De acordo com a IBM, o termo *DFIR* (*Digital Forensics and Incident Response*) refere-se ao conjunto de práticas relacionadas à investigação forense digital e à resposta a incidentes de segurança. A forense digital consiste na coleta, preservação, análise e interpretação de evidências digitais com o objetivo de identificar a origem, o funcionamento e os impactos de um ataque cibernético, enquanto a resposta a incidentes envolve os procedimentos adotados para detectar, conter, erradicar e recuperar sistemas comprometidos após um incidente de segurança.

Segundo a Fortinet, a integração entre forense digital e resposta a incidentes permite reduzir danos operacionais, proteger dados sensíveis e restaurar rapidamente

os serviços afetados, os processos de DFIR auxiliam na identificação de vulnerabilidades exploradas pelos invasores e na prevenção de novos ataques.

A forense digital é a área especializada da segurança da informação de investigação de crimes e incidentes envolvendo dispositivos eletrônicos, podendo ser aplicada em computadores, smartphones, servidores, redes corporativas, sistemas em nuvem e mídias de armazenamento digital, cujo objetivo é identificar evidências digitais válidas que permitam compreender como o ataque ocorreu, quais sistemas foram afetados e quais informações foram comprometidas, podendo ser arquivos, registros de acesso (*logs*), histórico de navegação, e-mails, mensagens, endereços IP, capturas de memória, metadados e atividades realizadas nos sistemas comprometidos, preservando a integridade dessas evidências para evitar alterações que comprometam sua validade técnica ou jurídica durante o processo investigativo.

A resposta a incidentes, por sua vez, corresponde ao conjunto de ações executadas para lidar com eventos de segurança da informação. Segundo a IBM, um incidente de segurança pode envolver vazamento de dados, invasões, infecção por malware, ataques de ransomware, phishing, negação de serviço (*DDoS*) e diversos outros tipos de ameaças digitais.

O processo de resposta a incidentes geralmente é dividido em etapas, sendo a preparação, fase em que organizações implementam políticas de segurança, ferramentas de monitoramento, planos de contingência e treinamentos preventivos, cujo objetivo é garantir que a equipe esteja preparada para agir rapidamente diante de possíveis incidentes; a identificação do incidente, onde equipes de segurança analisam alertas, registros e comportamentos suspeitos para confirmar a existência de uma ameaça; a fase de contenção, cujo objetivo é impedir que o ataque continue se espalhando, podendo incluir isolamento de dispositivos comprometidos, bloqueio de acessos suspeitos, desconexão da rede e suspensão temporária de serviços afetados; a erradicação da ameaça, onde são removidos malwares, arquivos maliciosos, contas comprometidas e vulnerabilidades exploradas pelos criminosos e também são realizadas atualizações de segurança e correções necessárias para impedir reinfecções; a recuperação dos sistemas afetados, onde os serviços são restaurados gradualmente, backups são recuperados e os ambientes passam a ser monitorados para verificar possíveis sinais de persistência da ameaça; e a análise pós-incidente, momento em que as organizações documentam o ocorrido, identificam falhas de segurança e implementam melhorias preventivas. Essa etapa é essencial

para fortalecer a postura de segurança e reduzir riscos futuros. Segundo a Fortinet, o monitoramento contínuo após o incidente é fundamental para garantir que os invasores não mantenham acessos ocultos aos sistemas comprometidos.

A resposta a incidentes também é importante para usuários comuns que sofrem golpes digitais ou ataques cibernéticos. Em muitos casos, a reação imediata da vítima pode reduzir os danos causados pelo incidente. Quando um usuário percebe que caiu em um golpe digital, uma das primeiras medidas recomendadas é interromper imediatamente qualquer interação com o criminoso. Também é importante desconectar o dispositivo da internet caso exista suspeita de malware ou acesso remoto indevido, reduzindo a possibilidade de continuidade do ataque. Além disso, vale alterar imediatamente as senhas de contas comprometidas, principalmente e-mails, aplicativos bancários, redes sociais e serviços corporativos. É importante utilizar senhas fortes e ativar autenticação multifator (MFA) sempre que possível. Para situações de golpes bancários ou vazamento de dados financeiros, a vítima deve entrar em contato imediatamente com a instituição financeira para bloquear cartões, contas e transações suspeitas. Também é recomendado realizar varreduras utilizando softwares antivírus e ferramentas de segurança confiáveis para identificar possíveis malwares instalados no dispositivo. Em casos mais graves, pode ser necessária a restauração para as configurações de fábrica para não deixar vestígios.

A preservação de evidências digitais também é importante após um ataque. Prints de tela, mensagens, e-mails fraudulentos, comprovantes de transações e registros de atividades suspeitas podem auxiliar investigações policiais e análises forenses, sendo, então, recomendado evitar apagar conteúdos relacionados ao golpe antes da coleta das evidências necessárias, e nesse cenário, também é importante registrar boletim de ocorrência junto às autoridades e comunicar o incidente aos órgãos responsáveis quando houver vazamento de dados pessoais ou corporativos. Em ambientes empresariais, ataques cibernéticos podem exigir notificação à Autoridade Nacional de Proteção de Dados (ANPD), conforme previsto na Lei Geral de Proteção de Dados (LGPD). Além disso, usuários devem monitorar suas contas e atividades financeiras após o incidente, a fim de verificar movimentações suspeitas, acessos desconhecidos e possíveis tentativas de reutilização de credenciais vazadas.

Dentre as técnicas de investigação de incidentes cibernético e análise de crimes digitais, na área de forense digital, há as opções de coleta de evidências digitais (podendo incluir incluir arquivos do sistema, registros de acesso (*logs*),

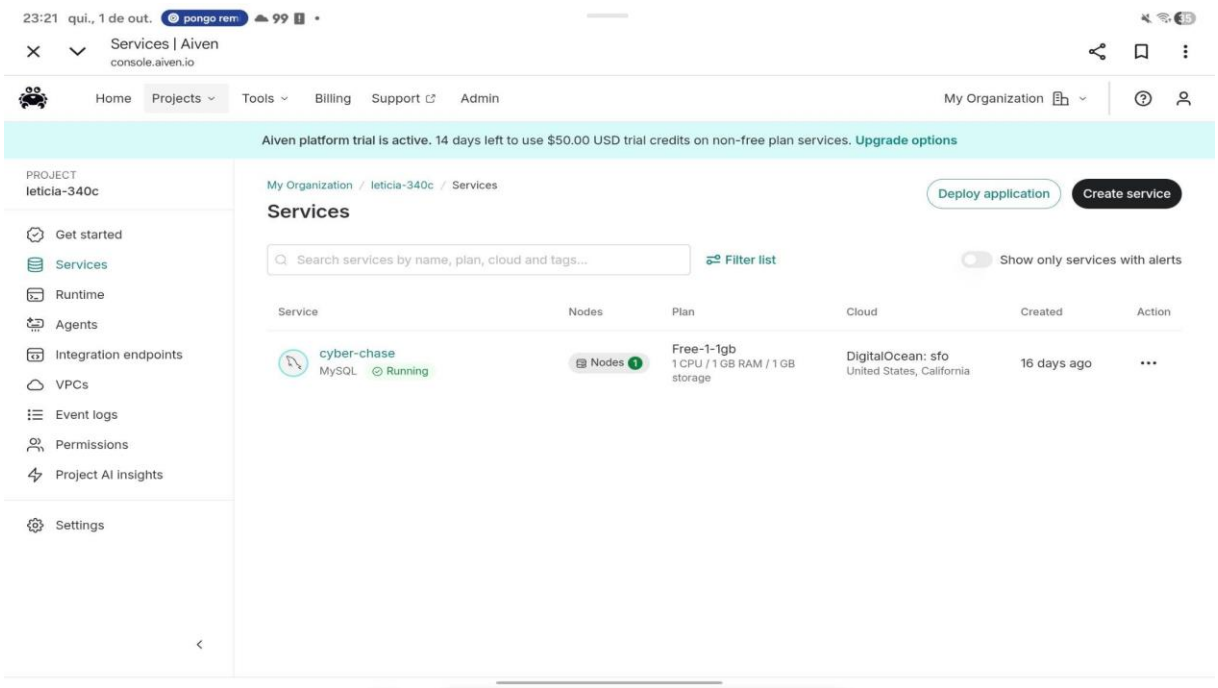
histórico de navegação, e-mails, mensagens, endereços IP, capturas de tela e dados armazenados em dispositivos eletrônicos), recuperação de arquivos apagados, análise de memória RAM e elaboração de relatórios periciais.

Dessa forma, a atuação da forense digital vai além da simples identificação de ataques cibernéticos, abrangendo procedimentos técnicos especializados que auxiliam na investigação, recuperação de dados, produção de provas digitais e fortalecimento da segurança da informação nas organizações e ambientes digitais.

2.9 Hospedagem Online

Para realizar o deploy, foi necessário hospedagem do banco de dados em MySQL no servidor online da plataforma Aiven, como consta a imagem a seguir:

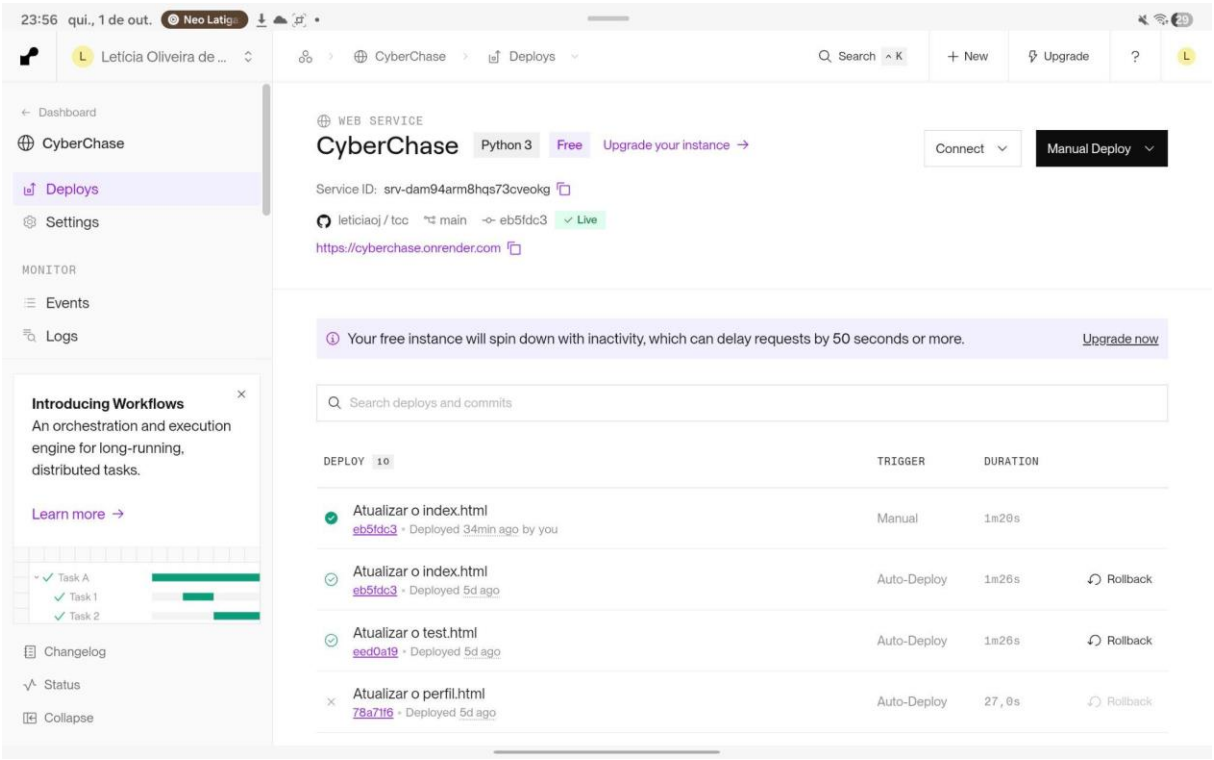
Figura 40: Hospedagem do Banco de Dados na nuvem



Fonte: captura de tela da plataforma Aiven, onde o Banco de Dados se encontra hospedado remotamente

Além disso, com o banco de dados no servidor remoto da Aiven e o repositório no GitHub disponível em https://github.com/leticiaoj/tcc, foi possível fazer o *deploy* do código da aplicação em um servidor *online* na plataforma do Render, conforme consta na imagem a seguir:

Figura 41: *deploy* da aplicação na nuvem



Fonte: captura de tela da plataforma Render, onde a aplicação da Cyber Chase encontra-se hospedada remotamente

A aplicação, então está hospedada neste endereço web: https://cyberchase.onrender.com/.

3 RESULTADOS

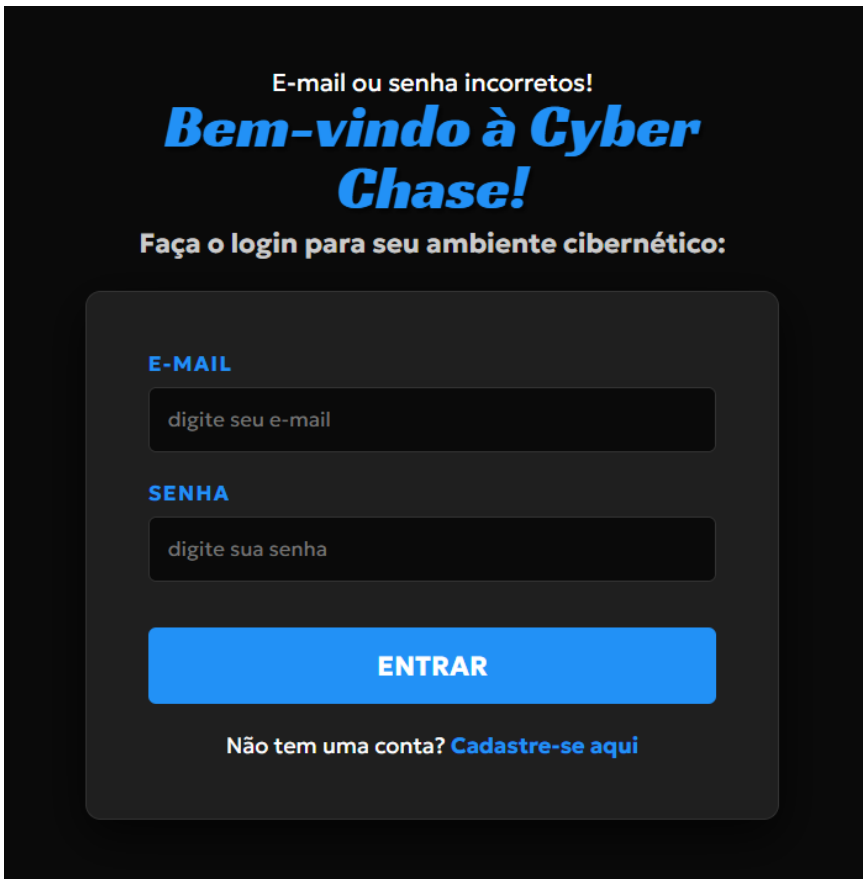
Após o desenvolvimento das funcionalidades, fez-se necessário os testes para garantir o funcionamento correto de toda a plataforma, a fim de analisar os resultados obtidos.

3.1 Testes de funcionalidades

Os testes de funcionamento da plataforma consistiram em aplicações de cenários modulares para cada funcionalidade da plataforma e validação dos resultados apresentados na interação com o sistema, que consistiu nos seguintes cenários:

1. Ao inserir dados incorretos de e-mail e senha, o sistema retorna que está incorreto e não permite prosseguir com o login.

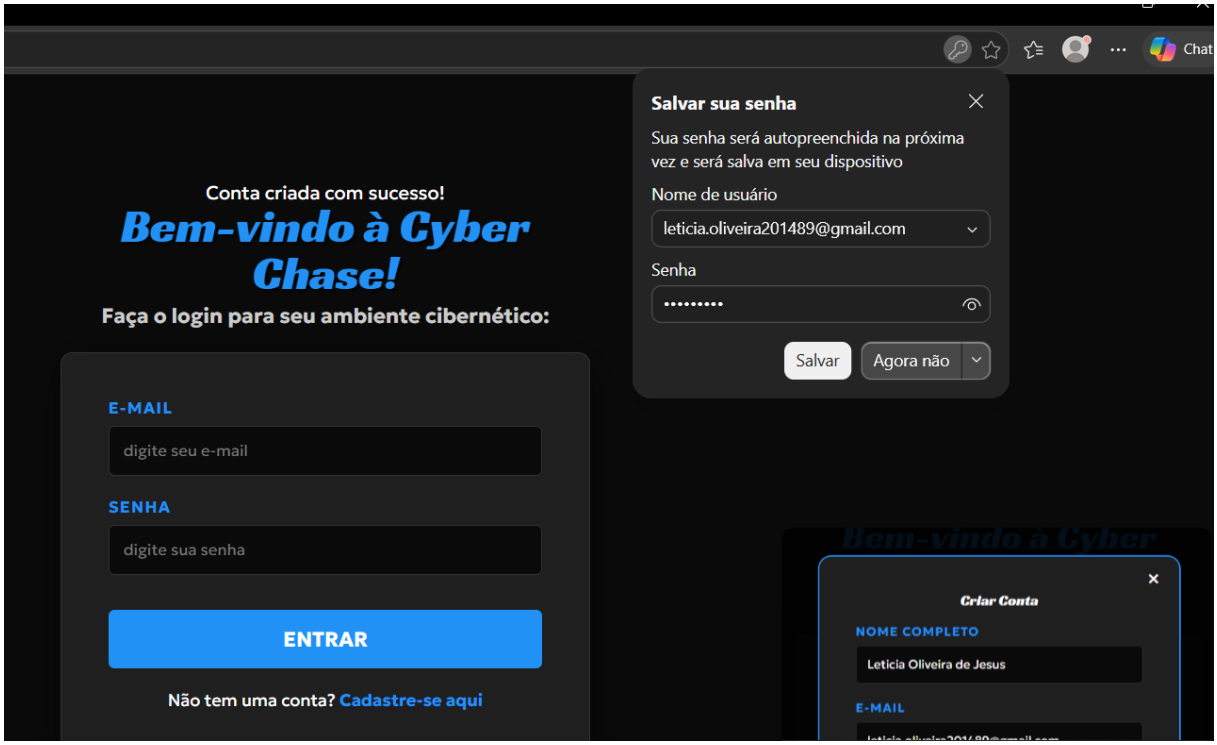
Figura 42: teste de inclusão incorreta de dados de login



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

2. Preenchendo a modal de cadastro, ele retorna uma mensagem sinalizando que o usuário foi cadastrado.

Figura 43: cadastro realizado com sucesso



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

3. Ao testar o chatbot escrevendo com o internetês, diferentemente da linguagem formal que alimentou o banco de dados, a Inteligência Artificial reconhece a mensagem e envia a resposta correta correspondente.

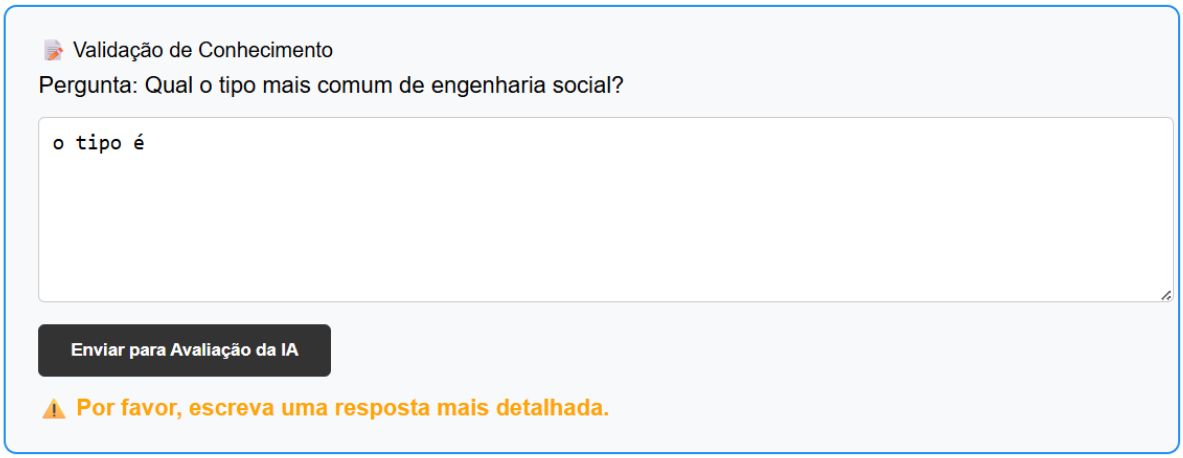
Figura 44: chatbot com Inteligência Artificial funcionando corretamente



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

4. Ao finalizar o curso e incluir uma resposta curta, a IA pede mais detalhes para avaliar, conforme consta a seguir:

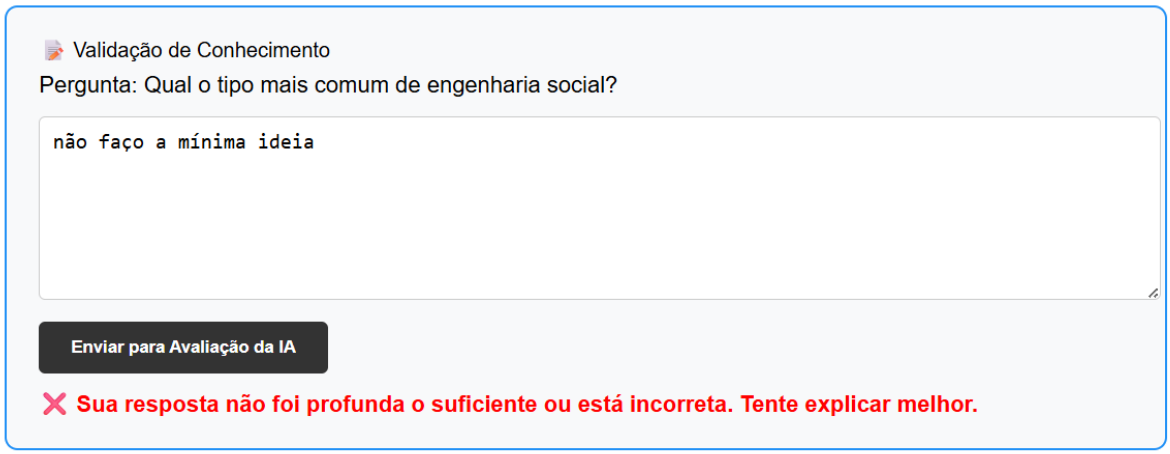
Figura 45: resposta curta para validação da IA recusada



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

5. Ao fornecer uma resposta incorreta, a Inteligência Artificial recusa e pede que o usuário revisite sua resposta.

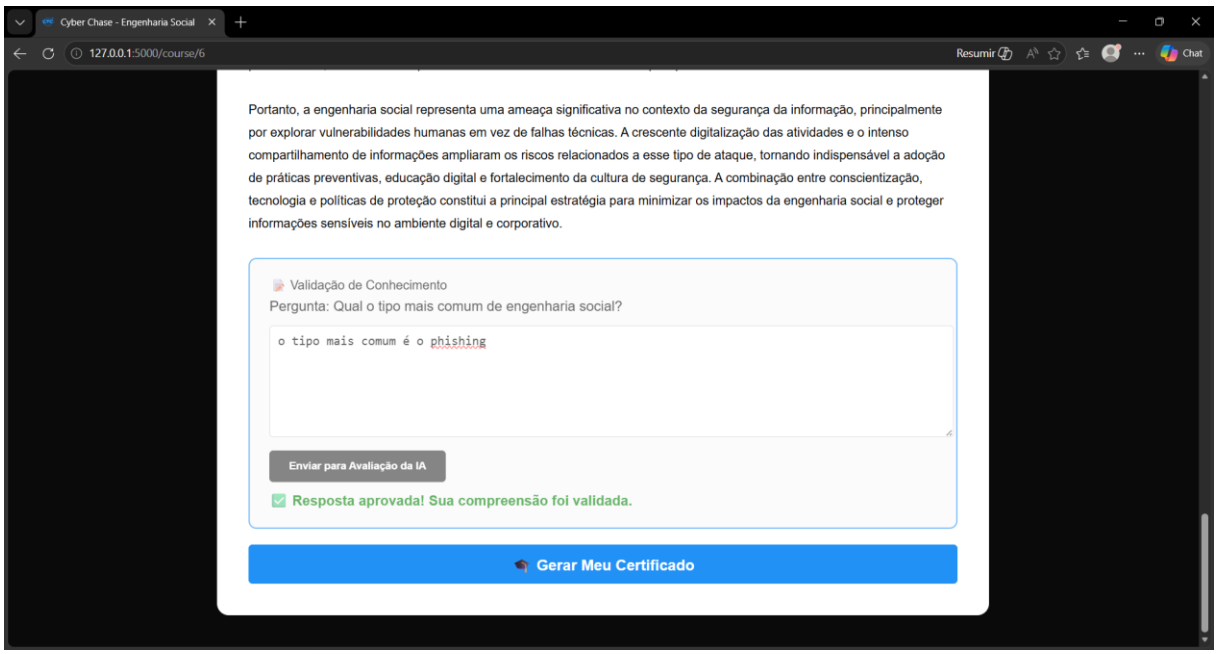
Figura 46: IA identifica resposta errada



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

6. Ao inserir a resposta correta, a Inteligência Artificial retorna uma mensagem de sucesso e permite que o usuário acesse o certificado.

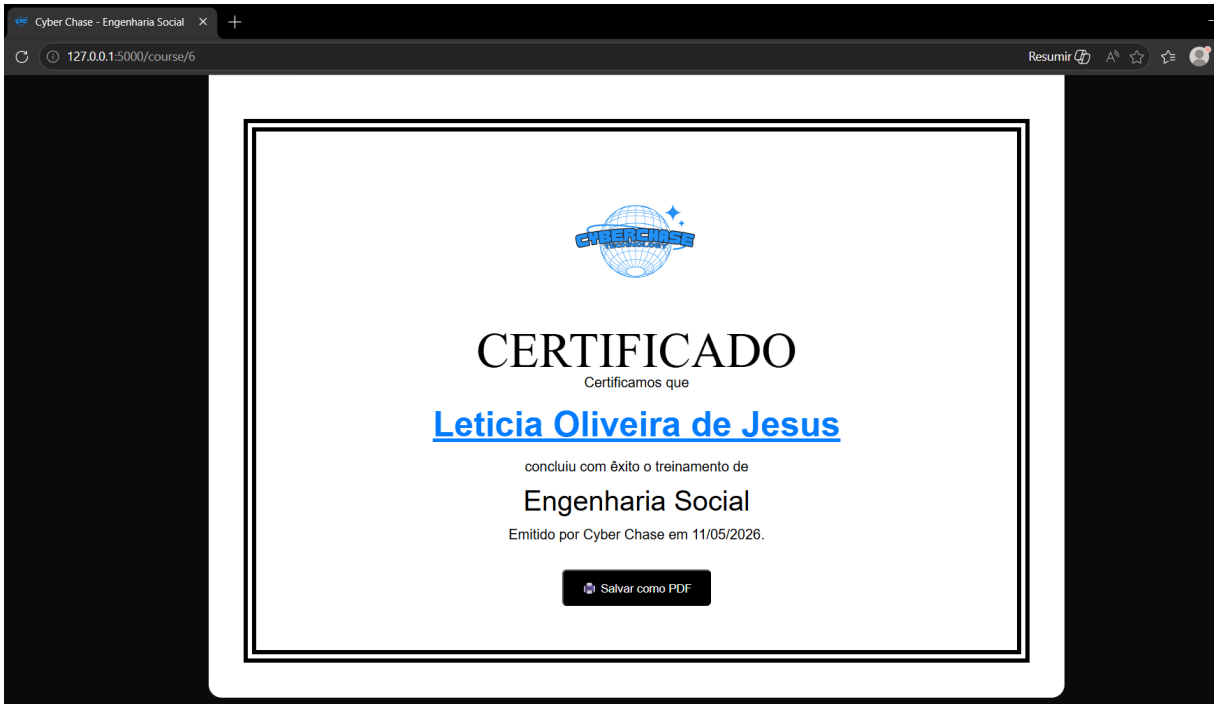
Figura 47: IA valida resposta correta e o botão do certificado é habilitado



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

7. Ao clicar no botão de gerar certificado, é gerado com as informações de nome do usuário, nome do curso e dia do ocorrido.

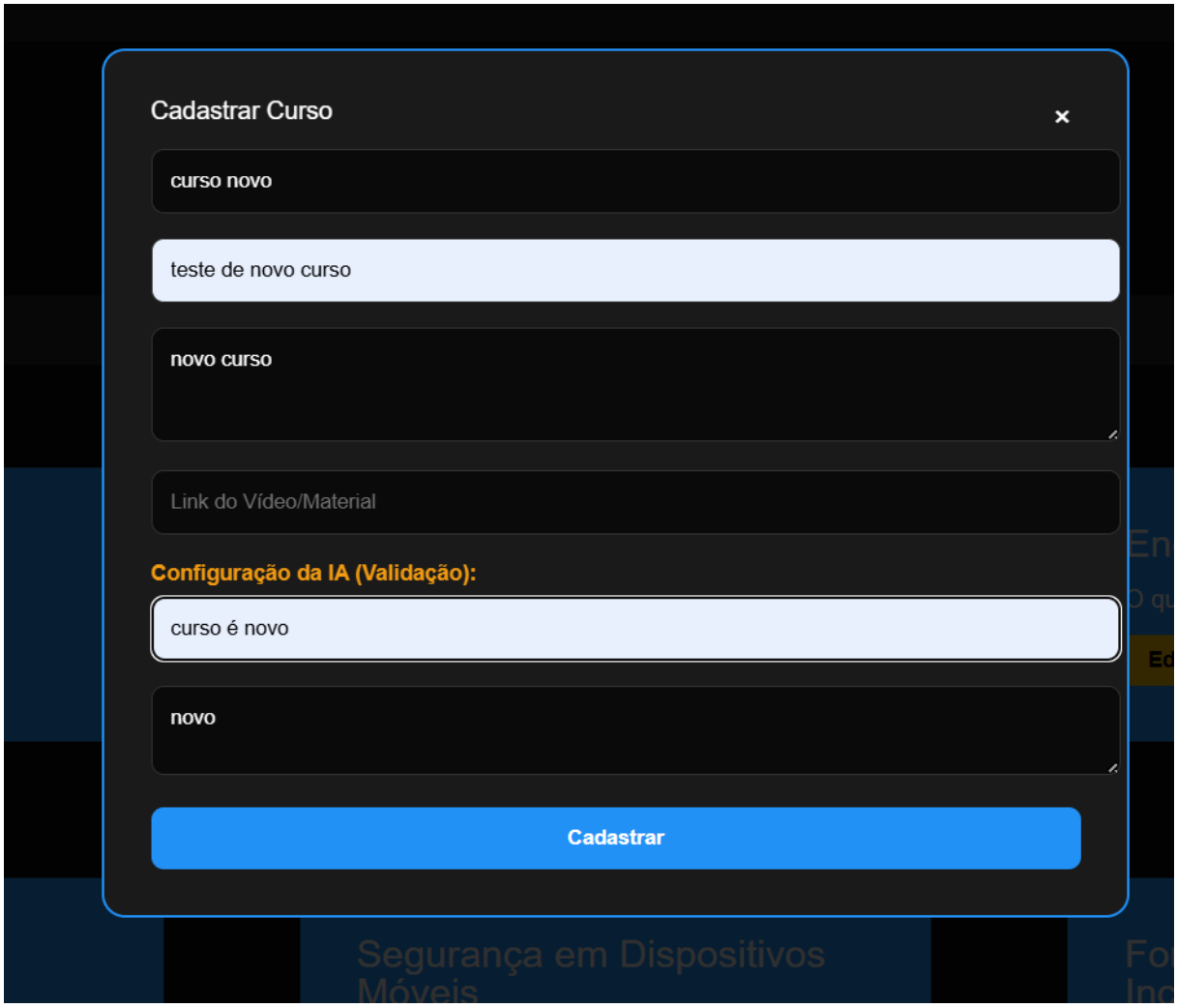
Figura 48: certificado gerado pelo usuário ao finalizar o curso



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

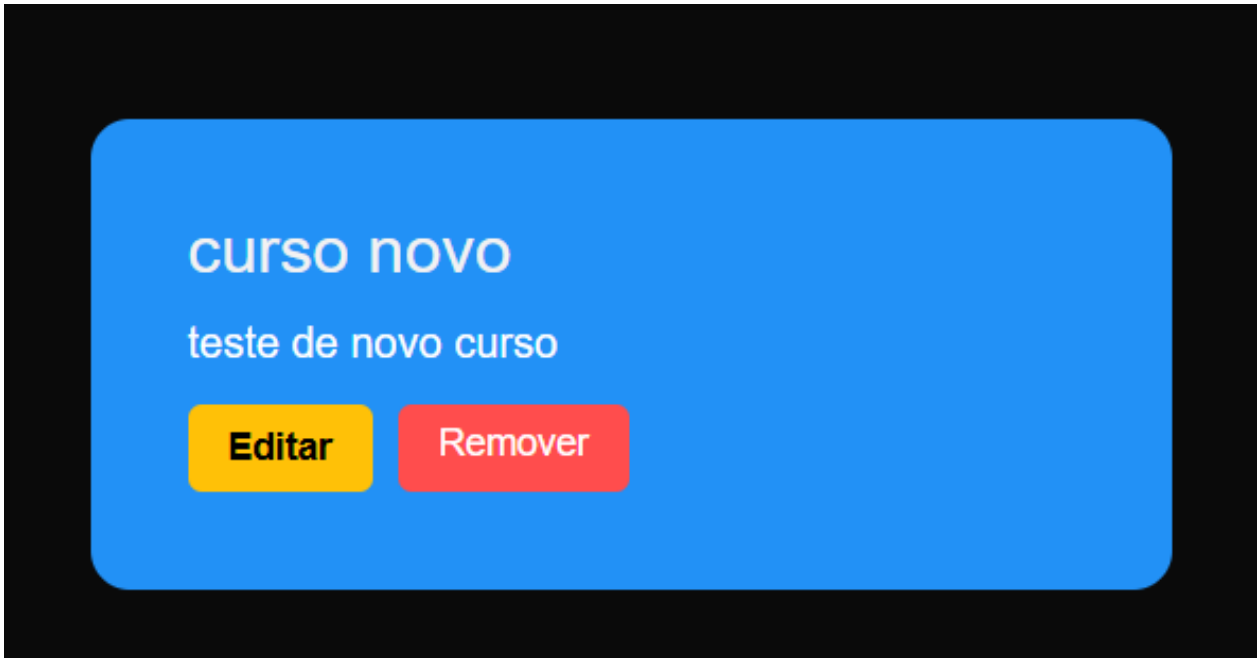
8. Na tela do administrador, é possível criar um curso novo com sucesso por meio da modal de cadastro de cursos.

Figura 49: modal de criação de cursos



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

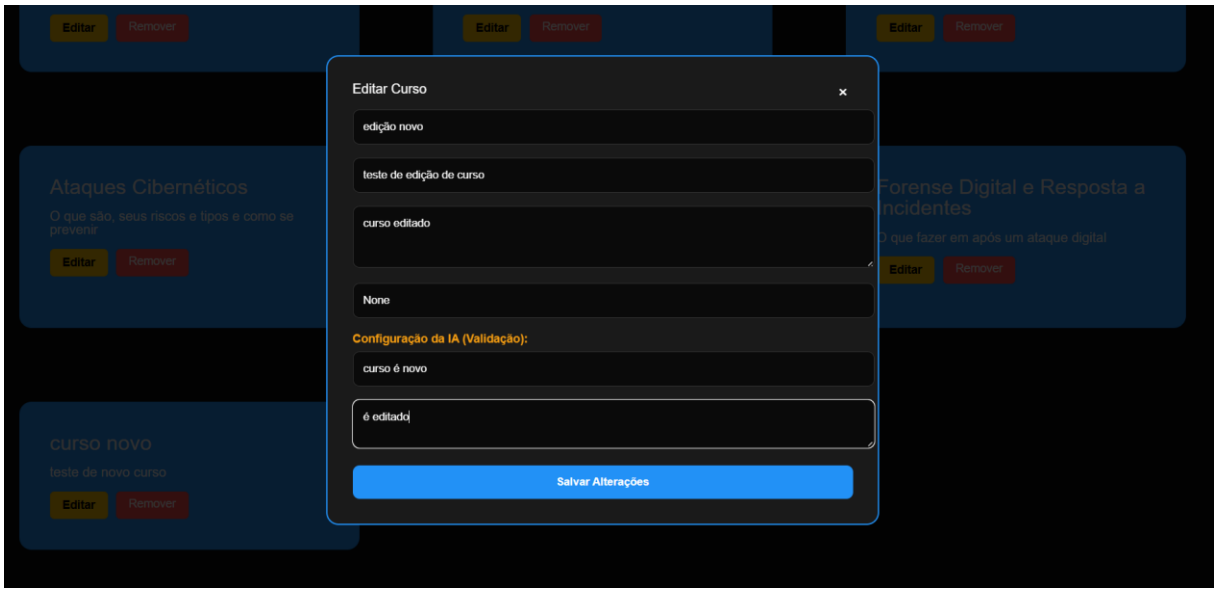
Figura 50: curso criado com sucesso



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

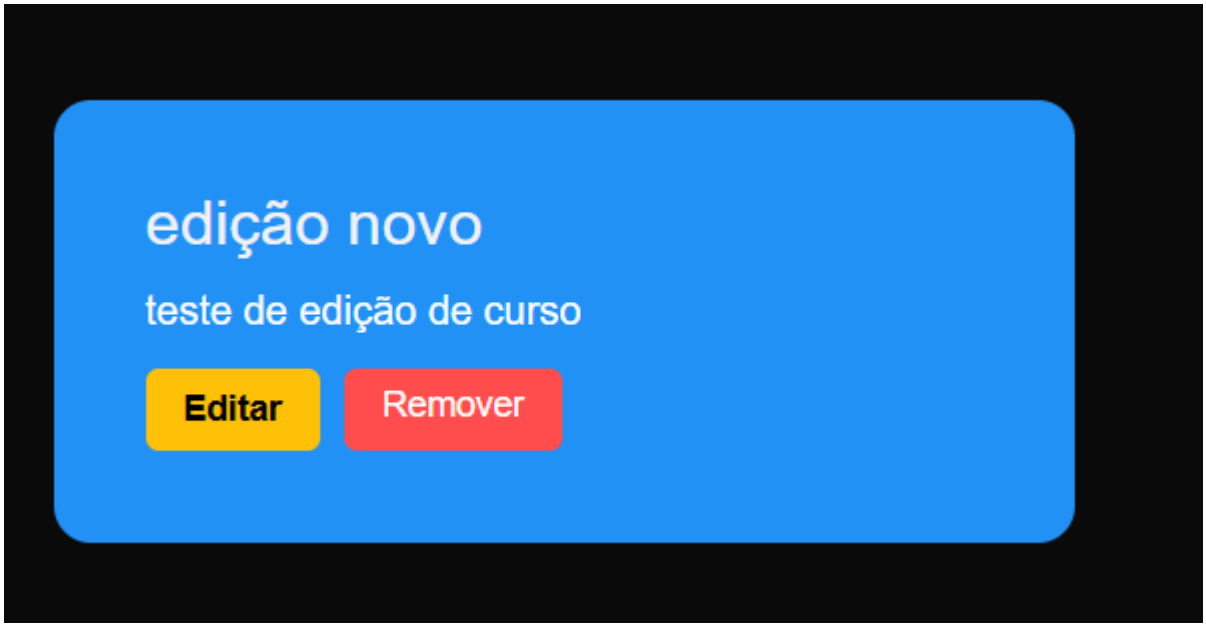
9. Ao editar um curso, o mesmo ocorre com sucesso com a abertura da modal de edição e, após confirmadas pelo botão de salvar mudanças, consta as alterações para o curso em questão.

Figura 51: modal de edição



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

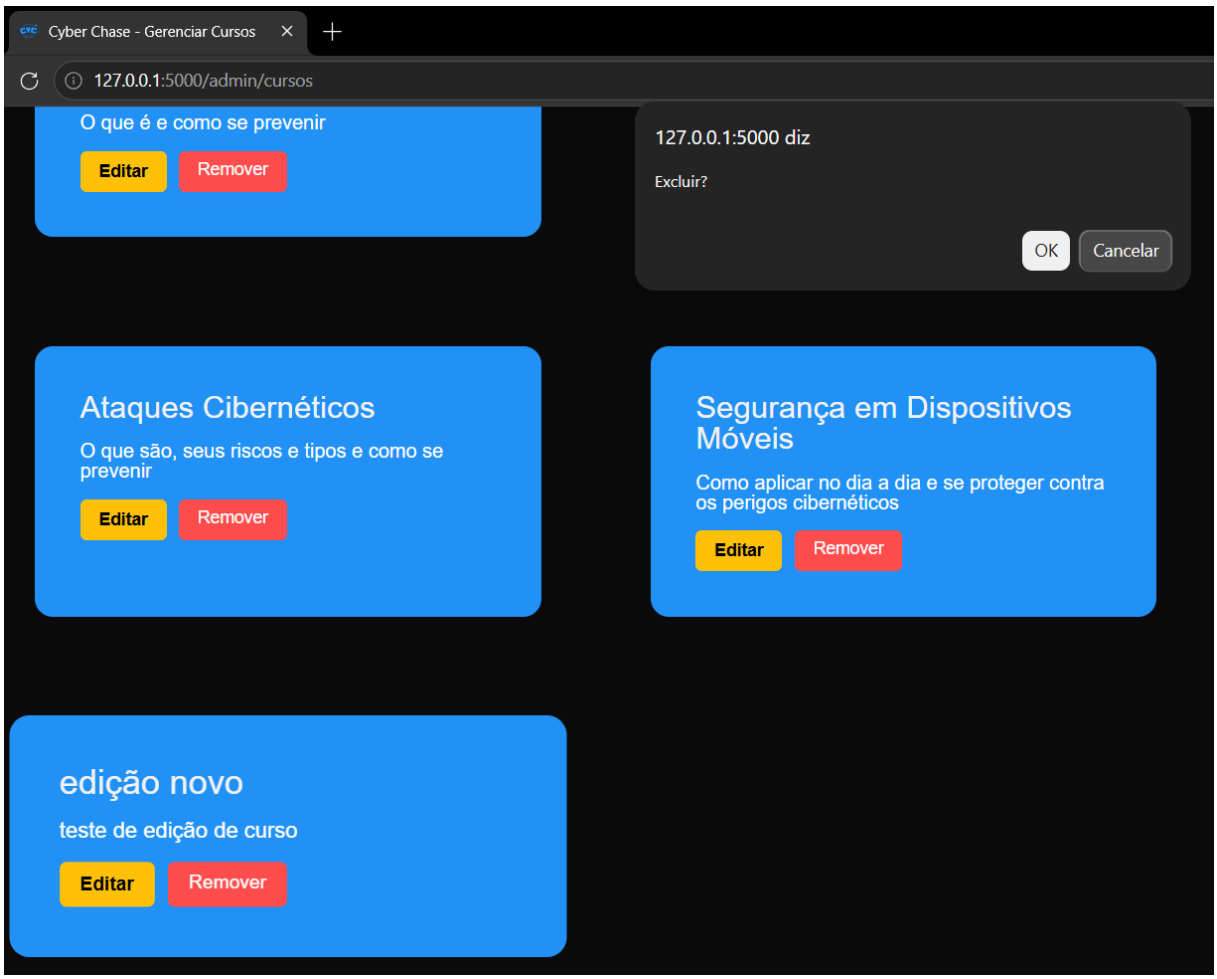
Figura 52: curso editado com sucesso



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

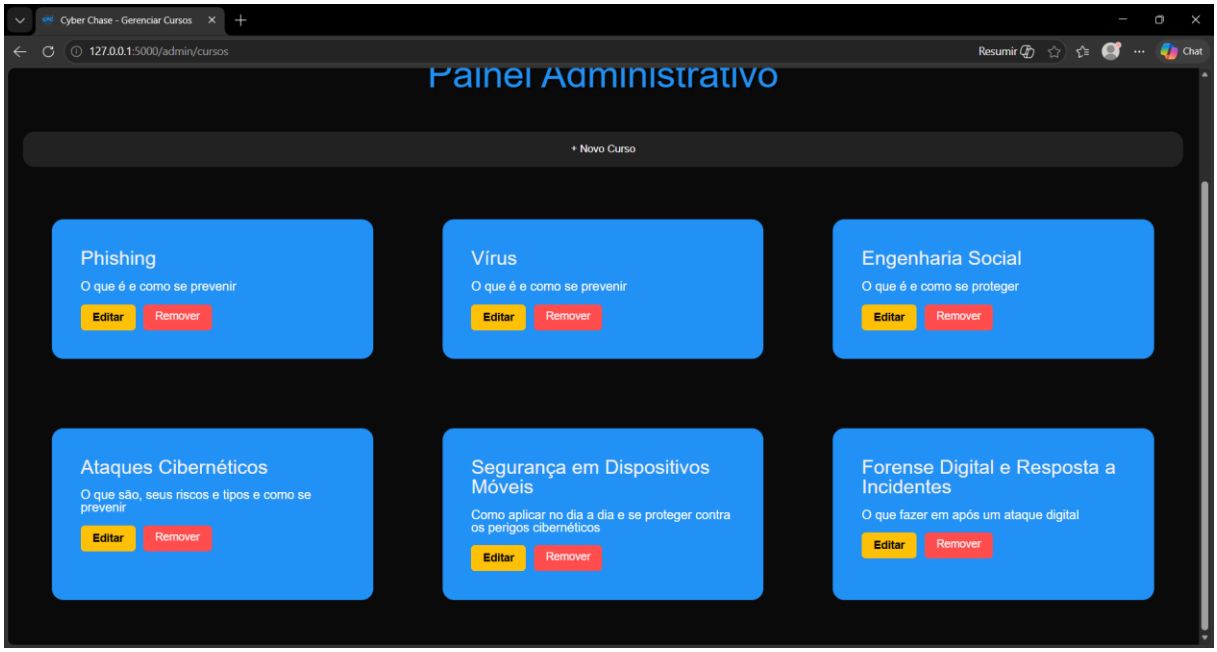
10. Ao excluir um curso, o mesmo ocorre com sucesso, onde é enviada ao administrador uma pergunta de confirmação da exclusão e, em caso positivo, o mesmo ocorre e o curso deixa de aparecer na listagem de cursos, conforme imagens a seguir:

Figura 53: confirmação de exclusão de curso



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Figura 54: captura de tela com todos os cursos



Fonte: captura de tela da plataforma desenvolvida para o presente projeto (2026)

Na captura, nota-se que o curso excluído não está mais presente, o que demonstra sucesso na ação.

3.2 Análise dos resultados obtidos

Dado os resultados analisados dos testes realizados após o desenvolvimento da plataforma, nota-se que o sistema realiza com sucesso as funcionalidades parametrizadas. Nisto, envolve-se as validações de login, o cadastro de novos usuários, o CRUD do administrador, a aplicação do Machine Learning e da Inteligência Artificial no chatbot e na pergunta validadora do curso e a geração do certificado para os usuários. Isto demonstra um desenvolvimento implementado com sucesso de um Front-End consumindo integralmente o Back-End e o Banco de Dados.

CONCLUSÃO

A plataforma Cyber Chase foi desenvolvida com o objetivo de oferecer um ambiente de aprendizado de educação cibernética, conscientizando sobre segurança digital de maneira interativa.

No desenvolvimento deste trabalho, foi integrado diferentes tecnologias, como o framework Flask, banco de dados MySQL e técnicas de inteligência artificial e

machine learning, resultando em um sistema funcional e coerente com os objetivos propostos.

A utilização de uma metodologia ágil adaptada, baseada nos princípios do Scrum, contribuiu com a organização das etapas do projeto, com o desenvolvimento incremental das funcionalidades e a realização de testes com resultados positivos para as funcionalidades desenvolvidas dentro do contexto proposto.

Um dos pontos fortes do sistema foi a implementação de machine learning e inteligência artificial em duas seções diferentes da plataforma, sendo uma delas o chatbot, e outra na validação automática das respostas dos usuários para as perguntas dos cursos. O processamento de linguagem natural e similaridade textual se mostrou eficaz dentro do contexto educacional proposto, pois nota-se que contribuiu para uma experiência mais dinâmica e interativa.

Portanto, os resultados obtidos indicam que a plataforma atende aos objetivos estabelecidos inicialmente, oferecendo uma solução funcional para apoio ao ensino de segurança cibernética. O sistema desenvolvido demonstra viabilidade técnica e potencial de expansão, podendo futuramente incorporar novos recursos, como relacionamento de tabelas no banco de dados, novas integrações e gamificação.

Sendo assim, podemos concluir que o desenvolvimento da Cyber Chase contribuiu significativamente para uma aplicação prática dos conhecimentos adquiridos ao longo do curso, abrangendo áreas como desenvolvimento web, banco de dados e inteligência artificial, além de reforçar conceitos de Segurança Cibernética abordados ao longo do curso e a importância da educação digital para todos.

REFERÊNCIAS BIBLIOGRÁFICAS

AGÊNCIA BRASILEIRA DE INTELIGÊNCIA (ABIN). **Engenharia social: guia para proteção de conhecimentos sensíveis. Programa Nacional de Proteção do Conhecimento Sensível (PNPC), 2021.** Disponível em: https://www.gov.br/abin/pt-br/institucional/acoes-e-programas/PNPC/boaspraticas/cartilha-engenharia-social-guia-para-protecao-de-conhecimentos-sensiveis. Acesso em: 2 maio 2026.

AGÊNCIA BRASILEIRA DE INTELIGÊNCIA (ABIN). **Programa Nacional de Proteção do Conhecimento Sensível (PNPC).** Disponível em: https://www.gov.br/abin/pt-br/institucional/acoes-e-programas/PNPC/o-que-e-1. Acesso em: 2 maio 2026.

AIQON. **Maiores ataques cibernéticos da história**. Disponível em: https://aiqon.com.br/blog/maiores-ataques-ciberneticos-historia. Acesso em: 2 maio 2026.

Ascenty. **Ataques cibernéticos: conheça os principais tipos e como evitá-los**. Disponível em: https://ascenty.com/blog/artigos/ataques-ciberneticos. Acesso em: 2 maio 2026.

CNN BRASIL. **Brasil lidera América Latina em ataques cibernéticos.** YouTube, 2025. Disponível em: https://youtu.be/vgzv2kzqrb0?si=bPfjWJBHW8bLTGsT. Acesso em: 2 maio 2026.

CNN BRASIL. **Golpes digitais crescem em 2025: entenda como se defender.** CNN Brasil, 2025. Disponível em: https://share.google/2euvRzuXQW72WXDvf. Acesso em: 2 maio 2026.

CROWDSTRIKE. **2026 Global Threat Report: Executive Summary.** CrowdStrike, 2026. Disponível em: https://go.crowdstrike.com/2026-global-threat-report-exec-summary-pt-br. Acesso em: 2 maio 2026.

FORTINET. **DFIR: Digital Forensics and Incident Response**. Fortinet Cyber Glossary, 2024. Disponível em: https://www.fortinet.com/br/resources/cyberglossary/dfir. Acesso em: 2 maio 2026.

IBDFAM – INSTITUTO BRASILEIRO DE DIREITO DE FAMÍLIA. **Brasil registra aumento nas denúncias de crimes cibernéticos; confira o levantamento.** Belo Horizonte: IBDFAM, 2025. Disponível em: https://share.google/4ClNWoOhwe5ZurJiu. Acesso em: 2 maio 2026.

IBM. **Phishing: o que é e como funciona**. IBM Brasil, 2024. Disponível em: https://www.ibm.com/br-pt/think/topics/phishing. Acesso em: 2 maio 2026.

IBM. **What is a Cyberattack?** IBM Think, 2025. Disponível em: https://www.ibm.com/br-pt/think/topics/cyber-attack. Acesso em: 2 maio 2026.

IBM. **What is DFIR (Digital Forensics and Incident Response)?** IBM Think, 2024. Disponível em: https://www.ibm.com/think/topics/dfir. Acesso em: 2 maio 2026.

IBSEC. **Ataques cibernéticos globais crescem 44%, diz estudo de 2025.** IBSEC, 2025. Disponível em: https://ibsec.com.br/ataques-ciberneticos-globais-crescem-44-diz-estudo-de-2025/. Acesso em: 2 maio 2026.

JESUS, Leticia. **tcc**. GitHub. Disponível em: https://github.com/leticiaoj/tcc. Acesso em: 11 maio 2026.

KASPERSKY. **Malware & Computer Virus Facts & FAQs**. Kaspersky Resource Center, 2024. Disponível em: https://www.kaspersky.com.br/resource-center/threats/computer-viruses-and-malware-facts-and-faqs. Acesso em: 2 maio 2026.

KASPERSKY. **Computer Viruses and Malicious Software**. Kaspersky Resource Center, 2024. Disponível em: https://www.kaspersky.com/resource-center/threats/viruses-malware. Acesso em: 2 maio 2026.

Ministério Público do Estado de Mato Grosso. **Mundo enfrenta forte aumento de ciberataques: quase 50% a mais em ciberameaças e acima de 120% em ransomware**. Portal CAO, 2024. Disponível em: https://www.mpmt.mp.br/portalcao/news/1217/159607/mundo-enfrenta-forte-aumento-de-ciberataques-quase-50-a-mais-em-ciberameacas-e-acima-de-120-em-ransomware/138. Acesso em: 2 maio 2026.

SENADO FEDERAL. **Golpes virtuais aumentam e não fazem distinção de idade.** Senado Notícias, Brasília, 2025. Disponível em: https://share.google/gpTDNWBD9uW5l9k1V. Acesso em: 2 maio 2026.

PF indicia cinco pessoas por dia por crimes cibernéticos, um aumento que chega a 221% em dois anos. *[S. l.: s. n.]*, 2025. Disponível em: https://share.google/1G81Y4FdNGvKLJoe8. Acesso em: 2 maio 2026.

Stefanini Brasil. **7 tipos de ataques cibernéticos e como se proteger deles**. Acesso em: 11 maio 2026.